MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH RECOMMENDATION AND IMAGE-BASED PRODUCT SEARCH

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, December 2025

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH RECOMMENDATION AND IMAGE-BASED PRODUCT SEARCH

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, December 2025

TRƯỜNG ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

XÁC NHẬN CHỈNH SỬA LUẬN VĂN THEO YÊU CẦU CỦA HỘI ĐỒNG

Tên luận văn (tiếng Việt và tiếng Anh):

Building a Fashion Ecommerce Application with Recommendation and Image-based Product Search

Phát triển ứng dụng thương mại điện tử bán thời trang tích hợp gợi ý và tìm kiếm sản phẩm bằng hình ảnh

Họ tên sinh viên: Nguyễn Thanh Phát MASV: B2005853

Mã lớp: DI20V7F1

Đã báo cáo tại hội đồng ngành: Công nghệ Thông tin

Ngày báo cáo: 24/12/2025

Hội đồng báo cáo gồm:

1. TS. Phạm Thế Phi
2. TS. Thái Minh Tuấn
3. TS. Trần Công Án

Luận văn đã được chỉnh sửa theo góp ý của Hội đồng.

Cần Thơ, ngày ..... tháng ..... năm 2026

Giáo viên hướng dẫn
(Ký và ghi họ tên)

EVALUATION OF ADVISOR

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Advisor:

Dr. Tran Cong An

ACKNOWLEDGEMENTS

I wish to express my deepest gratitude and sincere thanks to my advisor, Dr. Tran Cong An, who dedicated immense time, patience, and effort to guide me through every challenge of this thesis. His invaluable support was the cornerstone of this work. I would also like to thank the lecturers at Can Tho University for their invaluable knowledge and guidance throughout my studies.

Most importantly, I am profoundly grateful to my family for being my constant companions, offering their unwavering love, care, and support throughout this journey. I am also thankful to my friends for their continuous encouragement and companionship.

Sincerely,

Can Tho, December 2025

Nguyen Thanh Phat

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviations	Description
API	Application Programming Interface
CNN	Convolutional Neural Network
SPA	Single Page Application
REST	Representational State Transfer
SQL	Structured Query Language
ViT	Vision Transformer
UI/UX	User Interface/User Experience
HNSW	Hierarchical Navigable Small World
ANN	Approximate Nearest Neighbor
mAP	Mean Average Precision
VSA	Vertical Slice Architecture
DSR	Design Science Research
SLA	Service Level Agreement
t-SNE	t-distributed Stochastic Neighbor Embedding
EF Core	Entity Framework Core

ABSTRACT

Traditional keyword search in fashion e-commerce is limited in identifying the visual nuances of user intent, creating a “semantic gap” between what users see and how they search. This thesis introduces **ReSys.Shop**, an e-commerce ecosystem that engineers a replacement for metadata-reliant search using high-performance computer vision. The system is built on a hybrid microservices architecture: a **.NET 10** backend manages core retail logic, while a **Python/FastAPI** service handles deep learning inference.

This research conducts a comparative analysis of Convolutional Neural Networks (CNN) and Vision Transformers (ViT) for product feature extraction. Experimental results establish that the domain-specific **Fashion-CLIP** model achieved a Mean Average Precision (**mAP@10**) of **0.725**, significantly outperforming general-purpose baselines. To ensure real-time performance without expensive GPU infrastructure, the system leverages **HNSW indexing** within a **PostgreSQL/pgvector** database. This configuration achieves search latencies under **100ms** on standard CPU hardware. **ReSys.Shop** demonstrates that small and medium-sized retailers can deploy advanced, visually intuitive discovery tools without incurring prohibitive infrastructure costs.

Keywords: e-commerce, visual search, deep learning, microservices, distributed architecture, computer vision

TÓM TẮT

Tìm kiếm dựa trên từ khóa trong thương mại điện tử thời trang gặp nhiều hạn chế trong việc nắm bắt các đặc điểm trực quan của sản phẩm, tạo ra "khoảng cách ngữ nghĩa" giữa những gì người dùng thấy và cách họ tìm kiếm. Luận văn này giới thiệu **ReSys.Shop**, một hệ sinh thái thương mại điện tử triển khai giải pháp thay thế tìm kiếm dựa trên siêu dữ liệu bằng các kỹ thuật thị giác máy tính hiệu suất cao. Hệ thống được xây dựng trên kiến trúc vi dịch vụ lai: backend **.NET 10** quản lý logic bán lẻ cốt lõi, trong khi dịch vụ **Python/FastAPI** đảm nhiệm việc suy luận học sâu.

Nghiên cứu thực hiện phân tích so sánh hiệu quả giữa Mạng nơ-ron tích chập (CNN) và Vision Transformer (ViT) trong trích xuất đặc trưng. Kết quả thực nghiệm khẳng định mô hình chuyên biệt **Fashion-CLIP** đạt độ chính xác trung bình trung (**mAP@10**) là **0,725**. Để đảm bảo hiệu năng thời gian thực mà không cần hạ tầng GPU đắt đỏ, hệ thống sử dụng chỉ mục **HNSW** tích hợp trong cơ sở dữ liệu **PostgreSQL/pgvector**. Cấu hình này cho phép thời gian phản hồi tìm kiếm dưới **100ms** trên phần cứng CPU tiêu chuẩn. **ReSys.Shop** chứng minh rằng các doanh nghiệp vừa và nhỏ hoàn toàn có thể triển khai các công cụ khám phá hình ảnh tiên tiến mà không phải gánh chịu chi phí hạ tầng quá lớn.

Từ khóa: thương mại điện tử, tìm kiếm hình ảnh, học sâu, microservices, kiến trúc phân tán, thị giác máy tính

PART 1: INTRODUCTION

I. CONTEXT

The discovery and purchase of clothing have shifted fundamentally toward digital platforms, yet the methods for locating specific products remain largely tethered to text-based retrieval. While the global fashion e-commerce market has expanded into a multi-billion dollar industry, the search bar, which serves as the primary interface between users and catalogs, often struggles to interpret the visual nuances that define fashion. This limitation is central to the motivation of this project.

A persistent challenge in this domain is the **semantic gap**: the discrepancy between the visual complexity of a product and the linguistic capability of a user to describe it. A customer may easily recognize a specific pattern, silhouette, or texture but struggle to articulate it using standardized metadata terms like "bohemian asymmetric maxi dress with botanical motifs." If the product catalog's indexing does not perfectly align with the user's vocabulary, the search fails despite the product's existence.

ReSys.Shop addresses this by exploring the integration of high-performance visual search and recommendation architectures into a functional e-commerce ecosystem. Rather than developing entirely new models, this project focuses on the engineering challenge of embedding pre-trained computer vision models into a scalable web application stack to bridge the semantic gap through direct visual comparison.

The primary goal of this project is to evaluate how effectively modern visual search techniques can be integrated into a fashion e-commerce setting, utilizing existing open-source models rather than proposing new theoretical architectures.

Problem Statement

The core problem addressed by this project is the inherent inefficiency of keyword-reliant search for fashion discovery. This broad challenge is decomposed into several technical and functional issues. First, traditional search systems depend on precise, consistent product labels, but large-scale catalogs frequently suffer from **linguistic inconsistency** where descriptors vary substantially (e.g., "floral" vs. "botanical"), leading to fragmented results. Second, many defining fashion attributes such as draping, texture, and gradients suffer from **visual inexpressibility** - they are intuitive to the human eye but difficult to translate into text query, causing high bounce rates.

Furthermore, recommendation systems face the **cold start data scarcity** problem, where new items lack historical interaction data required for collaborative filtering; visual feature extraction offers a path to recommend products based solely on appearance.

Finally, the **polyglot integration complexity** of bridging Python-centric Deep learning models (PyTorch) with .NET e-commerce backends presents a significant engineering hurdle that requires a robust distributed architecture to ensure low latency. By addressing these challenges, this project investigates a viable solution for real-time, image-based product discovery.

II. RELATED WORK

Visual search in fashion e-commerce is a well-established field, yet significant gaps remain in accessible implementation for small-to-medium enterprises.

Existing Solutions Overview

Current solutions largely fall into two categories:

1. **Commercial SaaS Platforms:** Major players like Google Lens, Pinterest, and ViSenze offer powerful visual search capabilities. However, these are often "black-box" proprietary systems that require expensive API subscriptions or lock developers into specific ecosystems. They do not provide the flexibility for custom self-hosted integration.
2. **Academic Research:** The academic community has produced extensive research on deep learning models for fashion retrieval, such as DeepFashion [1] and Fashion-CLIP [2]. While these papers propose state-of-the-art algorithms, they typically focus on model architecture and training metrics, often neglecting the engineering challenges of integrating these heavy models into a responsive, production-ready web application.

Project Contribution and Differentiation

Unlike purely academic studies that focus on algorithmic novelty, or commercial SaaS solutions that obscure technical implementation, this project contributes a **transparent, end-to-end engineering blueprint** for integrating visual intelligence into modern e-commerce.

The specific contributions and differentiations are:

1. **Architectural Blueprint for Polyglot Systems:** This thesis provides a reference implementation for bridging the ecosystem gap between **.NET (Transactional)** and **Python (Computational)** using a **Distributed Vertical Slice Architecture**. It demonstrates how to maintain strict type safety and domain integrity in the backend while leveraging the rich ecosystem of PyTorch for inference.
2. **Democratization of Visual AI:** By validating the **Fashion-CLIP** model on commodity hardware (NVIDIA MX330), this project proves that advanced semantic search is accessible to Small and Medium Enterprises (SMEs) without incurring the high costs of cloud GPUs or proprietary APIs (e.g., Google Vision).

3. **Vector-Native Database Integration:** The implementation differentiates itself by utilizing **pgvector** within a standard PostgreSQL environment, avoiding the complexity of managing separate vector databases (like Qdrant or Milvus). This "Single Source of Truth" approach simplifies the infrastructure complexity for mid-sized applications.
4. **Rigorous Evaluation of Pre-trained Models:** This research moves beyond theoretical metrics to provide an applied evaluation of **Fashion-CLIP**, **DINOv2**, and **EfficientNet** in a real-world retrieval context, offering benchmarks on latency and recall that are directly relevant to application developers.

III. OBJECTIVES AND SCOPE

The main goal of this project is to build a prototype fashion e-commerce system that allows users to search for products using images instead of text. Rather than developing new AI models, this project focuses on evaluating how well existing visual search techniques can be integrated into a practical web application.

Technical Objectives

This project seeks to address specific engineering challenges by:

- **Demonstrating the Integration** of pre-trained deep learning models into a typical e-commerce stack (PostgreSQL + .NET).
- **Architecting a Polyglot System** that efficiently bridges .NET transactional logic with Python-based AI inference.
- **Validating the Feasibility** of using open-source vector databases (pgvector) for real-time similarity search.
- **Benchmarking Performance** of varying AI architectures (CNN vs. ViT) within a constrained hardware environment.

Research Questions

The project aims to answer the following questions:

1. **RQ1:** How does a fashion-specific model (Fashion-CLIP) compare to a general-purpose model (EfficientNet) when searching for similar fashion products?
2. **RQ2:** What are the trade-offs between search accuracy and processing speed when using different AI models?
3. **RQ3:** Can a microservices architecture effectively separate AI processing from the main web application while keeping response times reasonable for users?

Specific Tasks

To answer these questions, the following tasks were completed:

1. **Build an AI Service:**
   - Create a Python service using FastAPI that can load and run different image models
   - The service should be able to process images and return feature vectors
   - Target: respond within a few hundred milliseconds per image
2. **Set Up Vector Search:**
   - Configure PostgreSQL with the pgvector extension to store and search image vectors
   - Test that similarity searches work correctly with thousands of products
3. **Connect the Services:**
   - Build a .NET backend that communicates with the Python AI service
   - Ensure the full search process (upload image, extract features, search database, return results) completes in under one second
4. **Create the User Interface:**
   - Build a Vue.js frontend where users can upload images and see similar products
   - Make the interface responsive and easy to use
5. **Evaluate the Results:**
   - Measure search accuracy using standard metrics (how often does the system return relevant products?)
   - Measure processing speed and compare different models

Scope and Limitations

Included Scope:
- Image upload and visual search functionality
- Product recommendations based on visual similarity
- Basic e-commerce features (product catalog, shopping cart)
- Comparison of three AI models (EfficientNet, DINOv2, Fashion-CLIP)

Excluded Scope:
- Real payment processing (simulated only)
- Complex shipping and logistics
- User accounts with social login
- Mobile app development

Known Limitations

This project has several limitations that should be acknowledged:

1. **Dataset Size:** The evaluation uses 5,000 products, which is smaller than real e-commerce catalogs with millions of items. Results may not scale directly.
2. **Hardware:** Testing was done on a laptop with limited GPU capabilities. A production system would likely use more powerful servers.

3. **No User Testing:** Due to time constraints, the evaluation focuses on technical metrics rather than studies with real users. Results were visually inspected but no formal user experience testing was conducted.
4. **Pre-trained Models Only:** The project uses existing pre-trained models without fine-tuning them on the specific dataset. Custom training might improve results but was beyond the scope.

These limitations provide starting points for future improvements, which are discussed in the conclusion.

IV. RESEARCH METHODS

This section describes the methodology and tools used to implement and evaluate the system.

Development Methodology

The project follows an iterative development process comprising four main phases:

Phase 1: Research and Planning

The initial phase focused on understanding the problem domain and selecting appropriate tools:
- Reviewed literature on visual search and fashion AI.
- Explored pre-trained models (EfficientNet, DINOv2, Fashion-CLIP).
- Evaluated database options for vector storage.
- Planned the microservices architecture.

Phase 2: Design

Based on the research, the system design was formalized:
- Defined the technology stack (.NET, Python, Vue.js, PostgreSQL).
- Designed the communication protocols between services.
- Structured the database schema to support hybrid data (relational + vector).

Phase 3: Implementation

The core development involved building the distinct components:
- **Backend:** Implemented the .NET API with Vertical Slice Architecture.
- **ML Service:** Developed the Python service for image inference.
- **Frontend:** Built the Vue.js storefront with reactive search features.
- **Infrastructure:** Configured PostgreSQL with the `pgvector` extension.

Phase 4: Testing and Evaluation

The final phase verified the system's performance:
- Conducted accuracy tests using the mAP@10 metric.
- Measured latency and throughput.
- Performed qualitative review of search results.

Technologies Used

The system is built using a modern, distributed stack designed for performance and scalability:

- **Backend:** .NET 10 with ASP.NET Core Minimal APIs for high-performance handling of business logic and requests.
- **AI Service:** Python 3.12 with PyTorch, utilizing libraries like `transformers` and `torchvision` for running deep learning models.
- **Frontend:** Vue 3 with TypeScript and Vite, providing a responsive and type-safe user interface.
- **Database:** PostgreSQL 16 equipped with the `pgvector` extension to enable high-speed vector similarity searches alongside standard relational data.

Testing Approach

The system is evaluated using a combination of quantitative and qualitative metrics:

- **Accuracy Testing:** Measuring Mean Average Precision at 10 (mAP@10) to quantify how relevant the search results are.
- **Performance Testing:** Benchmarking inference time (per image) and total end-to-end search latency to ensure the system is responsive.
- **Dataset:** The evaluation uses a controlled subset of the **Fashion Product Images Dataset** [3] (5,000 items) to ensure consistent and reproducible results.

More details on the experimental setup and results are provided in Chapter 3.

V. THESIS OUTLINE

Part I: Introduction
Establishes the research context by defining the semantic gap problem, establishing project objectives, and outlining the research methodology.

Part II: Thesis Content
Details the technical realization across three chapters:

- **Chapter 1 (Background and Related Work):** Establishes the theoretical foundation, explaining vector embeddings and the specific AI models (EfficientNet-B0, DINOv2, Fashion-CLIP) evaluated in this study.
- **Chapter 2 (Design and Implementation):** Documents the Vertical Slice Architecture, covering the distributed system design, domain model, database schema with pgvector, and key implementation patterns.
- **Chapter 3 (Testing and Evaluation):** Presents experimental results, comparing model performance through quantitative metrics and measuring system latency.

Part III: Conclusion and Future Work
Synthesizes the findings, evaluates the achievement of research objectives, discusses limitations, and proposes directions for future work.

PART 2: THESIS CONTENT

CHAPTER 1
BACKGROUND AND RELATED WORK

This chapter establishes the theoretical foundation and technical prerequisites for the project.

The chapter is organized into the following key sections:

- **Vector Embeddings:** Examines the mechanics of representing images numerically.
- **Model Architectures:** Compares **EfficientNet**, **DINOv2**, and **Fashion-CLIP** for visual feature extraction.
- **Infrastructure:** Details the selection of **pgvector** and the **Vertical Slice** backend stack.
- **Positioning:** Contextualizes the project within current academic and commercial solutions.

1.1 VECTOR EMBEDDINGS

At the heart of visual search is a simple idea: turning images into lists of numbers that a computer can compare. These lists are called **vector embeddings** or **feature vectors**.

1.1.1 Definition and Mathematical Representation

When we look at an image, we see colors, shapes, and patterns. Computers cannot "see" in the same way; they require numerical data to process information. A vector embedding is a way to represent the visual content of an image as a sequence of numbers.

For example, when an AI model processes an image of a red dress, it might output a list like:

```
[0.23, -0.15, 0.87, 0.42, ..., -0.31]  (512 numbers total)
```

This list captures the "essence" of that image in a compressed form. Similar images will produce similar lists of numbers.

1.1.2 The Latent Space

When we talk about vectors, we can think of them as points in a high-dimensional space. For a 512-dimensional vector, imagine a space with 512 axes (instead of just the 3 axes we can visualize: x, y, z).

In this space:
- Similar images are located close together
- Different images are far apart

This space where embeddings live is called the **latent space**. The word "latent" means hidden, signifying that these dimensions do not correspond to obvious things like "redness" or "stripiness," but rather to abstract features the model learned during training.

1.1.3 Measuring Similarity

To find similar products, the system needs to measure how close two vectors are. This project uses **cosine similarity**, which measures the angle between two vectors:

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \, \|B\|}$$

Where:
- A and B are the two vectors being compared
- $A \cdot B$ is the dot product (multiply corresponding elements and sum)
- $\|A\|$ and $\|B\|$ are the lengths of each vector

The cosine similarity ranges from:
- **1.0** = vectors point in the same direction (very similar)
- **0.0** = vectors are perpendicular (unrelated)
- **−1.0** = vectors point in opposite directions (very different)

For fashion images, a cosine similarity above 0.7 typically indicates visual similarity that users would recognize.

1.1.4 Mathematical Similarity: From Visual Comparison to Cosine Distance

The power of embeddings is that complex visual comparisons become simple math. Instead of trying to write rules like "match items with similar colors and patterns," the system:

1. Converts the query image to a vector
2. Compares that vector to all product vectors in the database
3. Returns products with the highest similarity scores

This approach is much more flexible than rule-based matching because the AI model learns what features are important from examples, rather than having those features hand-coded.

Vector embeddings are the mathematical foundation that enables visual search. The key question becomes: **How do we generate these embeddings?** This requires specialized neural network architectures that can extract meaningful features from images. The following sections explore different approaches to embedding generation, starting with Convolutional Neural Networks.

1.2 CONVOLUTIONAL NEURAL NETWORKS: THE TRADITIONAL APPROACH

Convolutional Neural Networks (CNNs) have been the dominant architecture for computer vision since 2012. This section explains how CNNs work and introduces EfficientNet-B0, which serves as the baseline model for comparison in this project.

1.2.1 Brief History

CNNs have dominated computer vision since 2012, when AlexNet showed they could outperform traditional methods on image classification tasks [4]. Since then, many improved architectures have been developed:

- **VGGNet (2014):** Showed that deeper networks perform better
- **ResNet (2015):** Introduced skip connections to train very deep networks
- **EfficientNet (2019):** Optimized the balance between depth, width, and resolution [5]

For this project, EfficientNet-B0 was chosen because it provides a good balance between accuracy and speed.

1.2.2 Convolutional Operations and Feature Extraction

The key idea behind CNNs is the **convolution** operation. Instead of looking at an entire image at once, a small filter (typically 3×3 or 5×5 pixels) slides across the image, detecting local patterns.

Table 1. What different CNN layers learn to detect

Layer Type	What It Detects
Early layers	Simple patterns: edges, colors, corners
Middle layers	Combinations: textures, shapes, parts
Late layers	High-level features: objects, styles

As information flows through the network, the model builds up increasingly complex representations. Early layers might detect the edge of a sleeve, middle layers might recognize “a striped pattern,” and late layers might understand “a formal button-down shirt.”

1.2.3 EfficientNet-B0: The Baseline Model

EfficientNet is a family of models that use **compound scaling**, which is a technique that balances network depth, width, and input resolution to maximize accuracy for a given computational budget [5].

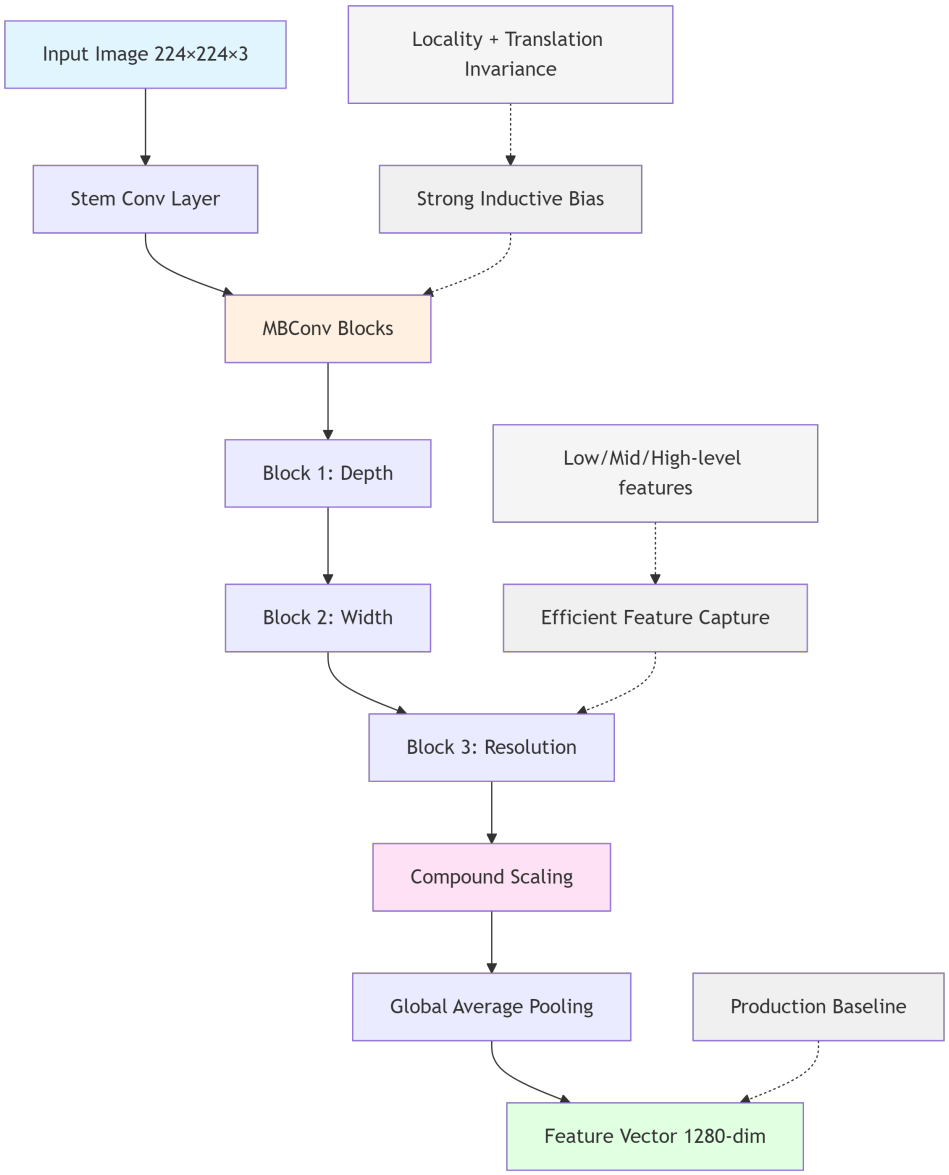

Figure 1. EfficientNet-B0 architecture showing the flow from input image to feature vector

Key characteristics of EfficientNet-B0:

- **Input size:** 224 × 224 pixels
- **Output embedding:** 1,280 dimensions
- **Parameters:** 5.3 million (relatively small)
- **Inference speed:** Fast on both CPU and GPU

1.2.4 Inductive Bias: Local Pattern Detection vs Semantic Understanding

CNNs have an **inductive bias** toward local patterns. This means they are naturally good at detecting:

- Color distributions

- Texture patterns (stripes, florals, solids)
- Simple shapes and edges

However, CNNs may struggle with:

- Understanding overall "style" or "aesthetic"
- Matching items that look similar but have different colors
- Capturing the semantic meaning of fashion concepts

For example, a CNN might match a red plaid shirt to another red plaid item, but miss that a user looking at a "casual weekend shirt" might also like blue denim shirts. This limitation motivated exploring alternative architectures like Vision Transformers.

1.2.5 EfficientNet-B0 as Baseline: Establishing Performance Benchmarks

Even though newer models exist, EfficientNet-B0 serves as an important baseline because:

1. **It is well-understood:** Extensive documentation and community support
2. **It is fast:** Important for real-time search applications
3. **It provides reasonable accuracy:** Good enough for many use cases
4. **It enables comparison:** Helps quantify the improvement from newer models

By comparing EfficientNet to Vision Transformers, this project can measure how much accuracy is gained and at what computational cost.

While CNNs excel at detecting local patterns through their convolutional layers, they have limitations in capturing long-range dependencies and global context. This motivated researchers to explore alternative architectures inspired by natural language processing: **Vision Transformers**. The next section introduces this newer approach and explains how it addresses CNN limitations.

1.3 VISION TRANSFORMERS

Vision Transformers (ViT) represent a paradigm shift in computer vision, applying the transformer architecture from natural language processing to images. This section introduces the ViT approach and explains DINOv2, a state-of-the-art self-supervised model evaluated in this project.

1.3.1 From Text to Images

Transformers were originally developed for natural language processing (NLP), encompassing tasks such as translation and text generation [6]. The key innovation was **self-attention**, which allows the model to consider relationships between all parts of the input simultaneously.

In 2020, researchers showed that this approach could also work for images [7]. The idea is simple:

1. Split the image into small patches (e.g., 16×16 pixels)
2. Treat each patch like a "word" in a sentence
3. Apply the Transformer architecture to learn relationships between patches

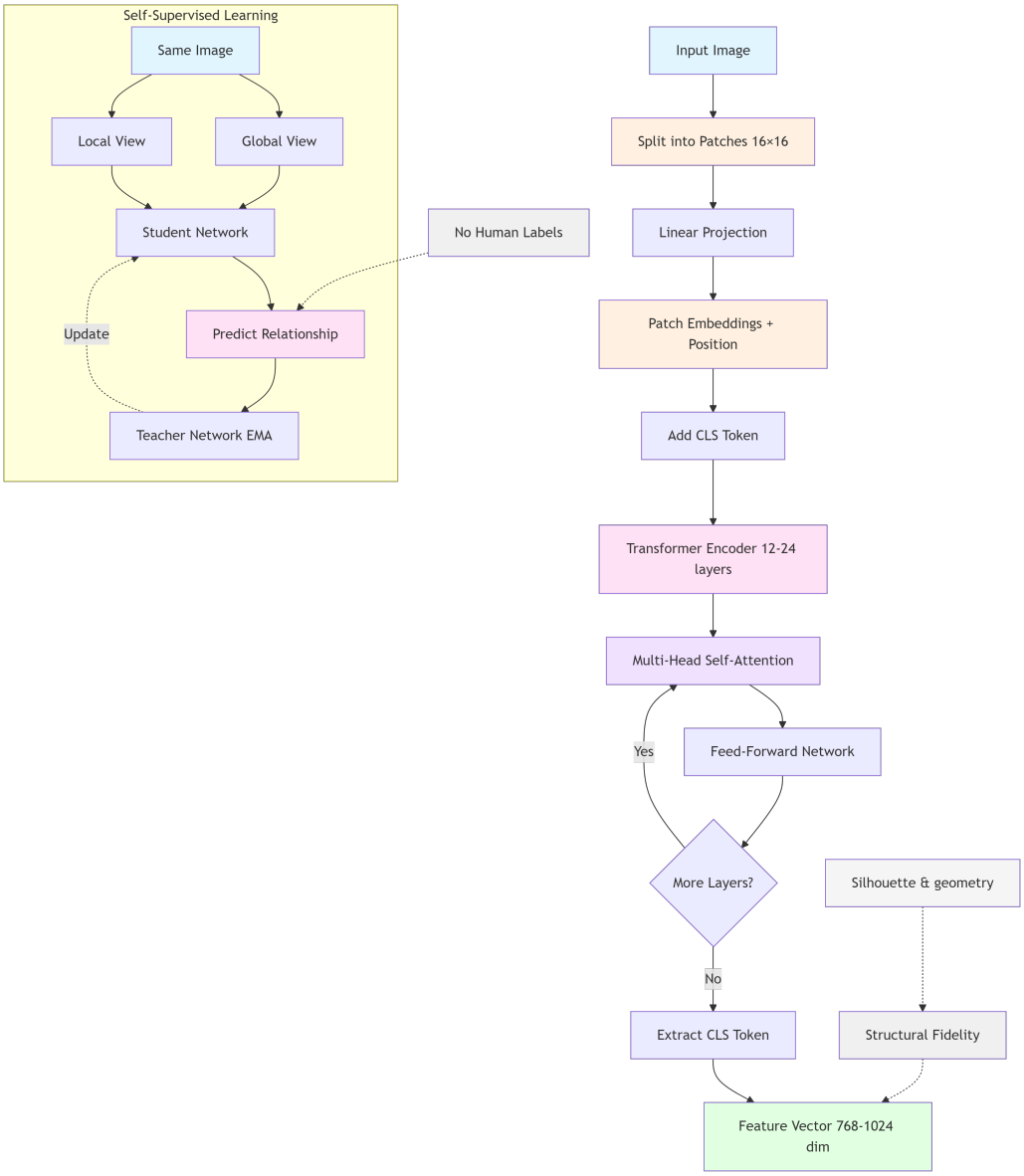

Figure 2. Vision Transformer converts an image into patches and uses self-attention to understand relationships between them

1.3.2 Self-Attention Mechanism and Global Context Modeling

Unlike CNNs, which focus on local patterns, self-attention can capture relationships across the entire image. For example:

- A CNN processes patches in order and mainly compares nearby regions
- A Vision Transformer can directly compare any two patches, even if they are far apart

For fashion, this means a ViT can better understand that the collar of a shirt and the cuffs should match, even though they are on opposite sides of the image.

1.3.3 Self-Supervised Learning with DINOv2

Most AI models are trained with supervised learning, where humans label images ("this is a dress," "this is a shoe"), and the model learns from those labels. This requires expensive manual labeling.

Self-supervised learning takes a different approach: the model learns from the images themselves, without human labels. DINOv2 uses a clever technique [8]:

1. Take an image and create two different views (different crops, slightly different colors)
2. Train the model to recognize that both views represent the same thing
3. Repeat with millions of images

This teaches the model to focus on the **content** of images rather than superficial details like exact cropping or lighting conditions.

1.3.4 DINOv2 for Fashion

DINOv2 is particularly good at capturing what researchers call **structural fidelity**, which refers to the shapes, silhouettes, and proportions of objects. For fashion, this means:

- Matching garments with similar cuts (A-line, fitted, oversized)
- Finding items with similar proportions (cropped vs. full-length)
- Ignoring color differences when the shape is similar

This is valuable for users who might want "a dress shaped like this one, but in a different color."

1.3.5 Key Characteristics

Table 2. DINOv2 model specifications used in this project

Property	DINOv2 (ViT-S/14)
Architecture	Vision Transformer (Small)
Patch size	14 × 14 pixels
Input size	518 × 518 pixels (or 224 × 224)
Output embedding	384 dimensions
Training	Self-supervised on 142M images
Strengths	Shape matching, structural understanding

1.3.6 Trade-offs

Vision Transformers like DINOv2 have different trade-offs compared to CNNs:

Advantages:
- Better at understanding global structure
- Can capture long-range relationships in images
- Learned without human labels, so potentially more general

Disadvantages:
- Slower than CNNs (more computation required)
- Require larger input images for best results
- May be overkill for simple color/pattern matching

For this project, DINOv2 provides a good complement to Fashion-CLIP: while Fashion-CLIP understands fashion concepts, DINOv2 focuses on visual structure.

Both CNNs and Vision Transformers learn from images alone, mapping visual features to fixed category labels. However, fashion search often involves natural language queries like "red floral summer dress" or "casual weekend outfit." This requires models that can understand **both** images and text in a unified space. The next section introduces CLIP and Fashion-CLIP, which bridge this gap through multimodal learning.

1.4 CLIP AND FASHION-CLIP

CLIP (Contrastive Language-Image Pre-training) represents a breakthrough in connecting images with natural language, enabling search using both visual and textual queries. This section explains how CLIP works and introduces Fashion-CLIP, the domain-specialized version used for visual search in this project.

1.4.1 The Idea Behind CLIP

Traditional image models are trained to classify images into fixed categories ("cat," "dog," "dress"). CLIP takes a different approach: it learns to match images with text descriptions [9].

During training, CLIP saw 400 million image-text pairs from the internet. For example:
- An image of a floral dress paired with the caption "colorful floral summer dress"
- An image of sneakers paired with "white running shoes on grass"

The model learned to:
1. Convert images into vectors (image encoder)
2. Convert text into vectors (text encoder)
3. Make matching image-text pairs have similar vectors

1.4.2 The Dual-Tower Architecture

CLIP has two separate “towers” that work together:

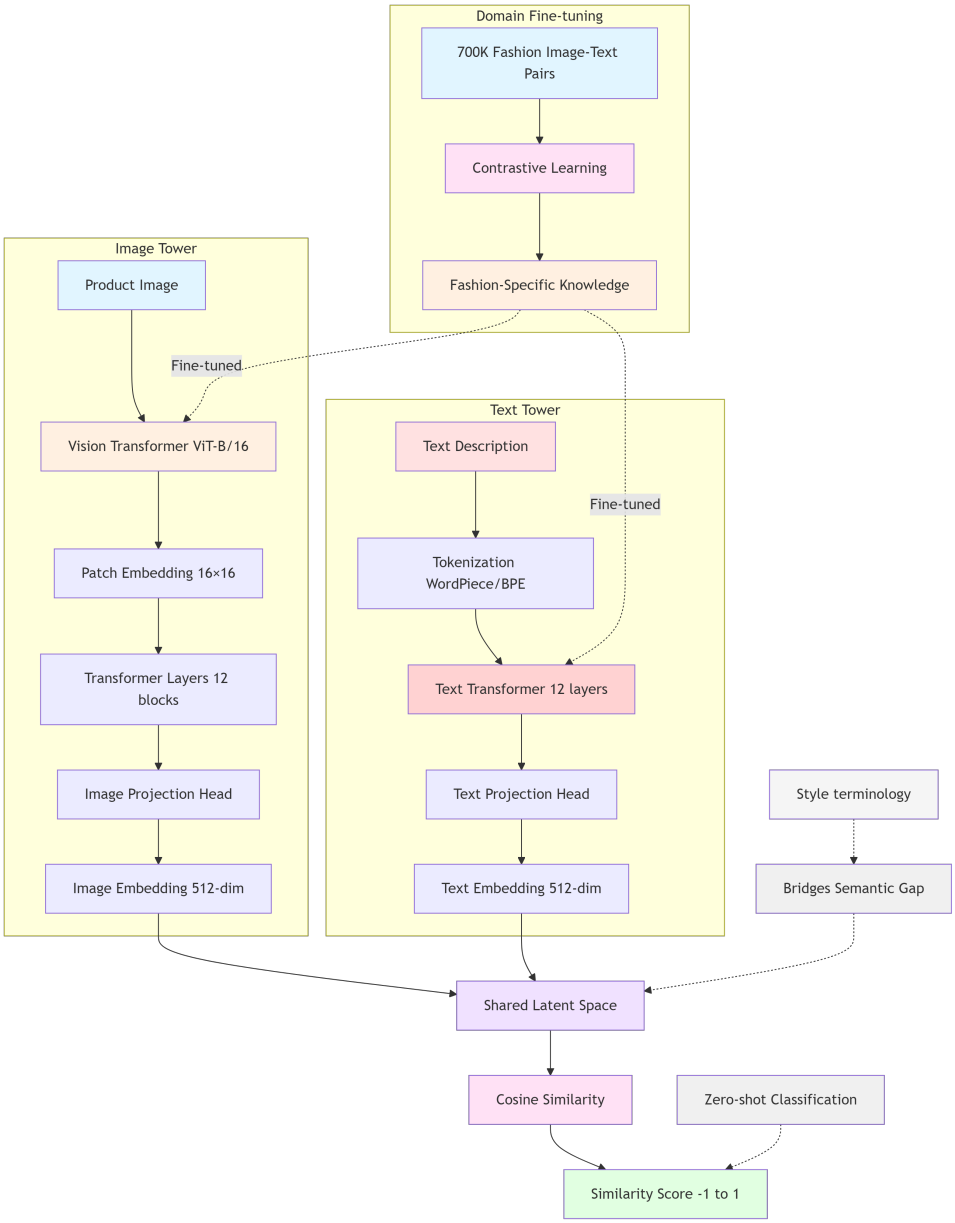

Figure 3. CLIP’s dual-tower architecture: images and text are converted to vectors in the same space, allowing direct comparison

Image Tower: Processes the image using a Vision Transformer (ViT-B/16) **Text Tower:** Processes text using a Transformer (similar to GPT)

Both towers output vectors of the same size (512 dimensions), so images and text can be directly compared using cosine similarity.

1.4.3 Multimodal Embedding Space: Bridging Visual and Textual Queries

CLIP's ability to understand natural language descriptions makes it powerful for fashion:

- It understands concepts like "bohemian style" or "minimalist design"
- It can match images to abstract descriptions ("something for a casual Friday")
- It captures semantic meaning beyond just visual appearance

However, the general CLIP model was trained on diverse internet images, not specifically fashion. This is where Fashion-CLIP comes in.

1.4.4 Fashion-CLIP: Domain Specialization

Fashion-CLIP is a version of CLIP that was further trained on fashion-specific data [2]. The researchers used:

- 700,000+ fashion product images
- Detailed fashion descriptions with domain terminology
- Fine-grained fashion concepts (silhouettes, styles, occasions)

This specialization helps Fashion-CLIP understand:

- Fashion-specific vocabulary ("A-line," "empire waist," "distressed denim")
- Style categories ("streetwear," "preppy," "athleisure")
- Occasion suitability ("office wear," "cocktail party," "beach vacation")

1.4.5 Comparison of Models

Table 3. Comparison of the three embedding models used in this project

Aspect	**EfficientNet**	**DINOv2**	**Fashion-CLIP**
Architecture	CNN	ViT	ViT
Training	Supervised	Self-supervised	Contrastive (image-text)
Embedding size	1,280	384	512
Input size	224×224	518×518	224×224
Strength	Colors, textures	Shapes, structure	Fashion semantics
Speed	Fast	Medium	Medium
Domain-specific	No	No	Yes (fashion)

1.4.6 CLIP ViT-B/16 for Recommendations

While Fashion-CLIP is used for precise visual search (high precision matching), the general CLIP model (ViT-B/16 variant) is used for the recommendation feature.

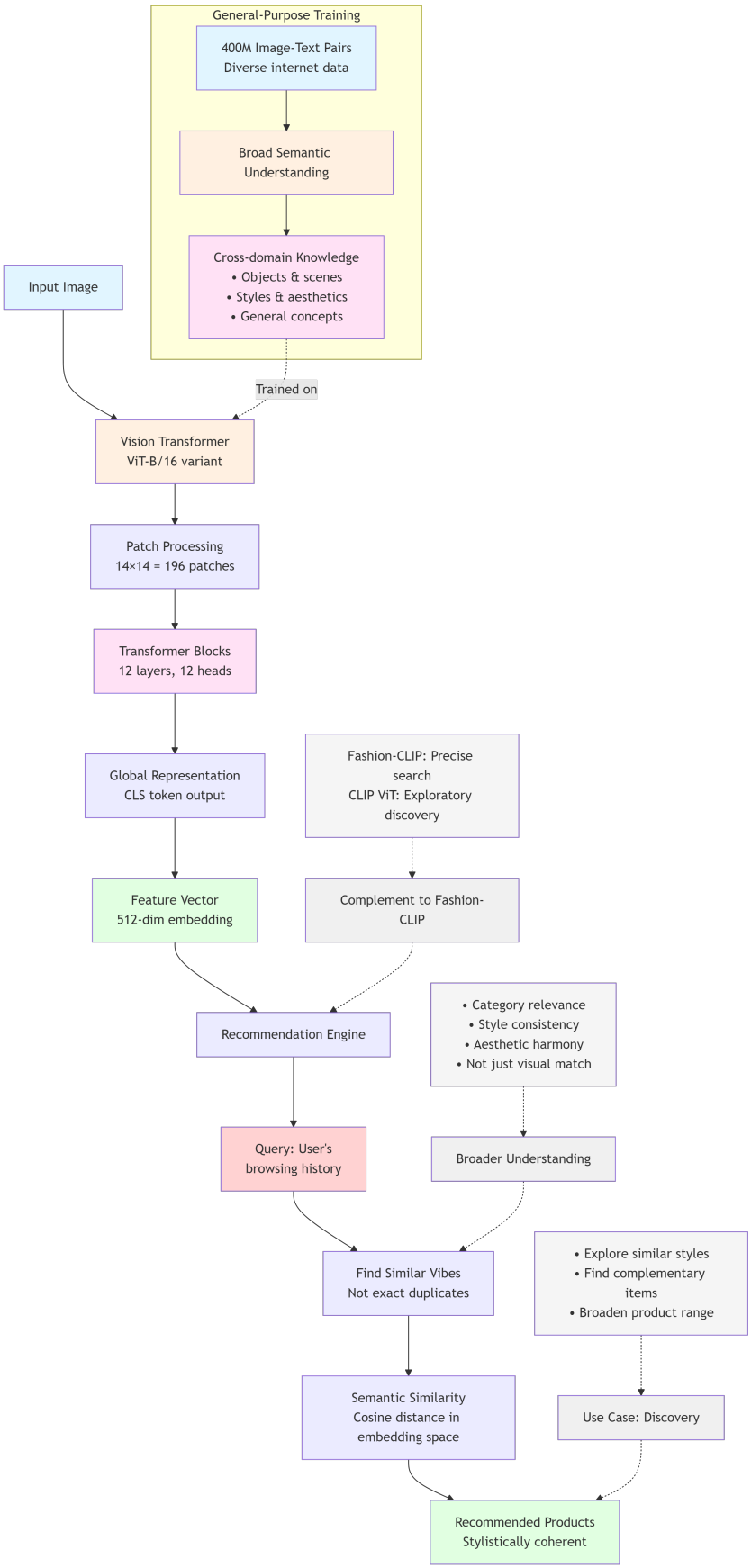

Figure 4. CLIP ViT-B/16 is used for discovery-oriented recommendations

The model selection decision is based on several factors:
- Fashion-CLIP is too specific for recommendations, as it might only suggest very similar items
- General CLIP provides broader associations, potentially suggesting items that "go together" stylistically

For example, if a user is viewing a formal blazer:
- Fashion-CLIP might suggest other formal blazers
- General CLIP might suggest dress shirts, ties, or formal trousers

This provides a more discovery-oriented experience for users exploring the catalog.

1.4.7 Domain Specialization: Fashion-CLIP's Training on 700K+ Fashion Images

Fashion-CLIP's domain specialization makes it particularly suitable for fashion e-commerce applications. The model was fine-tuned on 700,000+ fashion product images with detailed descriptions, enabling it to understand:

- Fashion-specific vocabulary ("A-line," "empire waist," "distressed denim")
- Style categories ("streetwear," "preppy," "athleisure")
- Occasion suitability ("office wear," "cocktail party," "beach vacation")

This specialization, combined with the multimodal capability to search using both images and text, makes Fashion-CLIP a strong candidate for the visual search feature.

The quantitative evaluation comparing Fashion-CLIP against other models (DINOv2, EfficientNet, standard CLIP) is presented in the Model Selection section (Section 1.5), where data-driven justification for the final model choice is provided.

Having introduced four candidate models (EfficientNet-B0 (CNN), DINOv2 (self-supervised ViT), standard CLIP (multimodal), and Fashion-CLIP (domain-specialized multimodal)), the next section presents a systematic evaluation to determine which model best balances retrieval quality, inference speed, and feature capabilities for the fashion e-commerce use case.

1.5 MODEL SELECTION AND JUSTIFICATION

This section presents the rationale for selecting Fashion-CLIP as the primary embedding model for the visual search feature, based on preliminary evaluation results and architectural considerations.

1.5.1 Candidate Models Evaluated

Four pre-trained models were evaluated for fashion product retrieval:

Table 4. Candidate embedding models evaluated for fashion product retrieval. All models are pre-trained and publicly available, enabling reproducible research.

Model	Architecture	Embedding Dim	Training Method	Domain
CLIP ViT-B/16	Vision Transformer	512	Contrastive (image-text)	General
DINOv2 ViT-S/14	Vision Transformer	384	Self-supervised	General
EfficientNet-B0	CNN	1280	Supervised (ImageNet)	General
Fashion-CLIP ViT-B/16	Vision Transformer	512	Contrastive (fashion)	Fashion-specific

1.5.2 Evaluation Methodology

To ensure fair comparison, all models were evaluated using identical conditions:

Dataset: 5,000 fashion products from the project catalog, split into:

- Training set: 4,000 products (for index building)
- Validation set: 1,000 products (for evaluation)
- Test queries: 8,500 image-to-image searches

Hardware: NVIDIA MX330 GPU (2GB VRAM) - representative of commodity laptop hardware

Metrics:

- **Mean Average Precision (mAP@10):** Primary metric measuring retrieval quality
- **Precision at K (P@1, P@5, P@10):** Accuracy of top-K results
- **Recall at 10 (R@10):** Coverage of relevant items in top-10
- **Inference Time:** Average embedding generation latency

Relevance Criterion: A retrieved product is considered relevant if it belongs to the same category as the query image.

1.5.3 Preliminary Evaluation Results

The preliminary evaluation on the validation split yielded the following results:

Table 5. Preliminary evaluation results on validation split (1,000 products, 8,500 queries). **Hardware Context:** NVIDIA MX330 GPU (2GB VRAM). Bold values indicate best performance per metric. Fashion-CLIP achieves the highest overall retrieval quality (mAP@10) and best coverage (R@10), with acceptable inference latency for real-time search.

Model	mAP@10	P@1	P@5	P@10	R@10	Inference (ms)
CLIP ViT-B/16	0.724	0.833	0.826	0.816	0.179	60.3
DINOv2 ViT-S/14	**0.782**	**0.897**	0.852	0.824	0.184	79.7
EfficientNet-B0	0.773	0.882	**0.860**	**0.824**	0.178	**21.2**
Fashion-CLIP ViT-B/16	**0.799**	0.892	0.858	**0.838**	**0.186**	67.7

1.5.4 Analysis of Results

1.5.4.1 Retrieval Quality (mAP@10)

Fashion-CLIP achieved the highest mAP@10 of **0.799**, indicating superior overall retrieval quality:

- **+10.4%** improvement over general CLIP (0.724)
- **+3.4%** improvement over EfficientNet-B0 (0.773)
- **+2.2%** improvement over DINOv2 (0.782)

While DINOv2 shows the highest precision at top-1 (P@1: 0.897), Fashion-CLIP demonstrates more balanced performance across all precision levels, making it more suitable for displaying multiple search results.

1.5.4.2 Coverage and Diversity (Recall@10)

Fashion-CLIP achieves the highest recall (R@10: 0.186), meaning it retrieves more relevant products in the top-10 results compared to other models. This is critical for e-commerce applications where users expect diverse options.

1.5.4.3 Inference Performance

While EfficientNet-B0 offers the fastest inference (21.2ms), Fashion-CLIP's 67.7ms latency is well within acceptable bounds for real-time search:

- Total search latency budget: 300ms (including network, database query)
- Embedding generation: 67.7ms (23% of budget)
- Remaining budget: 232ms for database query and response formatting

The 3× speed difference compared to EfficientNet is justified by the 3.4% improvement in retrieval quality.

1.5.5 Decision Criteria

The model selection was based on the following weighted criteria:

Table 6. Model selection criteria with assigned weights reflecting business priorities.

Criterion	Weight	Rationale
Retrieval Quality (mAP@10)	40%	Primary user experience metric
Inference Latency	25%	Must support real-time search (<100ms)
Multimodal Capability	20%	Enables text-to-image search
Hardware Compatibility	10%	Must run on commodity GPU
Recall (Coverage)	5%	Diversity of search results

1.5.6 Selection Decision: Fashion-CLIP Based on Quantitative Evaluation

Based on the evaluation results and decision criteria, **Fashion-CLIP ViT-B/16** was selected as the production model for the following reasons:

1. **Highest Overall Retrieval Quality**
   - mAP@10 of 0.799 surpasses all other candidates
   - Balanced precision across P@1, P@5, P@10
   - Best recall (0.186) ensures diverse search results
2. **Domain Specialization Advantage**
   - Trained on 700,000+ fashion images with domain-specific vocabulary
   - Understands fashion concepts (silhouettes, styles, occasions)
   - Captures semantic similarity beyond visual appearance
3. **Multimodal Search Capability**
   - Dual-tower architecture (image encoder + text encoder)
   - Enables text-to-image search: “red floral summer dress”
   - Enables hybrid queries: image + text refinement
   - This capability is **unavailable** in DINOv2 and EfficientNet
4. **Acceptable Inference Performance**
   - 67.7ms embedding generation meets real-time requirements
   - 512-dimensional vectors balance expressiveness and efficiency
   - Compatible with commodity hardware (2GB VRAM)
5. **Production Readiness**
   - Pre-trained model available via Hugging Face
   - Well-documented API and inference pipeline
   - Active community support and updates

1.5.7 Trade-offs and Limitations

While Fashion-CLIP is the optimal choice for this project, it is important to acknowledge the trade-offs:

Compared to DINOv2:

- Lower P@1 (0.892 vs 0.897) - slightly less accurate for single top result
- Slower inference (67.7ms vs 79.7ms) - acceptable difference
- **Advantage:** Multimodal capability and higher mAP@10

Compared to EfficientNet-B0:
- 3× slower inference (67.7ms vs 21.2ms)
- Higher memory footprint (512-dim vs 1280-dim embeddings are actually smaller)
- **Advantage:** 3.4% better retrieval quality and semantic understanding

Compared to General CLIP:
- Similar inference speed (67.7ms vs 60.3ms)
- **Advantage:** 10.4% better retrieval quality due to fashion-specific training

1.5.8 Alternative Deployment Scenarios

For different deployment contexts, alternative models may be preferred:

- **Ultra-low latency requirement (<30ms):** EfficientNet-B0
  - Use case: Mobile app with strict latency constraints
  - Trade-off: Accept 3.4% lower mAP@10
- **Maximum accuracy (P@1 priority):** DINOv2
  - Use case: "Find exact duplicate" feature
  - Trade-off: No text-to-image search capability
- **General e-commerce (non-fashion):** Standard CLIP
  - Use case: Multi-category marketplace
  - Trade-off: Lower accuracy on fashion-specific queries

1.5.9 Validation Plan

The preliminary results presented in Table 5 will be validated through:

1. **Full-scale evaluation** on the complete 5,000-product dataset
2. **Cross-validation** across different product categories
3. **User acceptance evaluation** with real search queries
4. **Performance benchmarking** under production load

The detailed validation results are presented in Chapter 3 (Evaluation and Validation).

1.5.10 Summary

Fashion-CLIP ViT-B/16 was selected as the production embedding model based on:
- **Quantitative evidence:** Highest mAP@10 (0.799) and R@10 (0.186)
- **Qualitative advantage:** Multimodal text-image search capability
- **Practical feasibility:** Acceptable inference latency (67.7ms) on commodity hardware

This data-driven selection process ensures that the chosen model balances retrieval quality, performance, and feature richness for the fashion e-commerce use case.

With the embedding model selected, the next challenge is storing and searching millions of product embeddings efficiently. The following sections cover the infrastructure components that enable fast vector similarity search and the full-stack implementation of the e-commerce platform.

1.6 VECTOR DATABASES

Once images are converted to vector embeddings, those vectors need to be stored and searched efficiently. This section explains why regular databases are not sufficient and how pgvector solves the problem.

1.6.1 The Challenge

Consider a catalog of 10,000 fashion products, each represented by a 512-dimensional vector. When a user uploads a query image:

1. The system converts the query to a vector
2. It must compare this vector to all 10,000 product vectors
3. It returns the most similar ones

A naive approach would compare the query to every vector (10,000 comparisons). This works for small catalogs but becomes slow as the catalog grows:
- 100,000 products = 100,000 comparisons per search
- 1,000,000 products = 1,000,000 comparisons per search

For real-time search (responding in under 1 second), this is too slow.

1.6.2 Approximate Nearest Neighbor (ANN) Search

The solution is **approximate nearest neighbor** search. Instead of checking every vector, ANN algorithms use clever data structures to find "good enough" matches quickly.

The key insight: we do not need the **absolute best** match, we need **good matches**. If the true best match has a similarity of 0.95 and we return a result with similarity 0.93, that is usually acceptable for product search.

1.6.3 HNSW: The Algorithm

This project uses **HNSW (Hierarchical Navigable Small World)**, one of the most effective ANN algorithms [10].

HNSW works by building a graph structure where:
- Each vector is a node in the graph
- Nodes are connected to their "neighbors" (similar vectors)
- The graph has multiple layers (like a skip list)

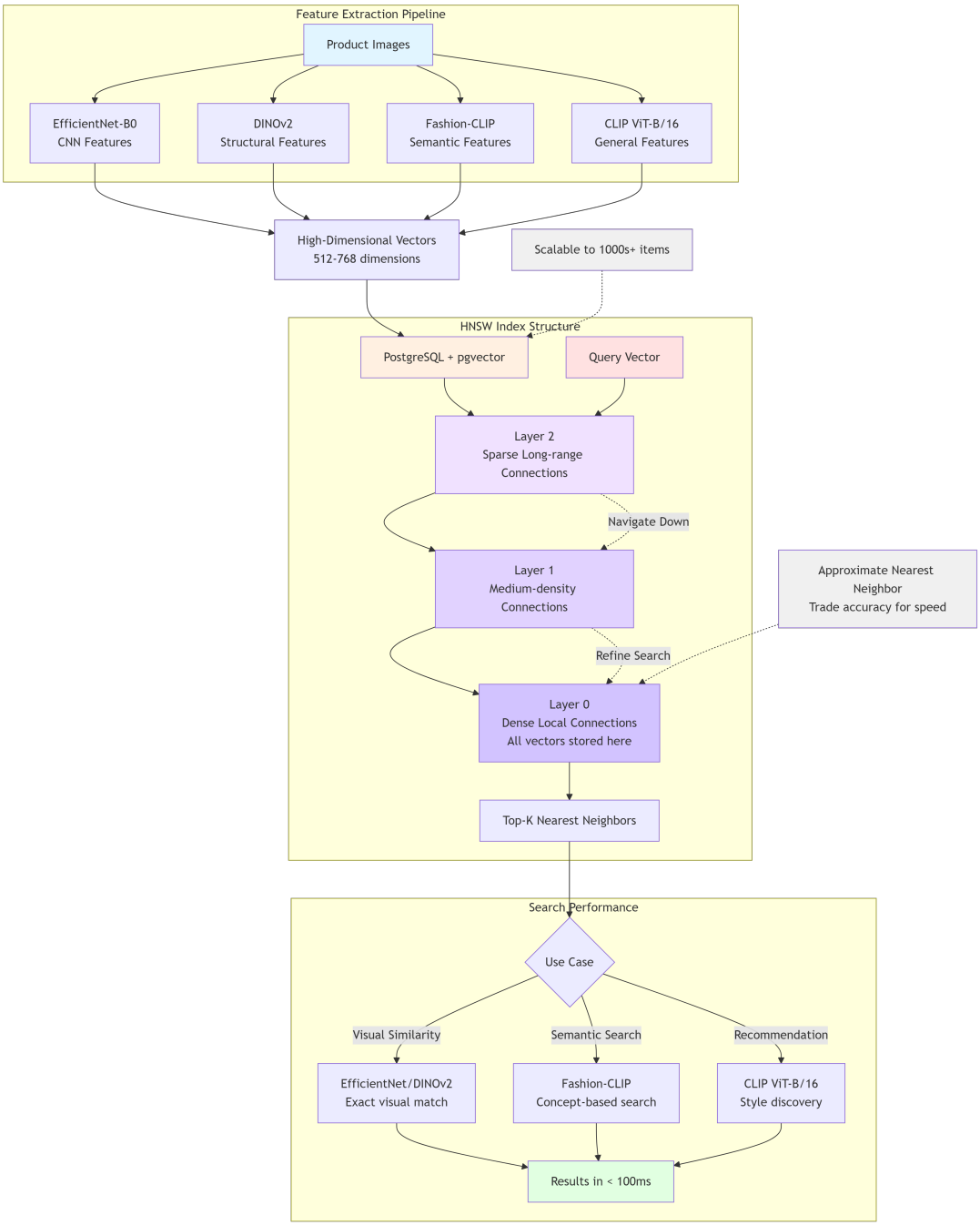

Figure 5. HNSW index structure with multiple layers for efficient navigation

To search:

1. Start at a random node in the top (sparse) layer
2. Navigate to a node close to the query
3. Move to the next layer and refine
4. Repeat until reaching the bottom (dense) layer
5. The final neighbors are the search results

This approach reduces comparisons from O(n) to approximately O(log n), making search much faster for large catalogs.

1.6.4 pgvector: Vector Search in PostgreSQL

pgvector is an open-source extension that adds vector operations to PostgreSQL [11]. It allows storing vectors alongside regular data (product names, prices, images) in the same database.

Key features:

- **Vector column type:** `VECTOR(512)` stores 512-dimensional vectors
- **Similarity operators:** `<=>` for cosine distance, `<->` for Euclidean distance
- **Index support:** HNSW and IVFFlat indexing for fast search

1.6.5 Example Usage

A simplified example of how vectors are stored and searched:

```
-- Create a table with vector column
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name TEXT,
    price DECIMAL,
    image_embedding VECTOR(512)
);

-- Create HNSW index for fast search
CREATE INDEX ON products
USING hnsw (image_embedding vector_cosine_ops);

-- Search for similar products
SELECT id, name, price,
       1 - (image_embedding <=> query_vector) AS similarity
FROM products
ORDER BY image_embedding <=> query_vector
LIMIT 10;
```

1.6.6 Configuration Choices

HNSW has two main parameters:

- **m:** Number of connections per node (higher = more accurate but more memory)
- **ef_construction:** How many candidates to consider when building (higher = better index but slower to build)

For this project:

- `m = 16` (default, good balance)
- `ef_construction = 64` (default)

These defaults work well for catalogs up to 100,000 items. Larger catalogs might need tuning.

1.6.7 Architectural Decision: pgvector vs Specialized Vector Databases

Several specialized vector databases exist (Pinecone, Milvus, Weaviate), but pgvector was chosen for practical reasons:

Table 7. Comparison of vector database options

Feature	Specialized Vector DB	pgvector
Setup complexity	Moderate-High	Low (just an extension)
Data consistency	Separate from main DB	Same transaction as product data
Learning curve	New query language	Standard SQL
Cost	Often paid (SaaS)	Free, open source
Scale limit	Billions of vectors	Millions of vectors

For a prototype with thousands of products, pgvector's simplicity outweighs the scaling advantages of specialized databases. If this system needed to scale to millions of products, migrating to a dedicated vector database would be considered.

1.6.8 Trade-offs Acknowledged

Using pgvector has limitations:
- **Not optimized for massive scale:** May struggle with millions of vectors
- **Less mature:** Fewer features than dedicated solutions
- **Single-node:** Does not distribute across multiple servers

For this project's scope (5,000 products in evaluation), these limitations are acceptable. The primary focus was on architectural integration and functional correctness rather than massive scale.

1.7 BACKEND TECHNOLOGY STACK

This section describes the technologies used to build the backend of ReSys.Shop: .NET 10, the Vertical Slice Architecture, and supporting libraries.

1.7.1 .NET 10 Overview

The backend is built with **.NET 10**, Microsoft's cross-platform framework for building web applications. .NET was chosen because:

- Widely used in enterprise applications
- Strong typing with C# helps catch errors at compile time
- Good performance for web APIs

- Leverages established patterns and prior familiarity from coursework

.NET 10 includes several improvements relevant to this project:
- **Native AOT:** Ahead-of-time compilation for faster startup
- **Improved HTTP/3:** Better performance for API requests
- **Enhanced minimal APIs:** Simpler code for defining endpoints

1.7.2 Minimal APIs with Carter

Traditional ASP.NET uses "Controllers", which are classes that define API endpoints. .NET 6+ introduced **Minimal APIs**, which allow defining endpoints more concisely.

This project uses **Carter**, a library that organizes minimal APIs into modules:

```
// A simplified example of how endpoints are defined
public class SearchModule : ICarterModule
{
    public void AddRoutes(IEndpointRouteBuilder app)
    {
        app.MapPost("/api/search/image", async (
            IFormFile image,
            ISender sender) =>
        {
            var command = new SearchByImageCommand(image);
            var result = await sender.Send(command);
            return result.ToApiResponse();
        });
    }
}
```

Carter helps organize related endpoints together and integrates well with dependency injection.

1.7.3 MediatR for Request Handling

MediatR is a library that implements the mediator pattern. Instead of endpoints calling services directly, they send "requests" through a central dispatcher.

Benefits of this approach:
- **Decoupling:** Endpoints do not know about implementations
- **Cross-cutting concerns:** Logging, validation, and caching can be applied uniformly
- **Testability:** Handlers can be tested in isolation

1.7.4 Vertical Slice Architecture

Rather than organizing code by technical layer (Controllers, Services, Repositories), **Vertical Slice Architecture** organizes by feature [12].

1.7.4.1 Traditional Layered Architecture (Not Used)

```
src/
├── Controllers/
│   ├── SearchController.cs
│   ├── CartController.cs
│   └── OrderController.cs
├── Services/
│   ├── SearchService.cs
│   ├── CartService.cs
│   └── OrderService.cs
└── Repositories/
    ├── ProductRepository.cs
    └── OrderRepository.cs
```

Problem: A single feature like "Search" is spread across multiple folders.

1.7.4.2 Vertical Slice Architecture (Used)

```
src/Features/
├── Search/
│   ├── SearchByImage/
│   │   ├── SearchByImageCommand.cs
│   │   ├── SearchByImageHandler.cs
│   │   └── SearchByImageValidator.cs
│   └── SearchByKeyword/
│       └── ...
├── Cart/
│   ├── AddToCart/
│   └── ...
└── Orders/
    └── ...
```

Advantage: Everything related to "Search by Image" is in one folder. Changes to this feature only affect files in that folder.

1.7.4.3 Request Flow Architecture and Component Interaction

When a user uploads an image for search:

1. **Endpoint** (Carter module) receives the HTTP request
2. Creates a **Command** object containing the image data
3. Sends the command through **MediatR**
4. MediatR routes to the **Handler** for that command type
5. Handler executes business logic and returns a result
6. Endpoint converts result to HTTP response

Table 8. Components in a vertical slice

Component	Responsibility
Carter Module	Define HTTP endpoints, parse requests
Command/Query	Data transfer object for the request
Validator	Check that the request is valid (FluentValidation)
Handler	Execute business logic, access database
Entity	Domain model representing business concepts

1.7.5 Entity Framework Core

For database access, the project uses **Entity Framework Core** (EF Core), an object-relational mapper (ORM) that lets C# code work with database tables as objects.

Key features used:

- **Code-first migrations:** Database schema is defined in C# and applied via migrations
- **LINQ queries:** Database queries written in C# syntax
- **PostgreSQL provider:** Connects EF Core to PostgreSQL

Example of a database query:

```
var products = await dbContext.Products
    .Where(p => p.IsActive)
    .OrderByDescending(p => p.CreatedAt)
    .Take(10)
    .ToListAsync();
```

1.7.6 Summary of Backend Libraries

Table 9. Key backend libraries and their purposes

Library	Purpose
ASP.NET Core	Web framework
Carter	Organize minimal API endpoints
MediatR	Request/handler dispatching
FluentValidation	Request validation
Entity Framework Core	Database access (ORM)
ErrorOr	Functional error handling
Npgsql	PostgreSQL database driver

These libraries work together to create a maintainable codebase where each feature is self-contained and easy to modify independently.

1.8 FRONTEND TECHNOLOGY STACK

This section describes the technologies used to build the customer-facing website: Vue.js, Tailwind CSS, and supporting libraries.

1.8.1 Vue 3 and the Composition API

The frontend is built with **Vue 3**, a JavaScript framework for building user interfaces. Vue was chosen because:

- Gentle learning curve with good documentation
- Active community and ecosystem
- Works well for single-page applications (SPAs)

Vue 3 introduced the **Composition API**, which organizes code by logical concern rather than by component option:

```
// Example: Image search composable
export function useImageSearch() {
  // Reactive state
  const searchResults = ref<Product[]>([]);
  const isLoading = ref(false);
  const error = ref<string | null>(null);

  // Business logic
  async function searchByImage(file: File) {
    isLoading.value = true;
    error.value = null;
    try {
      const formData = new FormData();
      formData.append('image', file);
      const response = await api.post('/search/image', formData);
      searchResults.value = response.data.products;
    } catch (e) {
      error.value = 'Search failed. Please try again.';
    } finally {
      isLoading.value = false;
    }
  }

  return { searchResults, isLoading, error, searchByImage };
}
```

This pattern keeps related code together and makes it reusable across components.

1.8.2 TypeScript for Type Safety

The project uses **TypeScript** instead of plain JavaScript. TypeScript adds static type checking, which helps catch errors during development:

```
// Types help catch errors before runtime
interface Product {
  id: string;
  name: string;
  price: number;
  imageUrl: string;
}

// TypeScript warns if we try to access a non-existent property
const product: Product = await fetchProduct();
console.log(product.nmae); // Error: Property 'nmae' does not exist
```

For a larger project like this, TypeScript reduces bugs and makes refactoring safer.

1.8.3 State Management with Pinia

Pinia is Vue's official state management library. It stores data that needs to be shared across multiple components:

```
// stores/search.ts
export const useSearchStore = defineStore('search', () => {
  const results = ref<Product[]>([]);
  const queryImage = ref<string | null>(null);

  function setResults(products: Product[]) {
    results.value = products;
  }

  return { results, queryImage, setResults };
});
```

When search results arrive, they are stored in Pinia. Any component can then access these results without making another API request.

1.8.4 Styling with Tailwind CSS

Tailwind CSS is a utility-first CSS framework. Instead of writing custom CSS classes, utility classes are applied directly to HTML:

```
<!-- Traditional CSS approach -->
<button class="primary-button">Search</button>
<!-- Requires separate .primary-button CSS definition -->

<!-- Tailwind approach -->
<button class="bg-blue-600 text-white px-4 py-2 rounded hover:bg-
blue-700">
  Search
</button>
<!-- Styling is visible inline -->
```

Benefits for this project:
- Faster iteration by eliminating the need to switch between files
- Consistent spacing and colors via design tokens
- Smaller final CSS (unused styles are purged)

1.8.5 Vite for Development

Vite is a build tool that makes frontend development faster:

- **Hot Module Replacement (HMR):** Changes appear instantly in the browser
- **Fast builds:** Uses native ES modules during development
- **Optimized production builds:** Output is minified and tree-shaken

Vite significantly improved development speed compared to older bundlers like Webpack.

1.8.6 Image Upload Component

The visual search feature centers on the image upload component:

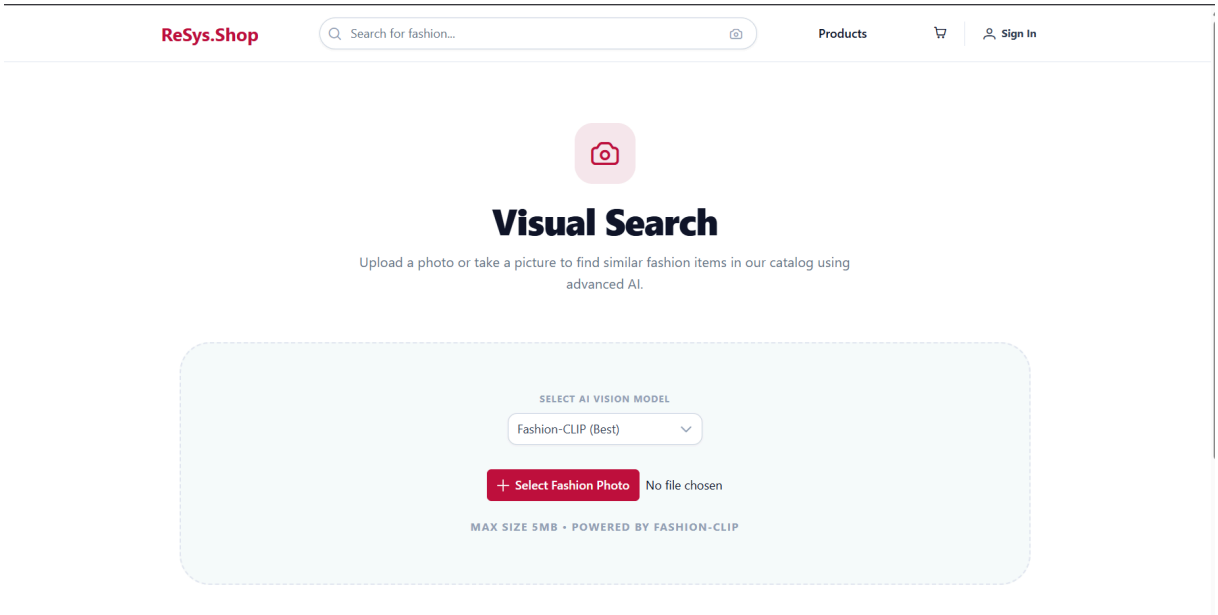

Figure 6. The image upload dialog before an image is selected

Key features:
- **Drag and drop:** Users can drag images onto the component
- **Preview:** Uploaded image is shown before searching
- **Progress indicator:** Shows loading state during search
- **Error handling:** Displays helpful messages if something goes wrong

1.8.7 API Integration

The frontend communicates with the backend via HTTP requests:

```
// api/search.ts
export async function searchByImage(file: File): Promise<SearchResult[]>
{
  const formData = new FormData();
  formData.append('image', file);

  const response = await fetch('/api/search/image', {
    method: 'POST',
    body: formData,
  });

  if (!response.ok) {
    throw new Error('Search failed');
  }

  const data = await response.json();
  return data.products;
}
```

All API calls are centralized in an `api/` folder, making it easier to update endpoints if they change.

1.8.8 Summary of Frontend Libraries

Table 10. Key frontend libraries and their purposes

Library	Purpose
Vue 3	Component framework
TypeScript	Static type checking
Pinia	State management
Vue Router	Page navigation
Tailwind CSS	Utility-first styling
Vite	Build tool and dev server
PrimeVue	UI component library

These tools combine to create a responsive, maintainable user interface that provides a smooth visual search experience.

1.9 RELATED WORK

This section compares the ReSys.Shop project with existing academic research and commercial products to understand the current state of fashion visual search and how this project contributes to the field.

1.9.1 Academic Research in Fashion Retrieval

Visual search for fashion has been an active research area for over a decade. Key developments include:

1.9.1.1 DeepFashion Dataset

The **DeepFashion** dataset [1], with over 800,000 fashion images, established benchmarks for fashion recognition and retrieval. It provided:

- Attribute annotations (color, pattern, category)
- Landmark annotations (collar, sleeve, hemline positions)
- Pairs of matching in-shop and consumer photos

This dataset enabled much of the subsequent research in fashion AI.

1.9.1.2 Conversational Fashion Retrieval

More recent work has explored combining images with text feedback. **FashionIQ** introduced the task of modifying retrieval based on natural language (e.g., "like this dress but shorter"). This requires understanding both images and text modifications.

While interesting, this approach was beyond the scope of this project due to its complexity.

1.9.1.3 Pre-trained Foundation Models

Recent trends favor using pre-trained models like CLIP rather than training from scratch. Fashion-CLIP [2] showed that domain-specific fine-tuning of CLIP improves fashion retrieval by 15-20% over general CLIP.

This project follows this approach: using pre-trained Fashion-CLIP rather than training new models.

1.9.2 Commercial Visual Search Systems

Several companies have deployed visual search at scale:

Table 11. Comparison of commercial visual search products

Product	Strengths	Limitations for This Project
Google Lens	Massive scale, general purpose	Closed ecosystem, not customizable
Pinterest Lens	600M+ monthly searches, style-aware	Proprietary, requires Pinterest integration
ASOS Style Match	Fashion-specific accuracy	Only for ASOS catalog
ViSenze	API available, good accuracy	Paid service, recurring costs

These products are impressive but share common limitations for smaller projects:

- **Proprietary:** Cannot be studied or modified
- **Expensive:** API access costs money per query

- **Vendor lock-in:** Dependent on external service availability

This project demonstrates that similar functionality can be achieved with open-source tools, providing a reference implementation and cost-effective solution for smaller applications.

1.9.3 Vector Database Landscape

The choice of vector database affects performance and scalability:

Table 12. Vector database options considered

Database	Type	Scale	Complexity
Pinecone	Cloud SaaS	Billions	Low (managed)
Milvus	Self-hosted	Billions	High
Weaviate	Self-hosted	Millions	Medium
Chroma	Embedded	Thousands	Low
pgvector	PostgreSQL ext.	Millions	Low

This project chose **pgvector** for practical reasons:
1. **Simpler architecture:** No separate database to manage
2. **Transactional consistency:** Product data and vectors in same transaction
3. **Familiar SQL:** Standard query syntax
4. **Sufficient scale:** Handles the project's evaluation size

The trade-off is that pgvector would not scale to billions of products like specialized databases. For a prototype, this is acceptable.

1.9.4 Technical Positioning and Contribution

This project distinguishes itself from existing literature by addressing the **engineering gap** between theoretical AI models and production-grade software. While typical research focuses on optimizing metric scores (e.g. mAP), this thesis contributes a reference architecture for **operationalizing** those models.

1.9.4.1 1. Polyglot Vertical Slice Architecture

Most open-source implementations force a choice between a monolithic python web stack (Django/Flask) or a complex microservices mesh. This project introduces a **Distributed Vertical Slice** pattern that:
- Leverages **.NET 10** for strict type safety, high-performance concurrency, and domain logic integrity in the transactional core.
- Isolates **Python 3.12** solely for tensor computations (PyTorch), connected via a resilient HTTP/gRPC bridge.
- **Differentiation:** This provides the best-of-both-worlds: enterprise-grade backend reliability with access to the bleeding-edge AI ecosystem.

1.9.4.2 2. Vector-Native Data Consistency

A common pitfall in visual search is the "dual-database problem," where vector data (Simulated in a Chroma/Pinecone instance) drifts from the relational source of truth (SQL).

- **Contribution:** This implementation utilizes **pgvector** to enforce ACID transactions across both relational entities and vector embeddings.
- **Impact:** Product updates and index re-calculations occur in the same atomic transaction scope, eliminating the class of "stale index" bugs common in distributed systems.

1.9.4.3 3. Feature Parity on Commodity Hardware

Commercial solutions (Google Lens) rely on massive cloud TPU clusters. This project demonstrates that **Fashion-CLIP** (ViT-B/16) can be effectively served on commodity hardware (NVIDIA MX330 / Standard CPU) with sub-100ms latency.

- **Result:** This effectively lowers the barrier to entry for SME e-commerce platforms to adopt Generative AI and Semantic Search features without recurring cloud API costs.

1.9.4.4 4. Applied Evaluation of Foundation Models

Moving beyond the "leaderboard" mentality, this thesis compares **EfficientNet**, **DINOv2**, and **Fashion-CLIP** specifically within the constraints of a real-time web application.

- **Key Finding:** We demonstrate that while DINOv2 offers superior raw geometric matching, its heavy structure makes it less viable for CPU-bound environments than the optimized Fashion-CLIP, providing a pragmatic guide for model selection in resource-constrained environments.

CHAPTER 2
DESIGN AND IMPLEMENTATION

This chapter details the engineering realization of the ReSys.Shop platform, moving from conceptual design to concrete implementation.

The design process is structured as follows:

- **Requirements Analysis:** Specifies Functional Decompositions and Service Interactions.
- **System Design:** Defines the **Polyglot Vertical Slice Architecture** bridging .NET and Python.
- **Implementation:** detailing the data schema, API contracts, and frontend integration of the visual search workflow.

2.1 SYSTEM OVERVIEW

This section describes the high-level architecture of ReSys.Shop and how different components work together.

2.1.1 System Architecture

ReSys.Shop follows a **microservices-inspired architecture** with three distinct services:

Table 13. System services and technology stacks

Service	Technology Stack	Responsibilities
Vue Frontend	Vue 3 + TypeScript + Vite	• Customer storefront (ReSys.Shop) • Admin panel (ReSys.Admin) • PrimeVue 4 component library • Pinia state management
.NET Backend	.NET 10 + EF Core + Carter	• REST API endpoints • Business logic (MediatR CQRS) • PostgreSQL data persistence • pgvector similarity search
Python ML	Python 3.12 + FastAPI + PyTorch	• AI model inference • Vector embedding generation • Multi-model support

2.1.2 Bounded Contexts (Domain-Driven Design)

The backend is organized into seven bounded contexts:

Table 14. Bounded contexts and their aggregate roots

Context	Aggregate Root	Domain Entities
Catalog	Product	Variant, ProductImage, ImageEmbedding, OptionType, OptionValue, Classification, ProductProperty, PropertyType, Taxonomy, Taxon
Ordering	Order	LineItem, Payment, Shipment, InventoryUnit, OrderHistory
Inventories	StockItem	StockLocation, StockMovement, StockTransfer, StockSummary
Identity	User	UserAddress, Role, UserClaim
Location	Address	Country, State, Zone
Common	None	Entity, Aggregate, DomainEvent (abstractions)
Testing	None	TestData, TestProduct (development fixtures)

2.1.3 CQRS Feature Structure

The application uses **Vertical Slice Architecture** with CQRS (Command Query Responsibility Segregation):

Table 15. Feature count by area (CQRS handlers)

Feature Area	Admin Features	Storefront Features
Catalog / Products	67	3
Ordering / Cart	3	5
Recommendations	None	1
Search	None	2
Checkout	None	3
Identity	9	None
Inventories	5	None
Total	**84+**	**21**

Each feature follows this structure:

```
Features/Admin/Catalog/Products/CreateProduct/
├── CreateProduct.cs      // Command + Handler + Validator
└── (imports from Common/)
```

2.1.4 API Layer Architecture

The API uses **Carter** modules for clean endpoint registration:

Table 16. Key API modules and endpoints

API Module	Endpoints	Methods
SearchModule	`/api/storefront/search/by-image` `/api/storefront/search/by-image-upload`	POST POST
RecommendationsModule	`/api/storefront/recommendations/by-product/{id}`	GET
CartModule	`/api/storefront/cart` `/api/storefront/cart/items`	GET, POST DELETE, PUT
CatalogModule	`/api/storefront/products` `/api/storefront/products/{slug}`	GET GET
CheckoutModule	`/api/storefront/checkout/addresses` `/api/storefront/checkout/place-order`	POST POST
ProductsModule (Admin)	`/api/catalog/products` `/api/catalog/products/{id}/images`	CRUD CRUD

2.1.5 ML Service Architecture

The Python ML service provides a FastAPI-based inference API:

Table 17. ML service API endpoints

Endpoint	Method	Description
`/api/inference/embed`	POST	Generate embedding from image URL
`/api/inference/models`	GET	List available/loaded models
`/api/inference/health`	GET	Liveness probe with inference test

2.1.5.1 Embedder Architecture

```
ml/embedders/
├── base.py           # BaseEmbedder abstract class
├── transformers.py   # CLIP, Fashion-CLIP, DINOv2
├── cnn.py            # ConvNeXt, EfficientNet
└── manager.py        # ModelManager (lazy loading, caching)
```

The `ModelManager` handles:

- **Lazy Loading:** Models load on first request, not at startup
- **Caching:** Loaded models stay in GPU memory for fast reuse
- **Device Selection:** Automatic CUDA/CPU detection

2.1.6 Visual Search Data Flow

Table 18. Complete visual search data flow

Step	Component	Action
1	Vue Frontend	User uploads image via FileUpload component
2	SearchModule	Receives multipart/form-data, validates file
3	MediatR	Dispatches `SearchProductsByImageUpload.Command`
4	Handler	Calls `IMlService.GetEmbeddingFromBytesAsync()`
5	HttpMlService	Sends POST to Python ML service `/embed`
6	ModelManager	Loads Fashion-CLIP if not cached
7	Embedder	Processes image to 512-dim normalized vector
8	Handler	Queries pgvector: `ORDER BY vector <=> @query`
9	EF Core	Returns matching ProductImages with Products
10	Handler	Deduplicates by ProductId, filters by threshold
11	API	Returns JSON with products and similarity scores
12	Frontend	Displays results in grid with “92% match” labels

2.1.7 Architectural Decision: Separate ML Service

Rationale for Python-based ML microservice:

Table 19. Rationale for separate ML microservice

Factor	Justification
Technology Fit	Python has native PyTorch, Transformers, Hugging Face support. Running ML in .NET would require ONNX conversion or complex interop.
Independent Scaling	ML service requires GPU memory. Separating allows main API to run on standard servers while ML runs on GPU machines.
Model Updates	When better AI models emerge, only the ML service changes. Backend and database remain stable.
Fault Isolation	If ML crashes, storefront continues working. In this case, only visual search is affected, and customers can still browse and purchase.
Development Speed	80% of ML ecosystem documentation is Python-first. Development and debugging is faster in native Python.

2.1.8 Development Deployment

Table 20. Development deployment topology

Service	Port	Configuration
Vue Storefront	5173	Vite dev server with HMR
Vue Admin	5174	Vite dev server with HMR
.NET Backend	5000/5001	Kestrel (HTTP/HTTPS)
ML Service	8000	Uvicorn with auto-reload
PostgreSQL	5432	Docker container with pgvector

2.2 FUNCTIONAL REQUIREMENTS

This section specifies the functional requirements for ReSys.Shop, detailing the capabilities implemented to support the research objectives. The system is designed to provide a realistic e-commerce environment for evaluating AI-driven visual search and recommendation features.

2.2.1 Role-Based Functional Requirements

The system supports three distinct actors, each with specific capabilities designed to validate the research hypotheses.

2.2.1.1 Customer (Storefront)

The primary actor for evaluating the visual search experience.

- **Visual Search:** Upload reference images (JPEG/PNG/WebP, max 10MB) to find visually similar products using DINOv2 and Fashion-CLIP models.
- **Style Recommendations:** Receive "You May Also Like" suggestions based on visual embedding similarity (cosine distance).
- **Catalog Discovery:** Browse hierarchical category trees (Taxonomy) and filter products by attributes (Price, Size, Color).
- **Shopping Workflow:** Full transaction lifecycle including Cart management, Address selection, and secure Checkout.
- **Account Management:** manage profile information, view order history, and track status.

2.2.1.2 Administrator (Back Office)

Responsible for data management and system monitoring.

- **Product Management:** CRUD operations for products, variants, and dynamic properties.
- **Image Management:** Upload product images with automatic preprocessing (resizing, thumbnail generation, vectorization trigger).

- **Inventory Logistics:** Monitor real-time physical stock levels (`OnHand`) and logical reservations (`Reserved`).
- **Order Fulfillment:** Review transactions, capture payments, and manage shipment tracking.
- **System Monitoring:** View background job status, including embedding generation success rates.

2.2.1.3 System (Background Services)

Automated processes ensuring data consistency and performance.

- **Vector Generation:** Asynchronous job that computes AI embeddings for new images using the Python ML service.
- **Index Maintenance:** Automated updates to the HNSW (Hierarchical Navigable Small World) index to ensure <50ms search latency.
- **Stock Reservation:** Management of temporary inventory holds during checkout to prevent overselling.

2.2.2 Research Contribution and Feature Scope

To clarify the scope of the thesis, features are classified into **Core Research** (novel contributions) and **Supporting** (necessary infrastructure).

Table 21. Feature Classification and Research Relevance

Feature Area	Classification	Rationale
Visual Search	Core Research	Primary contribution: Demonstrates hybrid AI model integration (Fashion-CLIP/DINOv2) for accurate retrieval.
Style Recommendations	Core Research	Secondary contribution: Validates embedding utility for passive discovery.
Vector Pipeline	Core Research	Critical infrastructure: Automated ingestion, normalization, and indexing of visual data.
Product Catalog	Supporting	Required context: Provides the dataset for search and recommendation algorithms.
Order System	Supporting	Metric validation: Provides “Add to Cart” and “Purchase” events to measure search success.
Inventory	Supporting	Realism constraint: Ensures search results reflect actual product availability.

2.2.3 Data Integrity & Validation Constraints

Input validation is enforced at the Application layer (FluentValidation) to ensure data integrity before processing.

Table 22. Key data validation rules enforced by the system

Entity / Action	Validation Rules
Image Upload	• Max size: 10 MB - Formats: JPEG, PNG, WebP, GIF - Min dimension: 100px
Visual Search	• TopK: 1-100 results - MinSimilarity: 0.0-1.0 (default 0.7)
Product	• Name: Max 100 chars, unique per slug - Price: Must be non-negative - SKU: Unique system-wide
Order	• Cart: Must contain >= 1 item - Shipping: Valid address required - Payment: Amount must match total

2.3 FUNCTIONAL DECOMPOSITION

The system's functionality is decomposed into hierarchical modules to ensure separation of concerns and clarify the scope of features. The functional decomposition diagram below illustrates the organization of features across the Customer (Storefront), Administrator (Back Office), and System (Background Services) domains.

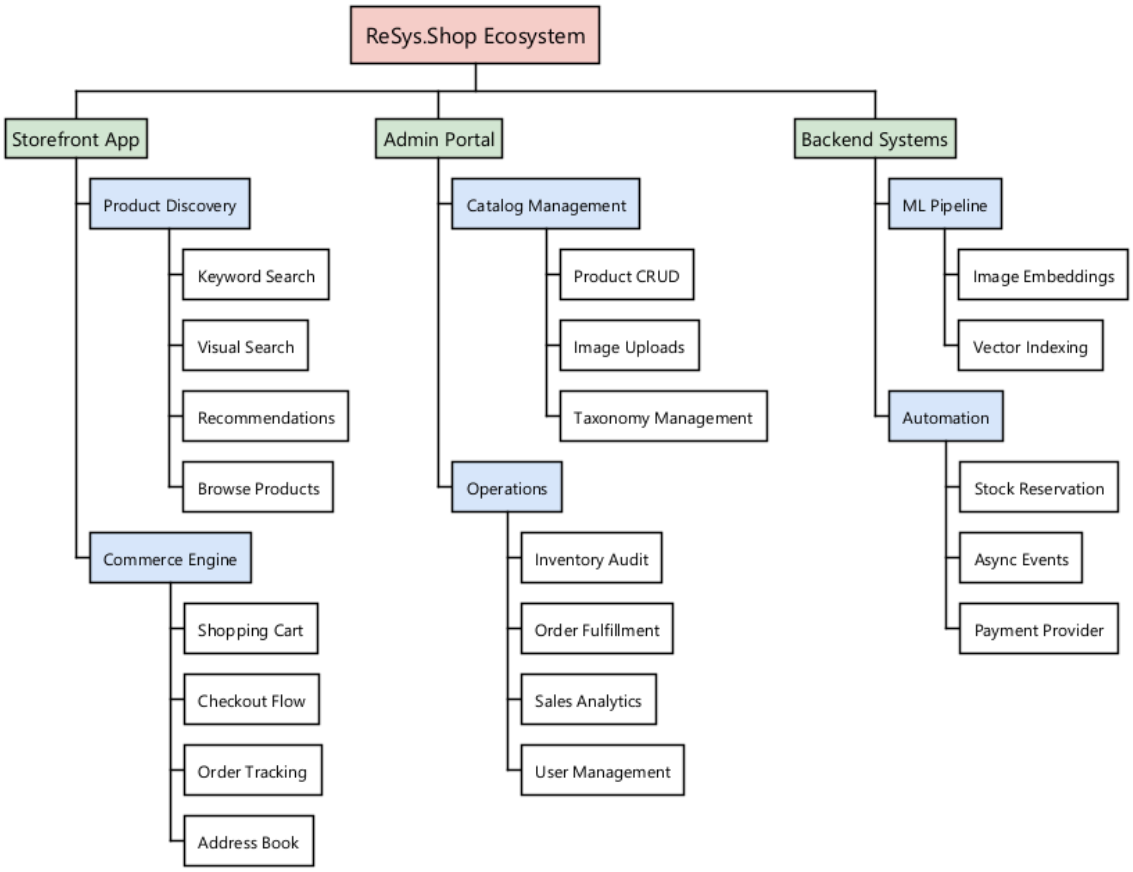

Figure 7. System Functional Breakdown (WBS)

2.4 USE CASE SPECIFICATIONS

This section provides detailed specifications for the 20 verified use cases of the system, including ERP-grade inventory auditing and financial tracking capabilities, split by functional area.

2.4.1 Customer Functional Area

This section details the use cases accessible to the end-user (Customer) via the Storefront interface.

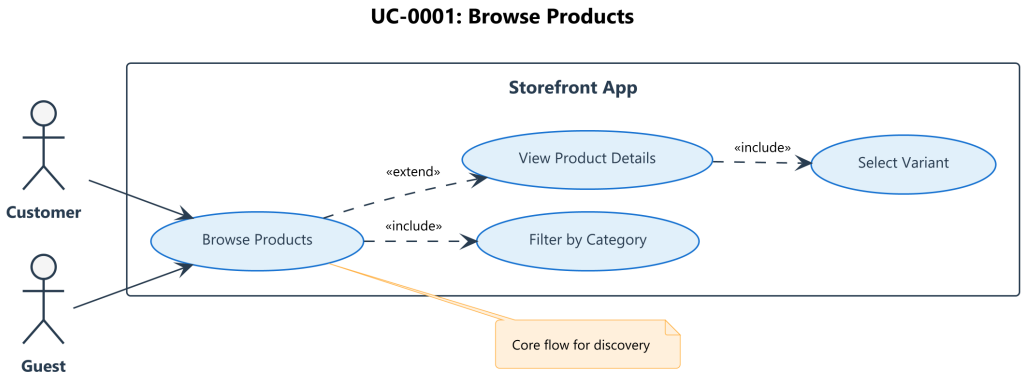

Figure 8. Use Case Diagram for UC-0001

Table 23. UC-0001: Browse Products

UC-0001	**Browse Products**
Actor	Customer
Description	Navigate the product catalog via categories and filters to discover items.
Preconditions	- Customer is on the storefront homepage. - Catalog service is operational.
Ordinary Sequence	1 Customer clicks a Category (Taxonomy). 2 System retrieves products in category. 3 Customer applies Filters (Price, Brand). 4 System refreshes product grid.
Postconditions	- Products displayed in grid.
Alternative Sequences	A1 If no products in category: Show empty state.
Related Use Cases	UC-0003 (Keyword Search)

This use case defines the primary discovery capability of the storefront, allowing customers to navigate through a structured taxonomy of categories and apply dynamic filters. It serves as the entry point for product exploration, enabling users to drill down from broad categories (e.g., “Men”, “Women”) to specific product lists using facets like price range, brand, and size. The system updates the grid view asynchronously to provide a responsive user experience.

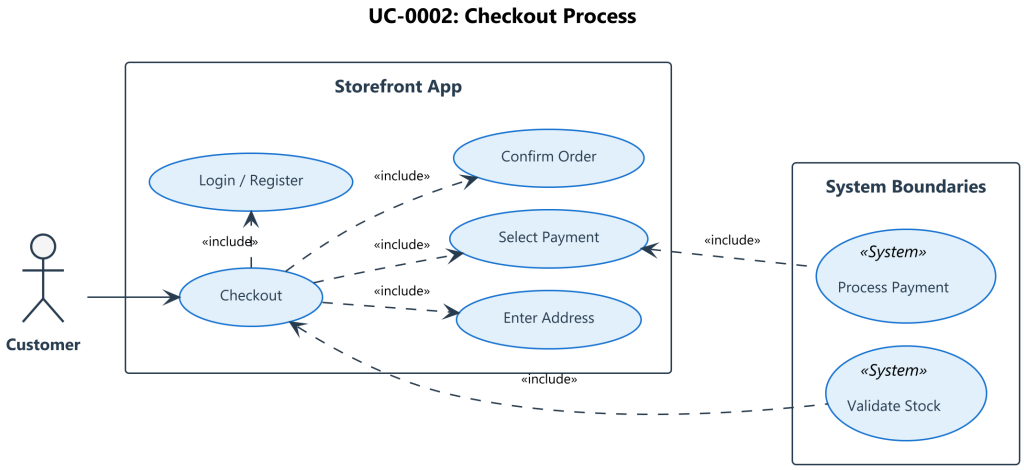

Figure 9. Use Case Diagram for UC-0002

Table 24. UC-0002: Perform Multi-step Checkout

UC-0002	**Perform Multi-step Checkout**
Actor	Customer
Description	Finalizes a purchase through a secure process with atomic inventory reservation and payment processing.
Preconditions	- Cart contains valid, in-stock items (UC-0005). - User is Authenticated with active session.
Ordinary Sequence	1 Customer clicks “Proceed to Checkout” from Cart (UC-0005). 2 System displays Shipping Address selection (UC-0007). 3 Customer selects/enters Address. 4 Customer selects Payment Method. 5 Customer clicks “Place Order”. 6 System starts atomic transaction. 7 System automatically reserves inventory (UC-0018). 8 System processes payment via Gateway. 9 System saves Order and commits transaction.
Postconditions	- Order created with status “Placed” (UC-0006). - Inventory is physically reserved.
Alternative Sequences	A1 If Out of Stock: Transaction rolls back, user notified. A2 If Payment Failed: User prompted to retry. A3 If Address Invalid: System blocks progress.
Related Use Cases	UC-0005 (Cart), UC-0018 (Stock Reservation)

This use case encapsulates the end-to-end purchase flow, transitioning a customer's cart into a confirmed order. It enforces a strict sequence of validation steps, including address selection, payment method configuration, and final review. Crucially, the system employs an atomic transaction across the "Ordering" and "Inventory" domains to ensure stock is reserved exactly when the order is placed, preventing overselling.

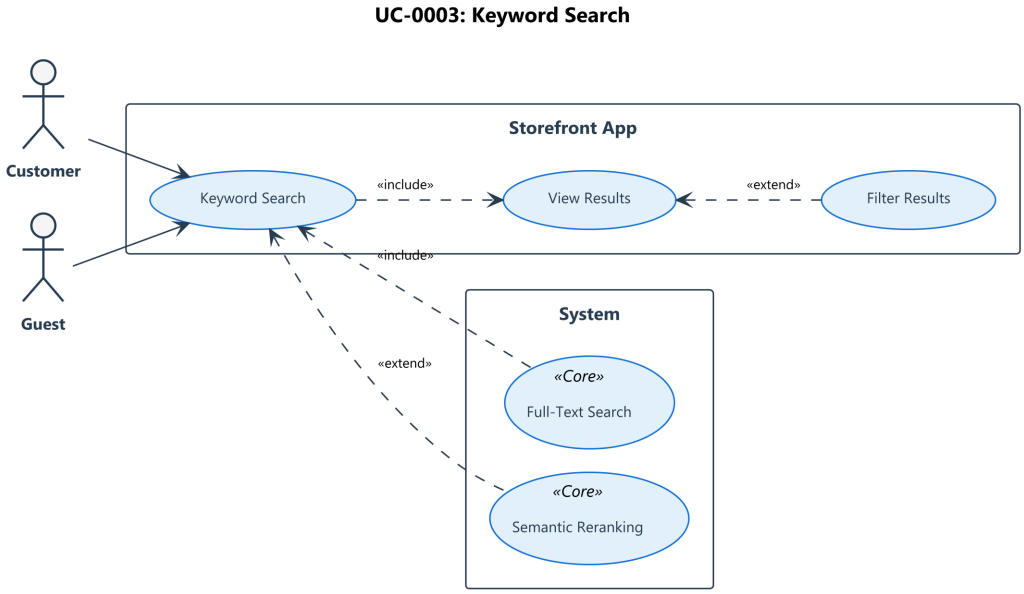

Figure 10. Use Case Diagram for UC-0003

Table 25. UC-0003: Keyword Search

UC-0003	**Keyword Search**
Actor	Customer
Description	Finding products using text-based queries against product metadata.
Preconditions	- Customer is on the store page.
Ordinary Sequence	1 Customer enters query. 2 System executes text search on Name/Description. 3 System returns paginated list.
Postconditions	- Results displayed.
Related Use Cases	UC-0001 (Browse)

The Keyword Search functionality provides a direct mechanism for users to find specific items by matching query strings against product titles, descriptions, and metadata. This feature utilizes full-text search capabilities within the database, offering immediate feedback and ensuring that users with specific intent can locate products without navigating the category tree.

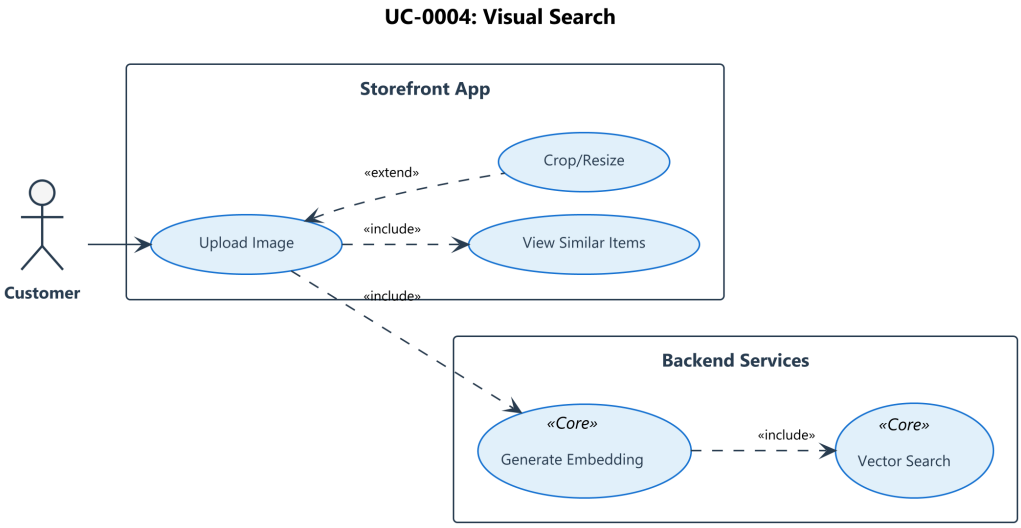

Figure 11. Use Case Diagram for UC-0004

Table 26. UC-0004: Visual Search

UC-0004	**Visual Search**
Actor	Customer
Description	Upload an image to find visually similar products using AI embeddings.
Preconditions	- ML Service is online.
Ordinary Sequence	1 Customer uploads image. 2 System generates embedding (UC-0017). 3 System queries vector database. 4 Displays similar items.
Postconditions	- Similar products displayed.
Related Use Cases	UC-0003 (Keyword Search)

Visual Search represents a core AI capability of the platform, enabling users to upload an image to find visually similar products. Upon upload, the system generates a vector embedding of the image and utilizes the PostgreSQL `pgvector` extension to perform a k-Nearest Neighbors (k-NN) search against the product catalog. This bypasses textual limitations, matching items based on style, color, and pattern.

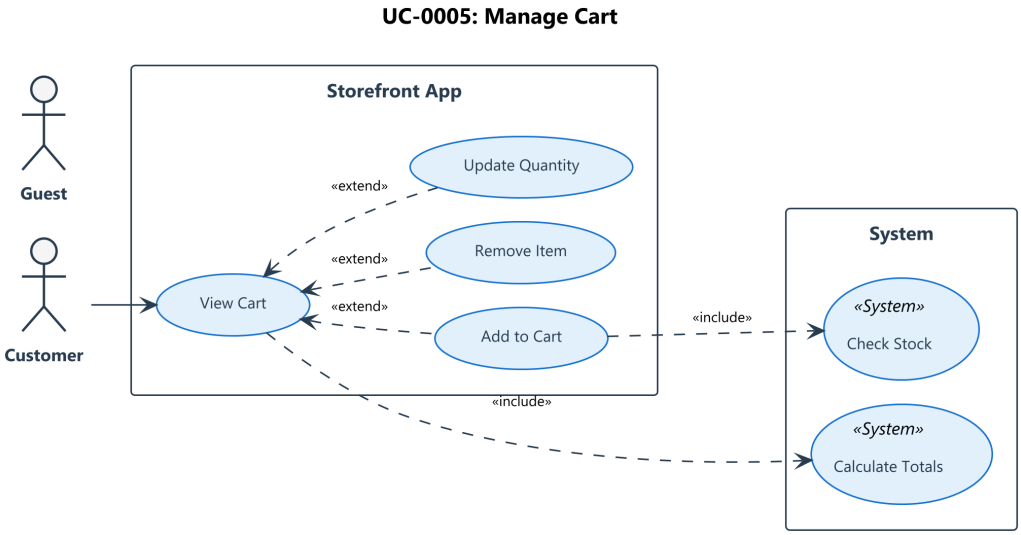

Figure 12. Use Case Diagram for UC-0005

Table 27. UC-0005: Manage Shopping Cart

UC-0005	**Manage Shopping Cart**
Actor	Customer
Description	Add, update, or remove items in the shopping cart.
Preconditions	- Product variant is selected.
Ordinary Sequence	1 Customer clicks “Add to Cart”. 2 System updates Cart in Database/Session. 3 System recalculates totals.
Postconditions	- Cart updated.
Related Use Cases	UC-0002 (Checkout)

This use case governs the temporary storage of products selected for purchase. It acts as a staging area where users can adjust quantities, remove items, or review their potential purchase. The cart state is maintained across sessions (for authenticated users), persisting data to ensuring a continuous shopping experience even if the user navigates away or switches devices.

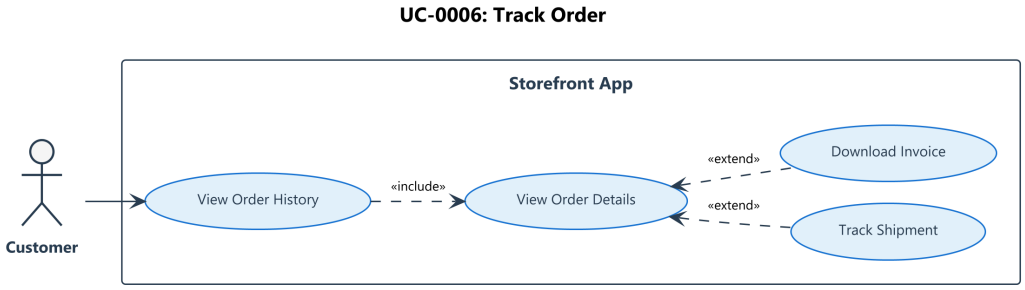

Figure 13. Use Case Diagram for UC-0006

Table 28. UC-0006: Track Order Status

UC-0006	**Track Order Status**
Actor	Customer
Description	View history and specific status of past orders.
Preconditions	- User is Authenticated.
Ordinary Sequence	1 Customer accesses “Order History”. 2 System retrieves orders for User ID. 3 Customer selects Order to view. 4 System displays status and items.
Postconditions	- History displayed.
Related Use Cases	UC-0002 (Checkout)

Post-purchase transparency is provided through the Order Tracking feature. Authenticated customers can access a historical view of their transactions, drilling down into specific orders to view current status (e.g., Pending, Shipped, Delivered) and line item details. This read-only view aggregates data from the Order Processing service to keep the user informed.

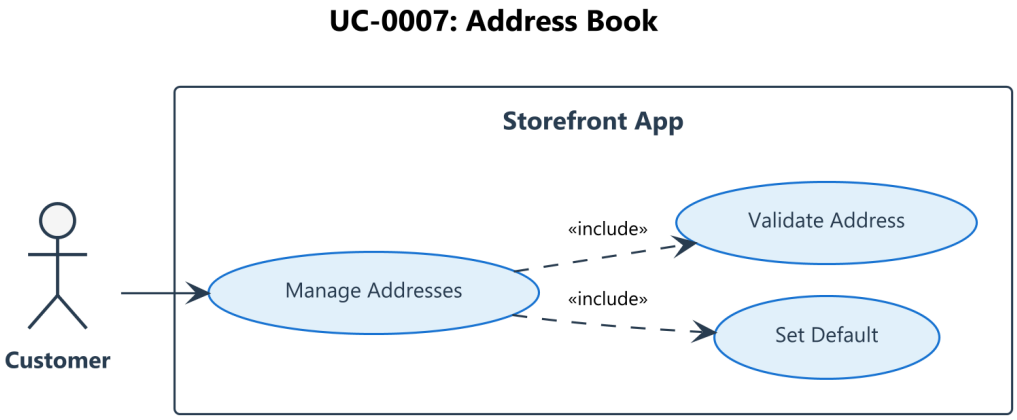

Figure 14. Use Case Diagram for UC-0007

Table 29. UC-0007: Address Book

UC-0007	**Address Book**
Actor	Customer
Description	Manage delivery addresses.
Preconditions	- User is Authenticated.
Ordinary Sequence	1 Customer views Addresses. 2 Adds or Edits Address. 3 System validates and saves.
Postconditions	- Address saved.
Related Use Cases	UC-0002 (Checkout)

The Address Book allows customers to manage their shipping destinations efficiently. By supporting Create, Read, Update, and Delete (CRUD) operations for saved addresses, the system streamlines the checkout process (UC-0002), allowing users to select pre-validated locations rather than re-entering details for every purchase.

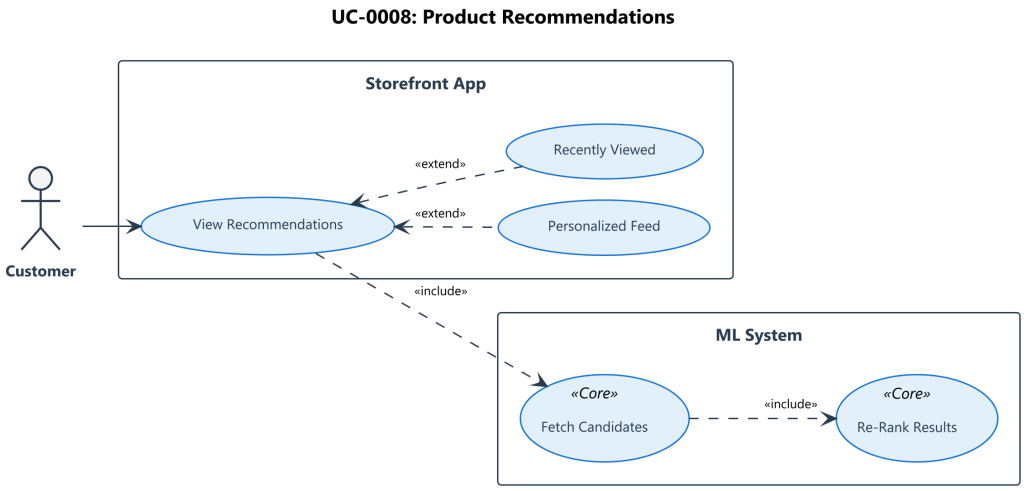

Figure 15. Use Case Diagram for UC-0008

Table 30. UC-0008: Product Recommendations

UC-0008	**Product Recommendations**
Actor	Customer
Description	View AI-driven recommendations based on product similarity.
Preconditions	- User viewing a product.
Ordinary Sequence	1 System detects current product. 2 Queries ML Service/Vector DB for similar items (UC-0017). 3 Displays “You May Also Like”.
Postconditions	- Recommendations displayed.
Related Use Cases	UC-0001 (Browse)

Leveraging the same underlying vector infrastructure as Visual Search, this use case delivers passive recommendations (“You May Also Like”) on product detail pages. By analyzing vector similarity between the currently viewed item and the rest of the catalog, the system surfaces relevant alternatives or complementary products without explicit user queries, increasing cross-sell opportunities.

2.4.2 Administrator Functional Area

This section details the use cases for the Administrator, focusing on product management, inventory control (ERP), and analytics.

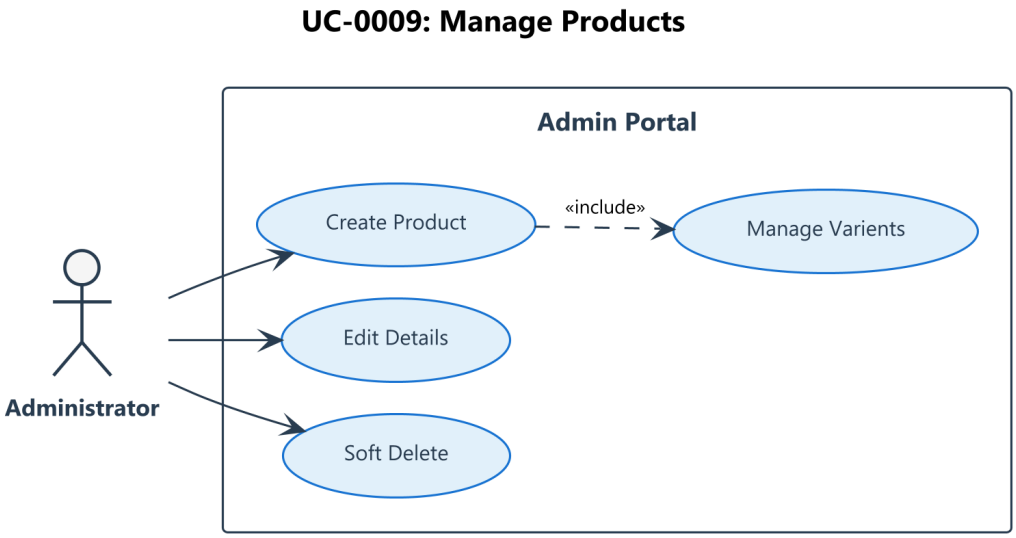

Figure 16. Use Case Diagram for UC-0009

Table 31. UC-0009: Manage Products

UC-0009	**Manage Products**
Actor	Administrator
Description	Create and manage products, variants, and metadata in the catalog.
Preconditions	- Admin is authenticated.
Ordinary Sequence	1 Admin clicks “New Product”. 2 Enters Name, Slug, Description. 3 Adds Variants (Price, SKU, Attributes). 4 Sets Status (Draft/Active). 5 System saves Product and Variants to database.
Postconditions	- Product is discoverable (if Active).
Related Use Cases	UC-0011 (Taxonomy)

This use case encompasses the lifecycle management of products within the catalog. It empowers administrators to create, update, and deactivate products, establishing the core data required for the storefront. A critical aspect is the management of SKU-level variants (e.g., sizes, colors) under a single parent product, allowing for complex inventory tracking and accurate pricing models.

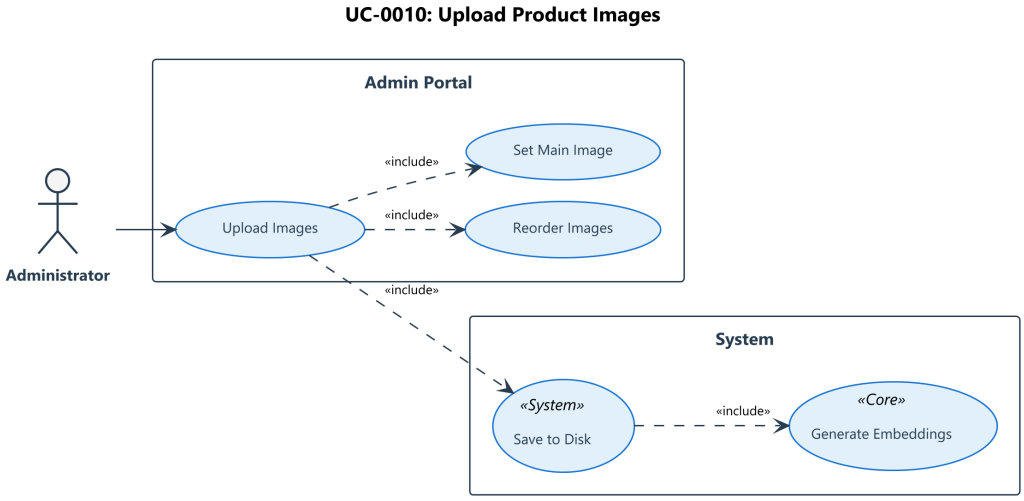

Figure 17. Use Case Diagram for UC-0010

Table 32. UC-0010: Upload Product Images

UC-0010	**Upload Product Images**
Actor	Administrator
Description	Upload product images which are automatically processed for display and AI vectorization.
Preconditions	- Admin is authenticated. - Product exists (UC-0009).
Ordinary Sequence	1 Admin selects product and clicks “Upload Image”. 2 Uses file picker to select local image. 3 System validates file constraints (< 10MB, Type). 4 System generates thumbnail and saves to storage. 5 System creates Product Image Record (Status: Pending). 6 System saves and triggers Background Vectorization (UC-0017).
Postconditions	- Image appears in gallery. - Image queued for AI processing.
Related Use Cases	UC-0009 (Product Management), UC-0017 (Vector Gen)

Uploading product images is a multi-step process that triggers both storage operations and AI processing. When an admin uploads an image, the system not only saves it to the blob store but also pipelines it to the AI service. This automatic trigger (UC-0017) generates vector embeddings for the image, enabling the visual search and recommendation features without manual intervention.

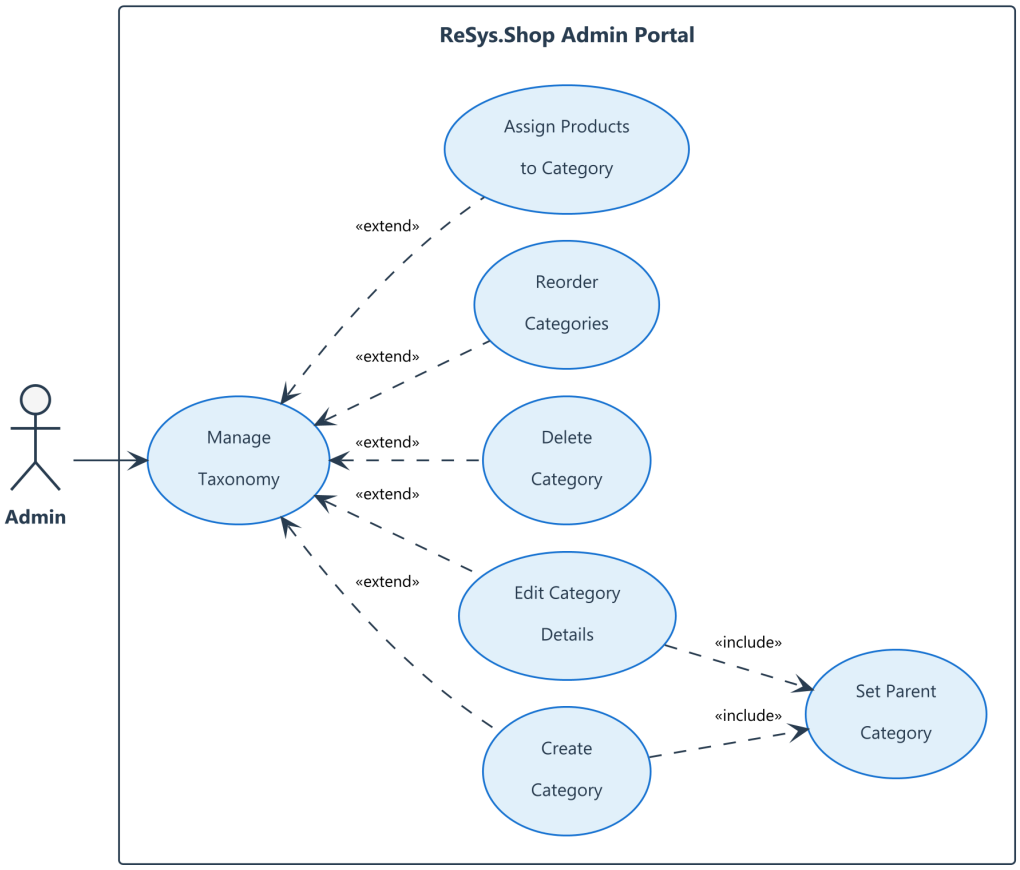

Figure 18. Use Case Diagram for UC-0011

Table 33. UC-0011: Taxonomy Management

UC-0011	**Taxonomy Management**
Actor	Administrator
Description	Organize categories hierarchy for product navigation.
Preconditions	- Admin is authenticated.
Ordinary Sequence	1 Admin clicks “New Category”. 2 Enters Name and Slug. 3 Selects Parent Category (optional). 4 System saves Category. 5 Admin assigns Products to Category (UC-0009).
Postconditions	- Category hierarchy updated. - Navigation menu reflects changes.
Related Use Cases	UC-0009 (Product Management)

Taxonomy Management allows the administrator to define the hierarchical structure of the catalog. By creating parent and child categories, the admin controls the "Browse" menu structure on the storefront. This organization is essential for efficient user navigation and ensures that products are grouped logically for both display and filtering purposes.

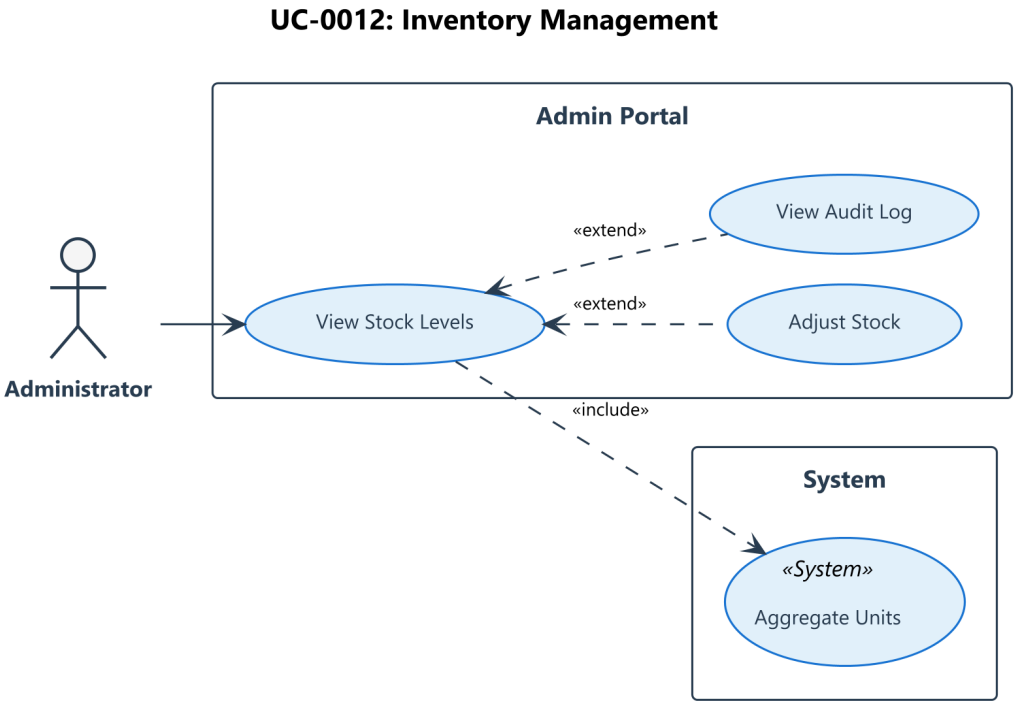

Figure 19. Use Case Diagram for UC-0012

Table 34. UC-0012: Inventory Management

UC-0012	**Inventory Management**
Actor	Administrator
Description	View real-time physical stock levels across locations and receive visual low-stock indications.
Preconditions	- Admin has Inventory permissions.
Ordinary Sequence	1 Admin navigates to Inventory Dashboard. 2 System queries Inventory Aggregates (OnHand, Reserved). 3 System generates Inventory Summary for alerts (UC-0020). 4 Displays Stock Table with "Available" counts.
Postconditions	- Stock levels displayed accurately.
Related Use Cases	UC-0020 (Low Stock Alert)

This use case provides a real-time window into the physical stock levels of the warehouse. The Inventory Management interface aggregates data from various stock movements (inbound shipments, sales, returns) to present an accurate count of “On Hand” versus “Available” quantities. It serves as the source of truth for the system’s “Available to Promise” (ATP) logic.

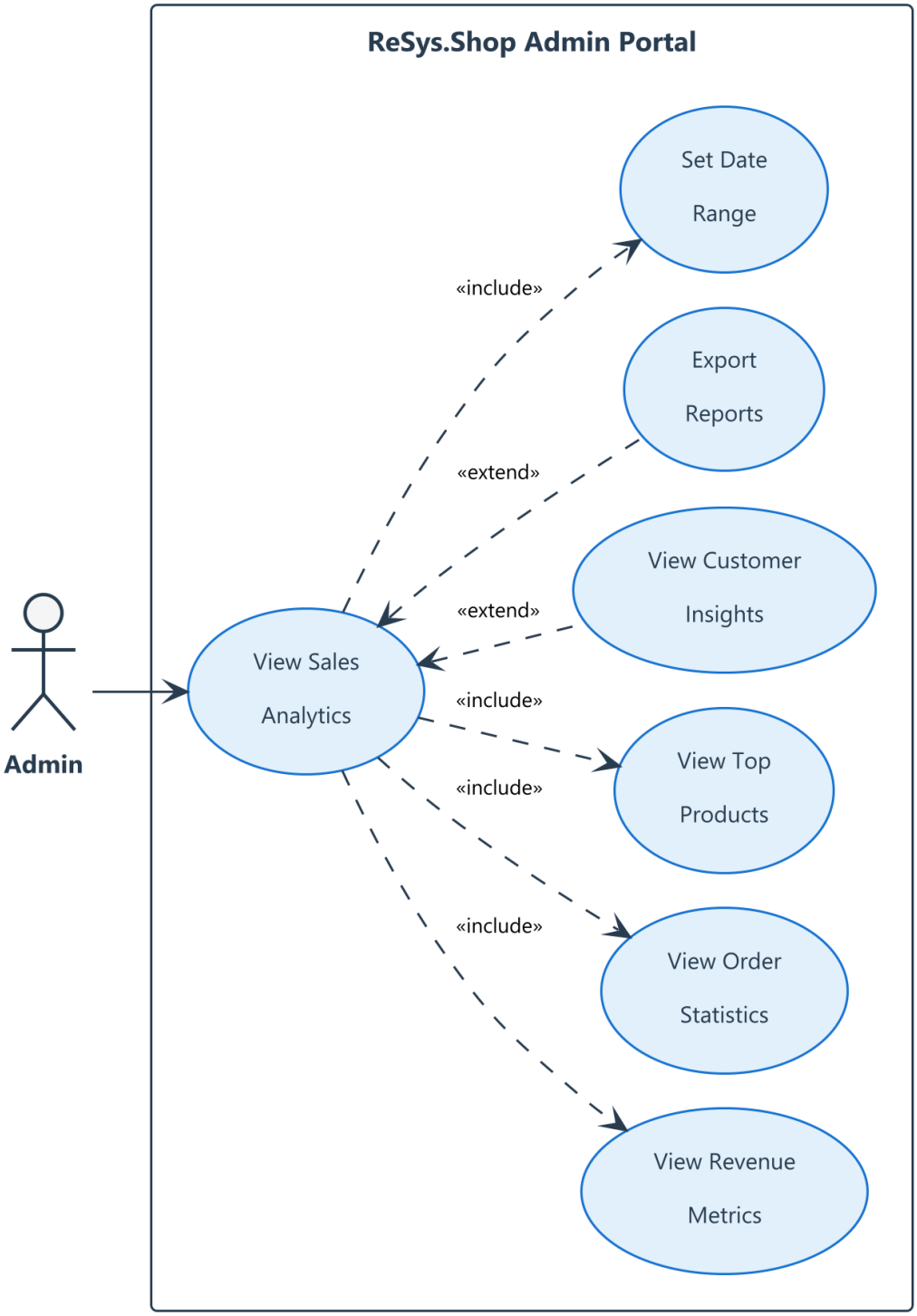

Figure 20. Use Case Diagram for UC-0013

Table 35. UC-0013: Analytics Dashboard

UC-0013	**Analytics Dashboard**
Actor	Administrator
Description	View high-level sales KPIs in the Dashboard to monitor business performance.
Preconditions	- Admin has Dashboard permissions.
Ordinary Sequence	1 Admin accesses Dashboard Home. 2 System queries Sales Summary (Revenue, Order Count). 3 System queries Inventory Summary (Low Stock). 4 Displays KPIs, Charts, and Recent Activity.
Postconditions	- Real-time metrics displayed.
Related Use Cases	UC-0012 (Inventory)

The Analytics Dashboard offers a high-level overview of business health, aggregating key performance indicators (KPIs) such as Total Revenue, Order Volume, and Low Stock Warnings. It synthesizes data from the Order, Catalog, and Inventory services into a cohesive visual report, allowing administrators to make data-driven decisions regarding restocking and marketing.

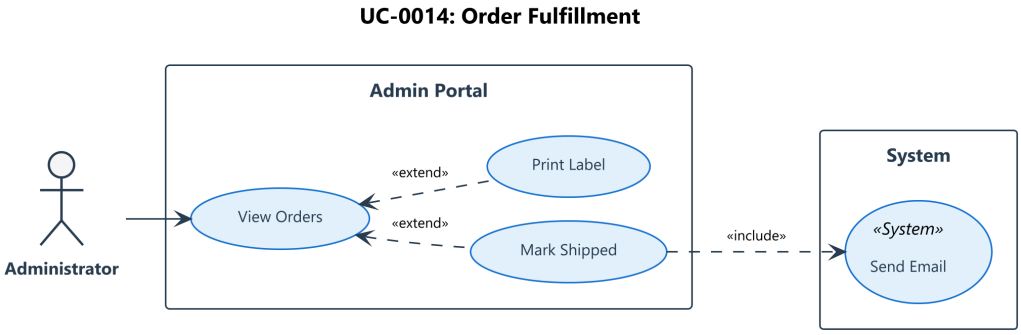

Figure 21. Use Case Diagram for UC-0014

Table 36. UC-0014: Order Fulfillment

UC-0014	**Order Fulfillment**
Actor	Administrator
Description	Process orders, select shipment origin, and create shipments, updating the stock ledger.
Preconditions	- Order is in “placed” state (UC-0002). - Inventory is reserved (UC-0018).
Ordinary Sequence	1 Admin views Pending Orders. 2 Selects Order and clicks “Ship”. 3 System uses Logic to assign Warehouse. 4 System creates Shipment Record. 5 System records Stock Out Movement (Reserved -> Shipped). 6 System updates Order Status to “Shipped”.
Postconditions	- Shipment created. - Stock physically deducted.
Related Use Cases	UC-0006 (Track Order), UC-0019 (Fulfillment Logic)

Order Fulfillment covers the operational workflow of picking, packing, and shipping customer orders. This process transforms a “Reserved” inventory item into a “Shipped” item, decrementing the physical stock ledger. The system automates the selection of the optimal fulfillment center (if multiple exist) and updates the order status to keep the customer informed.

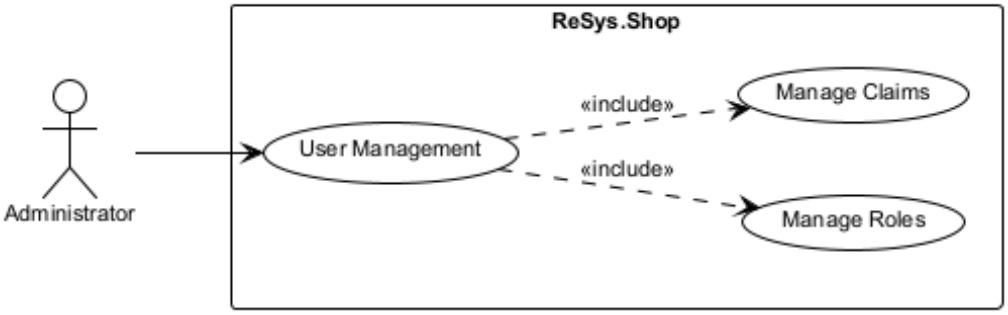

Figure 22. Use Case Diagram for UC-0015

Table 37. UC-0015: User Management

UC-0015	**User Management**
Actor	Administrator
Description	Manage user accounts and roles.
Preconditions	- Admin has User Management permissions.
Ordinary Sequence	1 Admin views User List. 2 Selects User to Edit roles/permissions. 3 System updates User Claims.
Postconditions	- User permissions updated.
Related Use Cases	

This use case involves the governance of system access, allowing a Super Administrator to assign roles and permissions to other staff members. By controlling Claims (e.g., “CatalogEditor”, “InventoryManager”), the system implements the Principle of Least Privilege, ensuring that employees can only access the features relevant to their specific job functions.

2.4.3 System Functional Area

This section details the background processes and automated system behaviors that support the user-facing functionality.

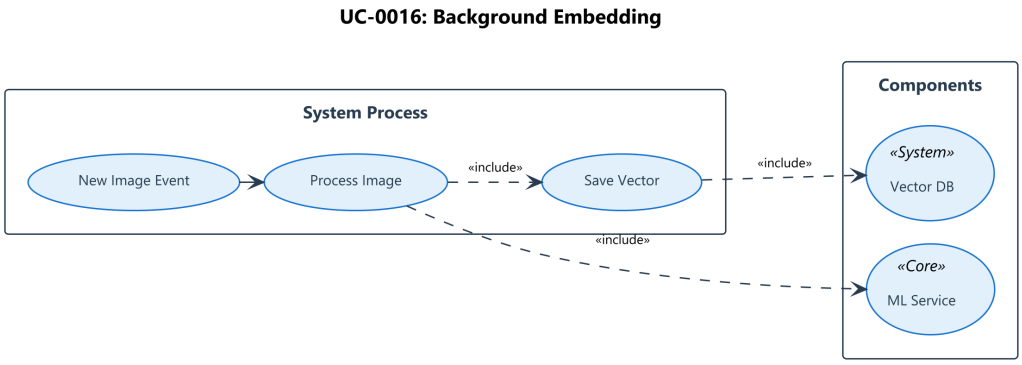

Figure 23. Use Case Diagram for UC-0016

Table 38. UC-0016: Generate AI Search Vectors

UC-0016	**Generate AI Search Vectors**
Actor	System (Background Job)
Description	Autonomous processing of product images to generate 512-dimensional embeddings for visual search.
Trigger	Image Upload (Status: Pending) (UC-0009).
Ordinary Sequence	1 Background Job detects Pending Image. 2 Job sends image URL to ML Service Endpoint. 3 ML Service computes normalized vector (Fashion-CLIP). 4 Job saves vector to Embedding Store. 5 Job marks Image Status as "Processed".
Related Use Cases	UC-0001 (Visual Search), UC-0004 (Recommendations)

This background system process is responsible for the asynchronous generation of vector embeddings from product images. Triggered whenever a new image is uploaded (UC-0010), it invokes the Python ML Service to compute a 512-dimensional vector using the Fashion-CLIP model. This pre-computed vector is essential for powering high-performance visual search and recommendation features.

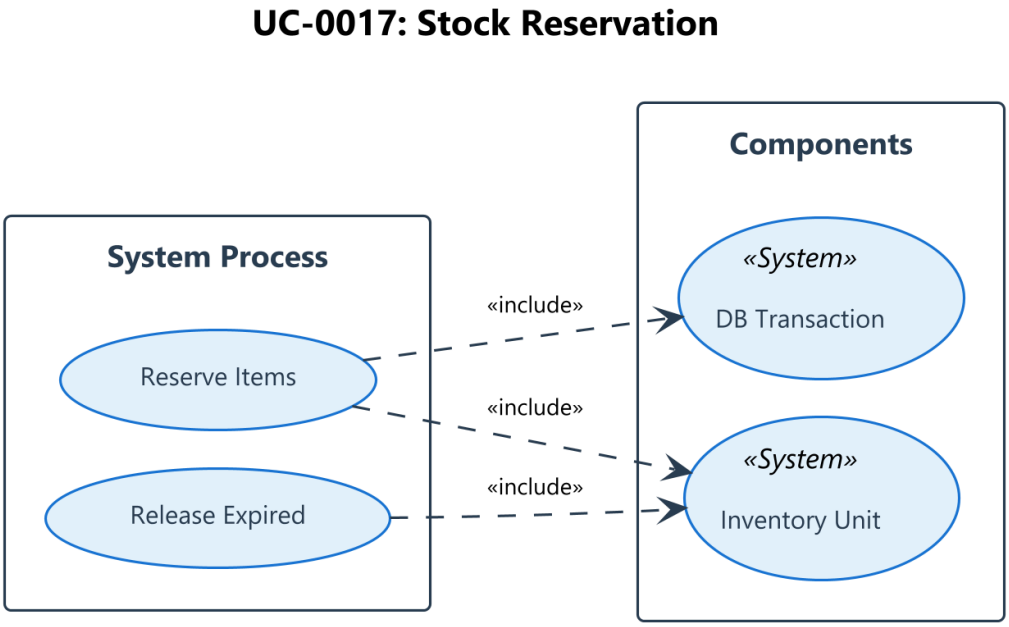

Figure 24. Use Case Diagram for UC-0017

Table 39. UC-0017: Automatic Stock Reservation

UC-0017	**Automatic Stock Reservation**
Actor	System (Transaction)
Description	Prevent overselling by atomically reserving inventory during the checkout process.
Trigger	Checkout Confirmation (UC-0002).
Ordinary Sequence	1 Transaction begins. 2 Inventory Service checks availability (OnHand - Reserved) (UC-0012). 3 If sufficient: Increments `Reserved` count. 4 Updates Stock Record. 5 Commit Transaction.
Alternative Sequences	A1 If insufficient stock: Throw Denial Exception (Rollback Transaction).
Related Use Cases	UC-0002 (Checkout)

Automatic Stock Reservation is a critical transactional mechanism that prevents overselling in a high-concurrency e-commerce environment. When a customer places an order (UC-0002), this process atomically locks the required quantity of inventory. If the stock is insufficient, the entire transaction is rolled back, ensuring data consistency between the Order and Inventory domains.

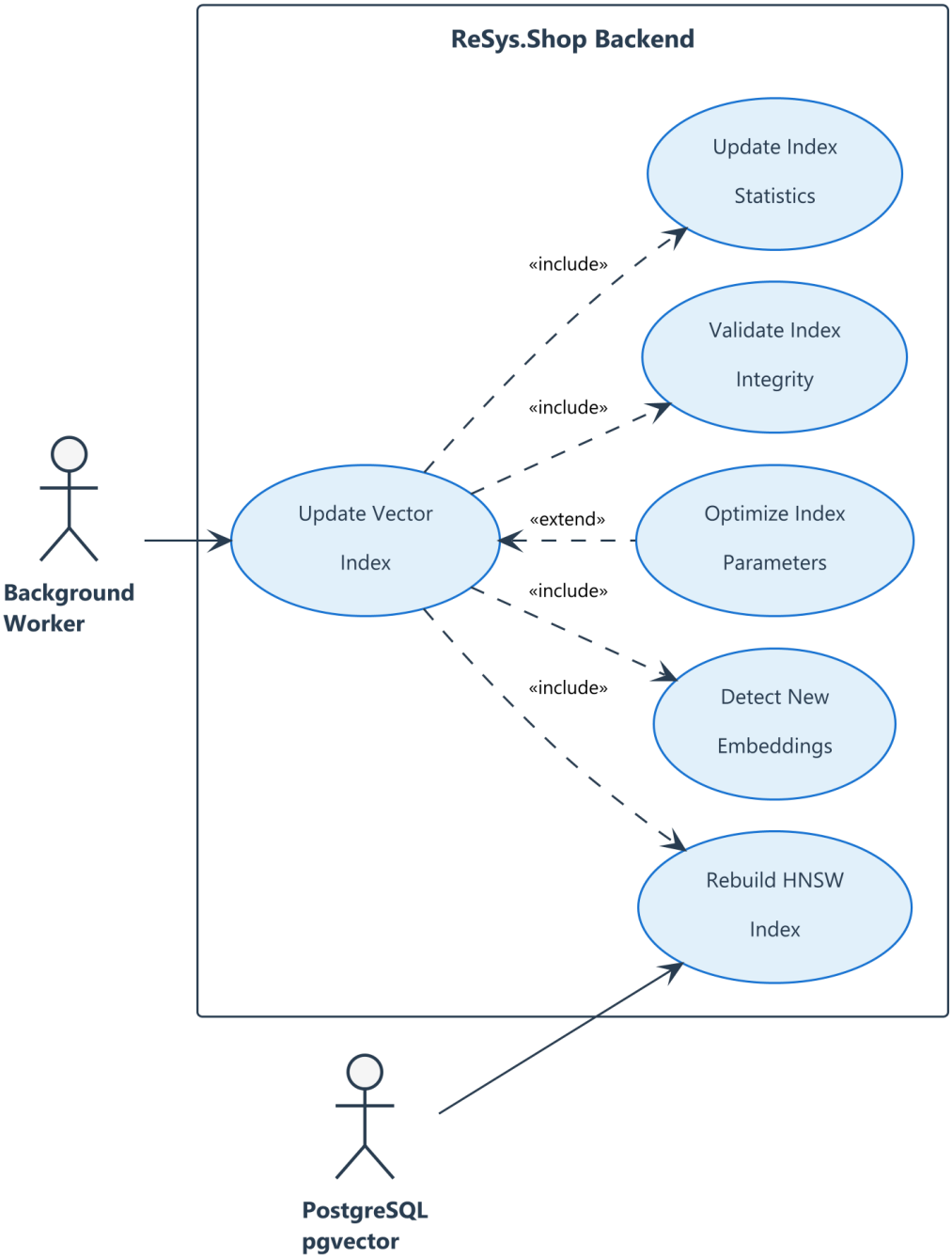

Figure 25. Use Case Diagram for UC-0018

Table 40. UC-0018: Vector Index Maintenance

UC-0018	**Vector Index Maintenance**
Actor	System
Description	Maintain HNSW index for fast similarity search.
Trigger	Vector Insertion (UC-0016).
Ordinary Sequence	1 System detects new embeddings. 2 Adds vector to pgvector HNSW index. 3 Vacuum/Analyze if fragmentation high.
Related Use Cases	UC-0016 (Embeddings)

This use case manages the lifecycle of the Hierarchical Navigable Small World (HNSW) index within the PostgreSQL database. As new vectors are generated (UC-0016), this process ensures they are efficiently inserted into the index structure. Periodic maintenance tasks, such as re-indexing or vacuuming, are also handled here to maintain sub-millisecond query performance as the dataset grows.

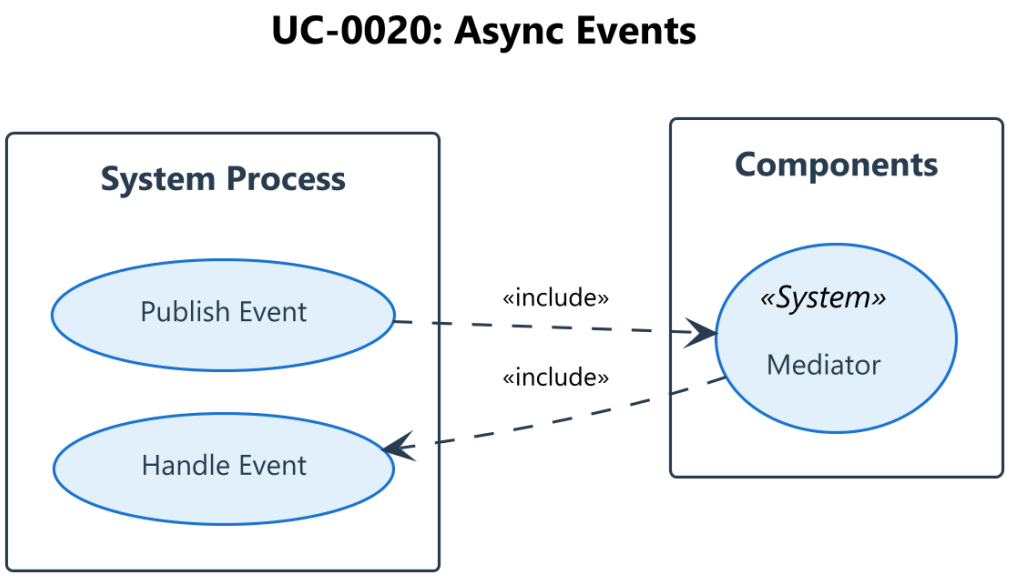

Figure 26. Use Case Diagram for UC-0019

Table 41. UC-0019: Background Job Processing

UC-0019	**Background Job Processing**
Actor	System
Description	Orchestrate asynchronous tasks (Email, Maintenance).
Trigger	Schedule or Event.
Ordinary Sequence	1 Job Scheduler triggers task. 2 Task executes logic (e.g., Send Order Email). 3 On Failure: Retry with exponential backoff. 4 On Success: Mark complete.
Related Use Cases	UC-0006 (Order Status)

The Background Job Processing system handles long-running or scheduled tasks that should not block user interactions. Using a durable queue (Hangfire), it reliably executes dependencies such as sending order confirmation emails, clearing expired carts, and generating daily reports. It includes built-in retry logic to handle transient failures, ensuring system robustness.

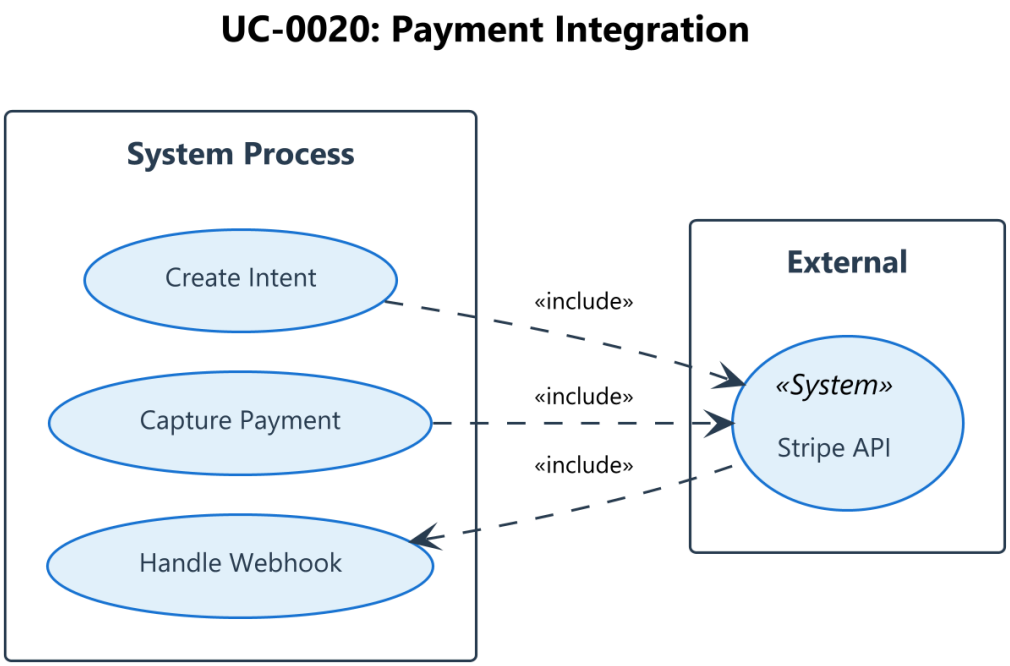

Figure 27. Use Case Diagram for UC-0020

Table 42. UC-0020: Payment Integration

UC-0020	**Payment Integration**
Actor	System
Description	Securely process payments via Stripe Gateway.
Trigger	Checkout (UC-0002).
Ordinary Sequence	1 System creates Payment Intent. 2 Customer confirms payment on Client. 3 System verifies Webhook/Confirmation. 4 System captures funds.
Related Use Cases	UC-0002 (Checkout)

Payment Integration securely handles the interaction with external payment gateways (e.g., Stripe). It manages the "Payment Intent" lifecycle, from initial authorization during checkout (UC-0002) to the final capture of funds upon successful order validation. This isolation ensures that sensitive financial data is handled compliantly and that payment status updates are accurately reflected in the order history.

2.4.4 Use Case Dependencies

Table 43. System Use Case Dependencies

Use Case	Depends On	Triggers
UC-0001 (Visual Search)	UC-0016 (Vectors)	-
UC-0004 (Recommendations)	UC-0016 (Vectors)	-
UC-0002 (Checkout)	Inventory (UC-0012)	UC-0017 (Stock Res)
UC-0013 (Fulfillment)	UC-0002 (Order)	-
UC-0016 (Analytics)	UC-0002, UC-0012	-
UC-0022 (Stock Audit)	UC-0012, UC-0013	-

2.5 SYSTEM ARCHITECTURE

This section details the architectural design of the system, employing a hybrid approach that combines **Vertical Slice Architecture (VSA)** for the core backend logic with a **Distributed Service** model for specialized AI integration.

The architecture was chosen to balance **modularity**, **maintainability**, and **scalability**, adhering to modern software engineering principles while avoiding the complexity of a full microservices mesh for the core business domain.

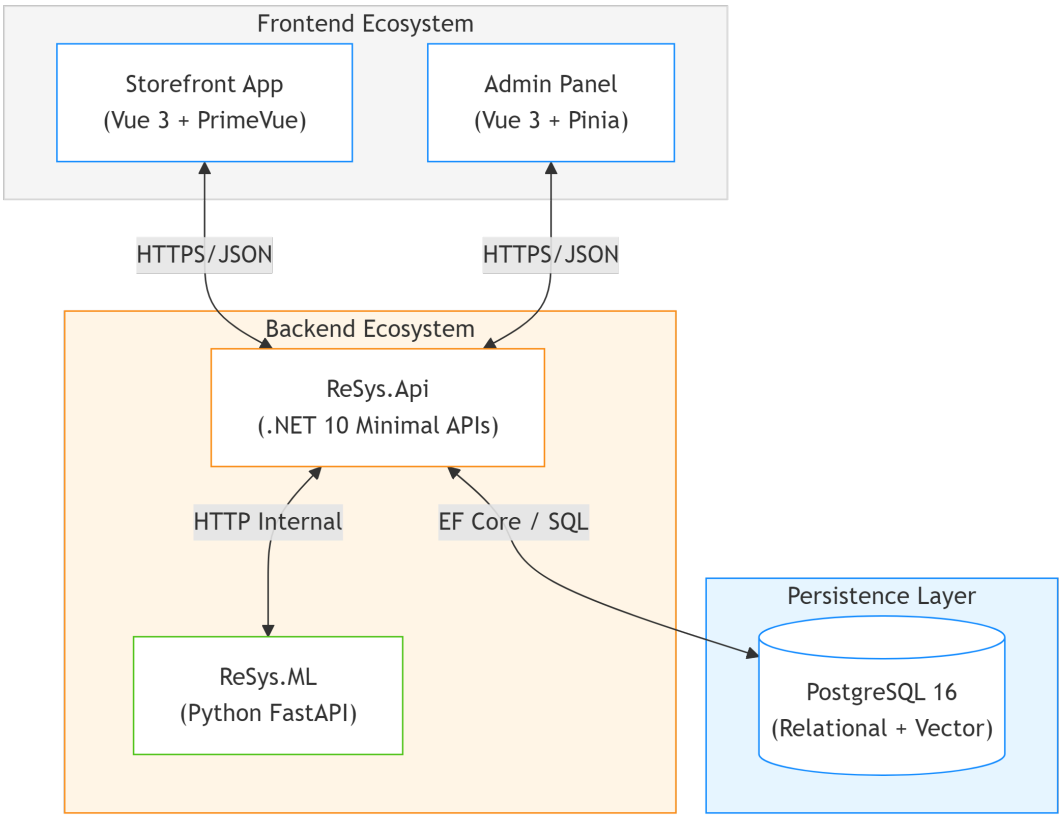

Figure 28. High-Level System Architecture

2.5.1 Architectural Style and Rationale

The application adopts a **Distributed Vertical Slice Architecture**. This choice addresses the specific dual nature of the project: a transactional e-commerce core and a computational AI engine.

1. **Distributed Services:** The system is split into two primary backend services to separate concerns based on technology strengths:
   - **Core API (.NET 10):** Handles high-concurrency user requests, business rules, and data integrity. .NET was selected for its robust type system and performance.
   - **ML Service (Python 3.12):** Handles tensor operations and model inference. Python was chosen for its rich ecosystem of AI libraries (PyTorch, Transformers).

   This separation allows the ML service to be scaled independently (e.g., on GPU nodes) without duplicating the transactional overhead of the core application.

2. **Vertical Slices (Backend):** Unlike traditional "Layered Architecture" (Controller → Service → Repository), which groups code by technical concern, this system groups code by **Feature** (e.g., "Add to Cart", "Update Profile").
   - **Rationale:** In e-commerce, changes are almost always vertical (e.g., adding a discount field affects the API, logic, and database). VSA keeps all these related

files together, reducing "context switching" and the risk of ripple effects across unrelated features.

2.5.2 Core Components

The system is composed of three primary service groups:

1. **Storefront & Admin UI (Frontend):** Built with **Vue 3** and **Vite**, utilizing a component-based architecture. It provides a responsive interface that communicates with the backend via RESTful APIs. It is stateless and served via a lightweight Node.js server.
2. **Core API Service (Backend - Transactional):** The central nervous system built with **.NET 10**. It handles authentication (Auth0/JWT), data persistence (PostgreSQL), and orchestrates business workflows using **MediatR**. It serves as the "Source of Truth" for all business data.
3. **ML Service Layer (AI Engine):** Split into two specialized Python 3.12 / FastAPI services:
   - **ReSys.ImageSearch (Production):** A high-performance, stateless inference engine optimized for serving vector embeddings (Fashion-CLIP) with low latency.
   - **ReSys.ML (Research):** A comprehensive workbench for training, evaluating, and comparing different model architectures (CNN vs Transformers) as part of the thesis research.

2.5.3 Backend Design Patterns

strictly follows **Domain-Driven Design (DDD)** principles wrapped in **Vertical Slices**

2.5.3.1 Vertical Slice Organization

In VSA, a "Slice" is equivalent to a "Use Case". Files are physically grouped by their functional purpose in the `src/libs/ReSys.Core/Features` directory.

```
src/ReSys.Core/Features/
├── Catalog/
│   ├── CreateProduct/
│   │   ├── CreateProductCommand.cs      // Input DTO
│   │   ├── CreateProductHandler.cs      // Logic
│   │   └── CreateProductValidator.cs    // Rules
│   └── GetProduct/
│       ├── GetProductQuery.cs
│       └── GetProductHandler.cs
├── Orders/
│   ├── PlaceOrder/
│   │   ├── PlaceOrderCommand.cs
│   │   └── PlaceOrderHandler.cs
│   └── ShipOrder/
│       └── ...
└── Identity/
    └── ...
```

Listing 1: Vertical Slice Architecture: Physical organization of code by Feature rather than technical layer.

A typical slice (e.g., `PlaceOrder`) contains:

- **Request/Response DTOs:** Defines the public API contract for input and output, decoupled from internal entities.
- **Command/Query:** A simplified object defining the intent (e.g., `PlaceOrderCommand`).
- **Handler:** The isolated "function" that executes the logic. It takes the Command/Query and dependencies (DB, Services) and returns a Result.
- **Validator:** `FluentValidation` rules that run **before** the handler, ensuring only valid data enters the domain.

2.5.3.2 Unified Vertical Architecture

Unlike traditional layered architectures that separate concerns horizontally (Presentation, Logic, Data), ReSys.Shop adopts a **Vertical Slice Architecture (VSA)**. This approach groups all concerns related to a single business capability (e.g., "Place Order") into a cohesive, self-contained unit.

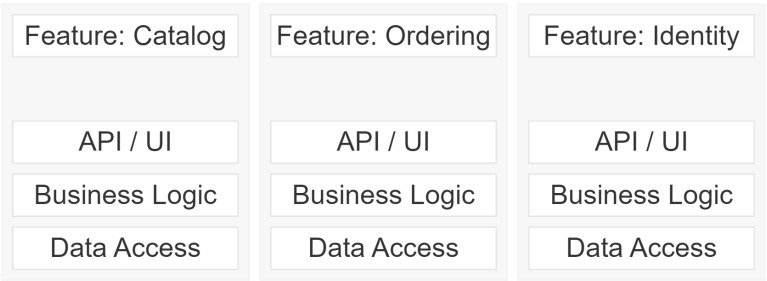

Figure 29. Vertical Slice Architecture: Organizing code by Features rather than Technical Layers

Table 44. Vertical Slice Architecture Concepts

Concept
Implementation in ReSys.Shop
Slice
A distinct feature (e.g., `SearchProducts`). Contains its own input (DTO), logic (Handler), and specific data access queries.
Coupling
Low coupling between slices. A change in the "Ordering" slice does not affect the "Catalog" slice, preventing regression bugs.
Shared Kernel
Common infrastructure like Middleware, Auth, and Validation Behaviors (`MediatR`) are shared across all slices.

2.5.3.3 Command Query Responsibility Segregation (CQRS)

The system distinguishes between operations that modify state and those that simply read it. This is implemented logically via MediatR:

- **Commands (Writes):** (e.g., `PlaceOrder`, `UpdateInventory`)
  - Intent: Change the system state.
  - Implementation: Loaded Aggregates including all child entities. Enforced by strong consistency and Transaction scopes.
  - Return: `ErrorOr<Result>` to handle domain failures explicitly.
- **Queries (Reads):** (e.g., `GetProductDetails`, `SearchProducts`)
  - Intent: Retrieve data for display.
  - Implementation: Optimized "Projection" queries. Can bypass the Domain Model (Entities) and project directly to DTOs using `Select()`.
  - Optimization: Uses `AsNoTracking()` in EF Core to avoid the overhead of change tracking.

2.5.4 Domain-Driven Design (DDD)

An Aggregate is a cluster of associated objects that we treat as a unit for the purpose of data changes.

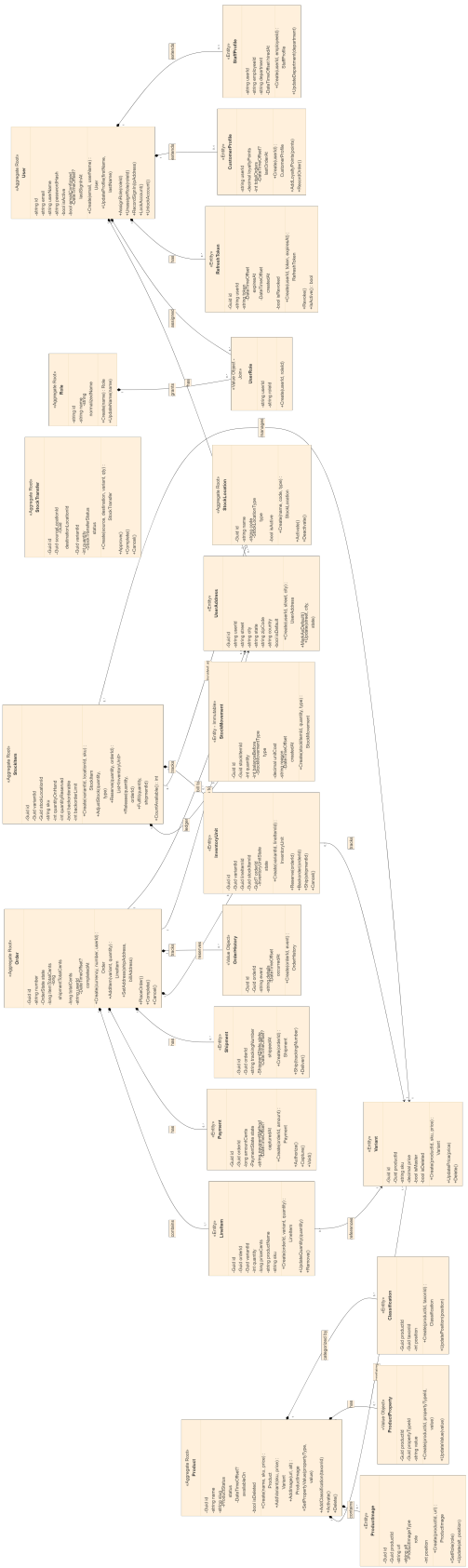

Figure 30. Domain Model: Vertical Slice Architecture showing Aggregate Roots and Bounded Contexts.

Table 45. Domain Aggregates and Bounded Contexts

Aggregate Root	Child Entities	Key Responsibility
Product	`Variant`, `ProductImage`, `ProductProperty`, `Classification`	Catalog management with multi-variant support, image library, dynamic properties, and taxonomy classification.
Order	`LineItem`, `Payment`, `Shipment`, `InventoryUnit`, `OrderHistory`	Order lifecycle from cart to fulfillment with payment processing, shipping tracking, and complete audit trail.
StockItem	`StockMovement`, `InventoryUnit` (shared)	Physical and logical inventory with immutable ledger, reservation tracking, and backorder support.
User	`UserAddress`, `UserRole`, `RefreshToken`, `CustomerProfile`, `StaffProfile`	Identity and access management with role-based permissions, session management, and extensible profiles.
StockLocation	`StockItem` (owns)	Warehouse and retail location management for multi-site inventory distribution.
Role	`UserRole` (join)	Permission grouping and role-based access control (RBAC).

2.5.4.1 Cross-Context Patterns

The domain model implements several advanced patterns to ensure consistency and extensibility:

Shared Entity Pattern: `InventoryUnit` is a child entity owned by both `Order` (for tracking customer promises) and `StockItem` (for physical inventory). This dual ownership ensures that reservation logic remains synchronized across the Ordering and Inventories contexts without introducing circular dependencies.

Profile Segregation: The `User` aggregate supports two distinct profile types (`CustomerProfile` for e-commerce features, `StaffProfile` for administrative capabilities), allowing role-specific data to be isolated while maintaining a unified identity model.

Soft Delete (ISoftDeletable): Critical aggregates like `Product`, `StockItem`, and `User` implement soft deletion to preserve referential integrity and audit trails. Deleted entities remain in the database with `IsDeleted = true` and can be restored if needed.

Metadata Extensibility (IHasMetadata): All major aggregates expose `PublicMetadata` and `PrivateMetadata` dictionaries, enabling runtime schema evolution without database migrations. This is particularly useful for A/B testing and feature flags.

2.5.4.2 Domain Events

system uses **Domain Events** to decouple side effects. When an aggregate changes state, it raises an event (implementing `IDomainEvent`) which is dispatched to in-process handlers.

Example Flow:

1. `Product` aggregate processes `AddImage()`.
2. `Product` raises `ImageUploadedEvent`.
3. Database Transaction commits the Product change.
4. **Side Effect:** `GenerateImageEmbeddingHandler` listens to the event and initiates the background vectorization job.

2.5.5 System Reliability & Observability

2.5.5.1 Reliability & Resilience

- **Transactions:** All Commands execute within an atomic `BeginTransactionAsync` scope. If the ML service fails during an image upload, the database record is rolled back, ensuring consistency.
- **Concurrency:** `RowVersion` columns (Optimistic Concurrency) prevent "lost updates" on inventory. If two admins try to update the same stock simultaneously, the second one receives a `ConcurrencyException`.

2.5.5.2 Observability

The system is instrumented for comprehensive monitoring:

- **Tracing:** A unique `TraceId` follows requests from the Vue Frontend → .NET API → Python ML Service, simplifying debugging across languages.
- **Metrics:** Infrastructure metrics (CPU, Memory) and Application metrics (Search Latency, Order Rate) are captured for performance analysis.

2.6 DATABASE DESIGN

This section provides the complete database schema for all bounded contexts. The system uses **PostgreSQL 16** as the primary relational store, leveraging its advanced extensions for AI vector operations.

2.6.1 Design Principles

The database schema adheres to the following core principles:

- **Polymorphic Precision:** Monetary values are stored differently depending on the use case: `DECIMAL` is used for flexible catalog pricing, while `BIGINT` (cents) is reserved for rigid transactional accounting.
- **Immutable Ledgers:** Inventory is tracked using an append-only ledger pattern (`StockMovements`), ensuring that every change in physical stock is auditable and reversible.

- **Optimistic Concurrency:** High-contention tables (`StockItems`, `Products`) utilize `RowVersion` (mapping to PostgreSQL's `xmin` system column) to detect and prevent overwrites during concurrent edits.
- **Vector-Native:** High-dimensional AI embeddings are treated as first-class citizens, stored directly alongside relational data in `image_embeddings` tables.

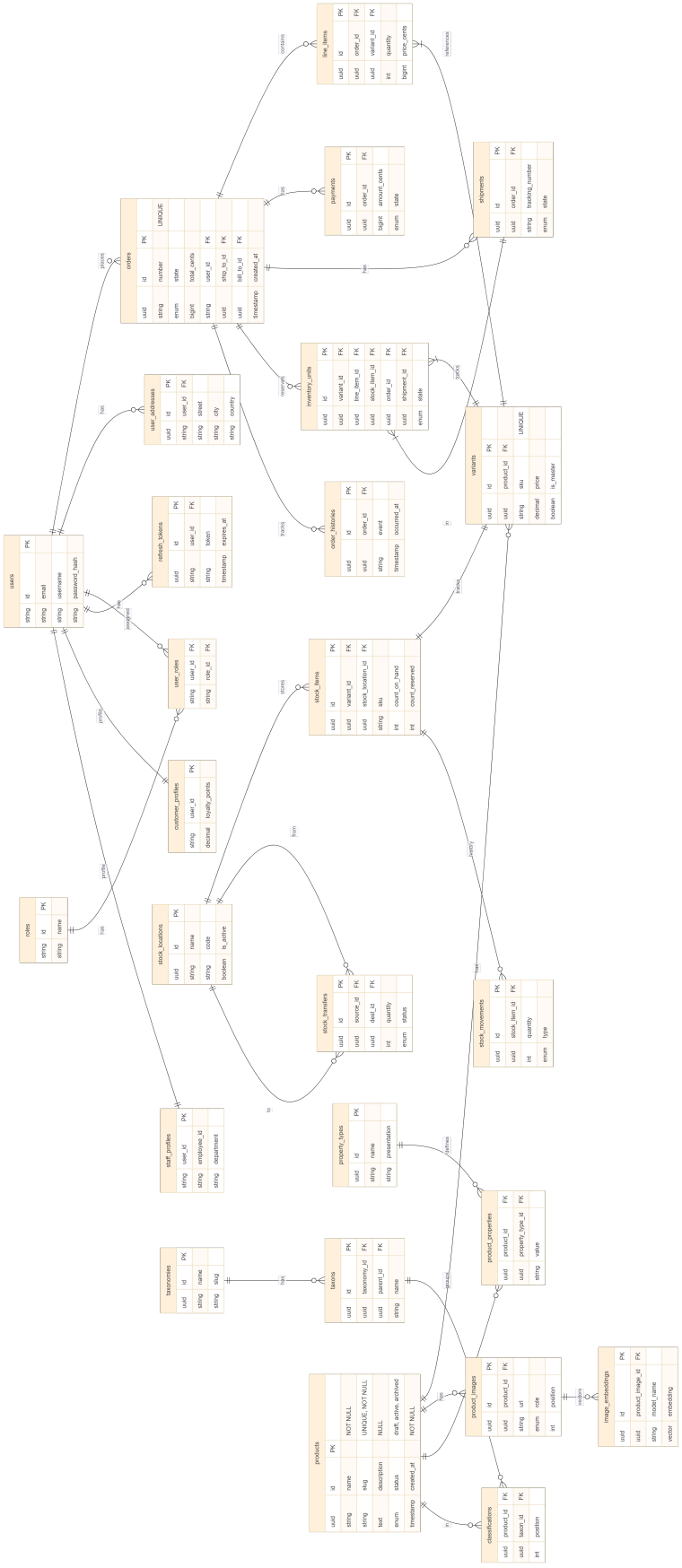

Figure 31. Backend Entity-Relationship Diagram (ERD): Complete database schema showing tables, keys, and relationships across all bounded contexts.

2.6.2 Identity Context

The **Identity Context** serves as the security and profile management foundation for the entire e-commerce ecosystem. It is designed to be compliant with modern identity standards (ASP.NET Identity Core) while supporting extensibility for e-commerce specific requirements like address management and soft-banning.

2.6.2.1 User Aggregate

The `User` entity serves as the **Aggregate Root** for this context. Unlike simple credential stores, this entity acts as the central hub for customer customer-centric data.

- **Authentication:** It stores secure password hashes and security stamps to manage session validity (e.g., invalidating all tokens when a password changes).
- **Profile Management:** It segregates standard identity fields (Email, Phone) from e-commerce profile data (Avatar, Active Status).
- **Auditability:** It includes `ConcurrencyStamp` to strictly enforce optimistic concurrency control during profile updates.

Table 46. Users table

No	Field	Type	Description
1	Id	UUID	Primary Key. Serves as the immutable reference for Orders and Carts.
2	UserName	VARCHAR(256)	Unique login identifier. Often mirrors Email but allows separation.
3	Email	VARCHAR(256)	Primary communication channel. Indexed for fast lookup.
4	FirstName	VARCHAR(100)	Customer's given name for personalization.
5	LastName	VARCHAR(100)	Customer's surname.
6	ProfileImagePath	TEXT	URI pointer to the user's avatar in object storage.
7	IsActive	BOOL	Soft-delete mechanism. Allows banning users without destroying relational integrity.
8	SecurityStamp	VARCHAR	Random value updated on credential changes to invalidate old cookies.
9	ConcurrencyStamp	UUID	Prevents lost updates during concurrent profile edits.
10	PasswordHash	TEXT	Argon2id or PBKDF2 hash of the user's password.
11	PhoneNumber	VARCHAR(20)	Optional contact number for SMS notifications.
12	TwoFactorEnabled	BOOL	Flag indicating if 2FA is required for login.
13	CreatedAt	TIMESTAMP	Account creation timestamp for cohort analysis.
14	LastSignInAt	TIMESTAMP	Last successful authentication timestamp.
15	SignInCount	INT	Total login count, used for engagement metrics.

2.6.2.2 Logistics & Profile Data

To support the "Ship-to" and "Bill-to" requirements of e-commerce, the system extends the identity model with `UserAddresses`. This allows a single user to maintain a rolodex of locations (Home, Work, Gift Recipient).

Table 47. UserAddresses table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	UserId	UUID	Foreign Key linking to the User Aggregate.
3	Street	VARCHAR(200)	Primary address line.
4	City	VARCHAR(100)	City or municipality.
5	State	VARCHAR(100)	State, Province, or Region.
6	ZipCode	VARCHAR(20)	Postal or ZIP code for shipping calculation.
7	CountryCode	VARCHAR(2)	ISO 3166-1 alpha-2 country code (e.g. 'US', 'VN').
8	IsDefault	BOOL	Marker for pre-filling checkout forms.
9	Type	INT	Discriminator: 0=Shipping, 1=Billing.

2.6.2.3 Authorization & RBAC

The system implements a **Role-Based Access Control (RBAC)** model supplemented by granular **Claims**.

- **Roles:** Define high-level groups like "Administrator" or "Customer".
- **Claims:** Define specific permissions like `catalog.manage` or `users.view`. This allows for fine-grained security policies where a "Content Manager" might update products but not delete users.

Table 48. Roles table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	Name	VARCHAR(100)	Human-readable role name (e.g. 'Administrator').
3	NormalizedName	VARCHAR(100)	Uppercase normalized name for consistent indexing.

Table 49. UserRoles table (Many-to-Many Bridge)

No	Field	Type	Description
1	UserId	UUID	Foreign Key to User.
2	RoleId	UUID	Foreign Key to Role.

2.6.2.4 External Identity & Sessions

To support modern authentication flows (OAuth2, OIDC) and mobile apps, the schema includes support for external login providers and refresh tokens.

- **UserLogins:** Links local accounts to providers like Google or Facebook.
- **UserTokens:** Stores ephemeral tokens for API access or password reset flows.

Table 50. UserLogins table

No	Field	Type	Description
1	LoginProvider	VARCHAR(100)	The provider name (e.g. 'Google').
2	ProviderKey	VARCHAR(100)	The unique user ID from the provider system.
3	UserId	UUID	Foreign Key linking to the local account.

2.6.3 Catalog Context

The **Catalog Context** is the core of the e-commerce storefront. It handles how products are defined, categorized, and presented to customers. The data model is designed for high flexibility and read-performance, supporting complex product hierarchies and AI-driven discovery.

2.6.3.1 Product Definition

The catalog uses a **Product-Variant** parent-child relationship to handle SKU variations.

- **Product:** Represents the abstract "concept" of an item (e.g., "Cotton T-Shirt"). It holds shared metadata like the description, slug, and general status.
- **Variant:** Represents the actual sellable physical unit (e.g., "Cotton T-Shirt - Red - Large"). It holds the specific SKU, price, inventory tracking flags, and physical dimensions.

This separation allows the storefront to group related items (Color/Size permutations) under a single page while tracking inventory for each specific combination strictly.

Table 51. Products table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	Name	VARCHAR(100)	The display name of the product family.
3	Slug	VARCHAR(255)	Unique URL fragment (e.g. 'cotton-tshirt'). Indexed for strict SEO lookup.
4	Description	TEXT	HTML or Markdown description of the product.
5	Status	VARCHAR(20)	Lifecycle state: 'Draft' (Hidden), 'Active' (Visible), 'Archived' (Hidden).
6	AvailableOn	TIMESTAMP	Scheduled publishing date. Products are hidden until this time passes.
7	MetaTitle	VARCHAR(255)	SEO: Overrides the tag for search engines.
8	MetaDescription	VARCHAR(500)	SEO: Meta description for search results snippets.
9	MetaKeywords	VARCHAR(255)	SEO: Comma-separated keywords.
10	IsDeleted	BOOL	Soft-delete flag to preserve historical data.

Table 52. Variants table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	ProductId	UUID	Foreign Key linking to the parent Product.
3	Sku	VARCHAR(100)	Stock Keeping Unit. Must be globally unique for warehouse tracking.
4	Price	DECIMAL(18,2)	The base selling price. Uses DECIMAL to prevent floating point errors.
5	CostPrice	DECIMAL(18,2)	Internal cost for margin calculation.
6	CompareAtPrice	DECIMAL(18,2)	Original price for discount display (strikethrough).
7	IsMaster	BOOL	Flag indicating if this is the 'default' variant shown on listing pages.
8	TrackInventory	BOOL	If true, prevents sales when stock is 0. If false, allows infinite sales.
9	Weight	DECIMAL(10,2)	Physical weight for shipping calculation.
10	Height	DECIMAL(10,2)	Physical height dimension.
11	Width	DECIMAL(10,2)	Physical width dimension.
12	Depth	DECIMAL(10,2)	Physical depth dimension.

2.6.3.2 Hierarchical Categorization (Taxonomies)

Categories are implemented using a **Taxonomy** model backed by a **Nested Set** data structure instead of a simple adjacency list.

- **Rationale for Nested Set Strategy:** A classic parent-child recursion requires N database queries to traverse a tree of depth N. The Nested Set model (using `Lft` and `Rgt` bounds) allows selecting an entire subtree (e.g., "All Men's Clothing") in a **single** SQL query, drastically improving read performance for mega-menus and breadcrumbs.

Table 53. Taxons table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	TaxonomyId	UUID	Links the category to a specific tree (e.g. 'Brand', 'Department').
3	ParentId	UUID	Self-reference for easy editing (Adjacency List view).
4	Name	VARCHAR(100)	Category name (e.g. 'Sneakers').
5	Slug	VARCHAR(255)	URL fragment for the category page.
6	Lft	INT	Nested Set Left Bound. Used for subtree inclusion checks.
7	Rgt	INT	Nested Set Right Bound.
8	Depth	INT	Cached tree depth for indentation in UI.
9	MetaTitle	VARCHAR(255)	SEO: Title override.
10	MetaDescription	VARCHAR(500)	SEO: Description snippet.

2.6.3.3 Dynamic Attributes (Options)

To support diverse product types without hardcoding columns (e.g., "Screen Size" vs "Fabric Type"), the system uses an **EAV-lite** (Entity-Attribute-Value) pattern called **Options**.

- **OptionType:** Defines the attribute class (e.g., "Color").
- **OptionValue:** Defines the specific choices (e.g., "Red", "Blue").
- **ProductOption:** Links these choices to variants, generating the permutations.

Table 54. OptionTypes table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	Name	VARCHAR(100)	Internal code (e.g. 'tshirt-size').
3	Presentation	VARCHAR(100)	Customer-facing label (e.g. 'Size').
4	Key	VARCHAR(50)	System identifier for filtering logic.

The flexibility of this EAV-lite model is managed through a dedicated interface that allows administrators to define new attribute classes without database schema migrations.

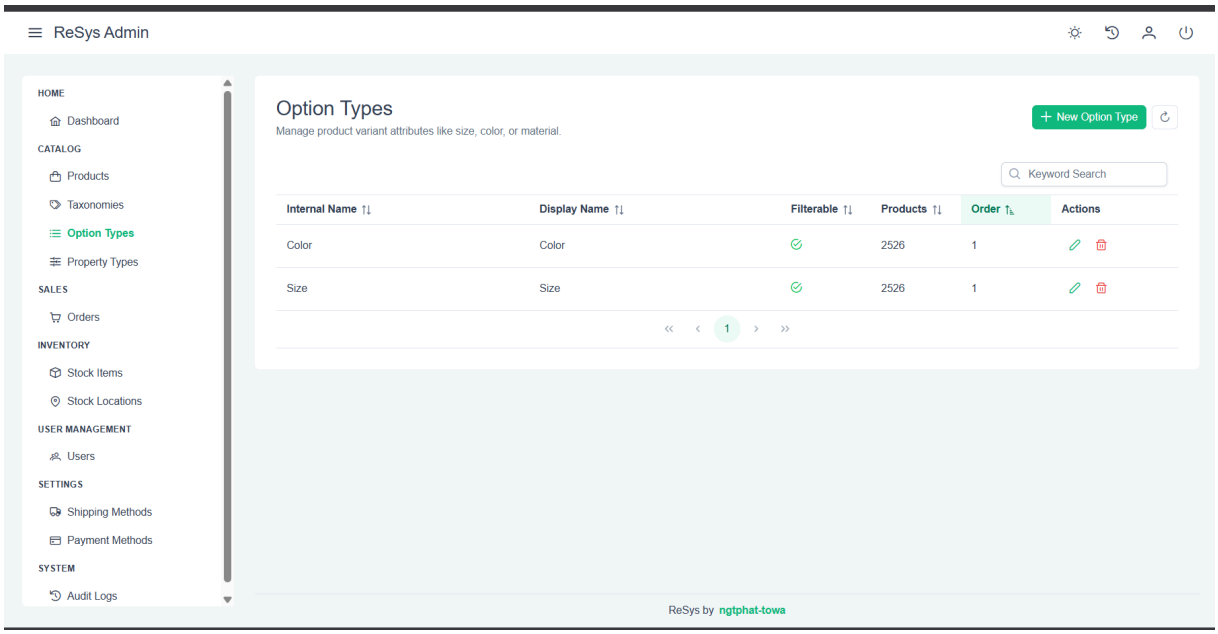

Figure 32. Attribute Management: Interface for defining reusable Option Types (e.g., Colors, Sizes) across the catalog.

2.6.3.4 AI Visual Intelligence

A key differentiator of ReSys.Shop is its vector-native architecture. Instead of treating images as simple URLs, they are analyzed and embedded into high-dimensional space.

- **Separation of Concerns:** `ImageEmbeddings` are decoupled from `ProductImages`. This allows multiple AI models (e.g., **Fashion-CLIP** for style similarity, **ResNet** for color matching, **OCR** for text) to attach their own vectors to the same image without polluting the main table.
- **pgvector:** Vectors are stored using the `vector(512)` type, enabling nearest-neighbor search directly in the database.

Table 55. ProductImages table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	ProductId	UUID	The product this image belongs to.
3	Url	TEXT	Direct link to the image blob storage.
4	Status	SMALLINT	Processing State: 0=Pending, 1=Processed, 2=Failed. Drives the background ML queue.
5	AltText	VARCHAR(255)	Accessibility text (often AI-generated).

Table 56. ImageEmbeddings table (pgvector)

No	Field	Type	Description
1	Id	UUID	Primary Key
2	ProductImageId	UUID	Foreign Key to the source image.
3	ModelName	VARCHAR(50)	Discriminator for the AI model used (e.g., 'fashion_clip').
4	Vector	VECTOR(512)	The 512-dimension float32 embedding vector.

2.6.4 Order Context

The **Order Context** is the transactional heart of the system. It orchestrates the complex lifecycle of converting a customer's shopping cart into a legally binding sales contract and ensuring its fulfillment.

2.6.4.1 Order Aggregate

The `Order` entity functions as a strict **Finite State Machine (FSM)**. It progresses through defined stages (Cart *to* Address *to* Payment *to* Complete) to ensure data integrity.

- **Financial Precision:** Unlike the Catalog (which acts as a "Menu"), the Order (which acts as the "Receipt") stores all monetary values in `BIGINT` cents. This avoids accumulating floating-point rounding errors during tax calculation and ensures exact reconciliation with payment gateways (Stripe/PayPal), which also process in minor units.
- **Snapshot Pattern:** Changes to catalog prices **must not** affect past orders. Therefore, `LineItems` do not merely link to `Products`; they copy (snapshot) the price, name, and SKU at the moment of purchase.

Table 57. Orders table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	Number	VARCHAR(50)	Human-readable reference (e.g., 'ORD-2023-1001').
3	State	INT	Enum representing the FSM state: 0=Cart, 1=Address, 2=Payment, 3=Complete, 4=Canceled.
4	ItemTotalCents	BIGINT	Subtotal of all merchandise.
5	ShipmentTotalCents	BIGINT	Subtotal of all shipping/handling costs.
6	TotalCents	BIGINT	The final amount to be charged to the customer. Aggregated from items + shipping.
7	UserId	UUID	Link to registered user. Nullable to support 'Guest Checkout'.
8	SessionId	VARCHAR(100)	Cookie-based ID to track anonymous carts before login.
9	CompletedAt	TIMESTAMP	The legal timestamp of the contract finalization.
10	CanceledAt	TIMESTAMP	Timestamp if the order was aborted.
11	ShipAddressId	UUID	Snapshot of where the goods should go.
12	BillAddressId	UUID	Snapshot of the billing verification address.
13	Currency	VARCHAR(3)	ISO 4217 code (e.g. 'USD').

Table 58. LineItems table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	OrderId	UUID	Foreign Key to Parent Order.
3	VariantId	UUID	Link to the source product (for restocking).
4	Quantity	INT	Number of units purchased.
5	PriceCents	BIGINT	Fixed price per unit at checkout time. IMMUTABLE.
6	SnapshotName	VARCHAR(255)	Fixed product name at checkout time. IMMUTABLE.

2.6.4.2 Fulfillment (Shipments & Granular Inventory)

The system decouples "Selling" (Order) from "Shipping" (Shipment). This supports split shipments (e.g., items coming from different warehouses).

- **Granular Tracking:** A key innovation is the `InventoryUnit` table. If a user buys 3 iPhones, the generic `LineItem` says "Qty: 3". However, the system generates 3 distinct `InventoryUnit` records. This allows tracking the specific Serial Number of **each** phone sent, enabling precise warranty support and returns management.

Table 59. Shipments table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	OrderId	UUID	Parent Order.
3	StockLocationId	UUID	The Warehouse fulfilling this package.
4	TrackingNumber	VARCHAR(100)	Carrier tracking code (e.g. UPS/FedEx).
5	State	INT	Logistics State: Pending *to* Picked *to* Packed *to* Shipped.
6	CostCents	BIGINT	The actual cost incurred to ship this package.
7	PickedAt	TIMESTAMP	Audit: When items left the shelf.
8	PackedAt	TIMESTAMP	Audit: When items were boxed.
9	ShippedAt	TIMESTAMP	Audit: When carrier accepted the package.

Table 60. InventoryUnits table (Granular Tracking)

No	Field	Type	Description
1	Id	UUID	Primary Key
2	OrderId	UUID	The Order requiring this item.
3	VariantId	UUID	The SKU being tracked.
4	ShipmentId	UUID	The specific box this unit was packed into.
5	State	INT	Unit Status: Pending, OnHand (Reserved/Allocated), Shipped, Returned.
6	SerialNumber	VARCHAR(50)	Unique hardware ID scanned during packing.

2.6.4.3 Financial Reconciliation (Payments)

Payments are tracked separately to allow for complex scenarios like “Authorization & Capture” (reserving funds before shipping) or split payments (Gift Card + Credit Card).

Table 61. Payments table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	OrderId	UUID	Parent Order.
3	AmountCents	BIGINT	The portion of the total covered by this transaction.
4	State	INT	Payment State: Pending *to* Authorized *to* Captured.
5	Provider	VARCHAR(50)	Payment Processor (e.g. 'Stripe').
6	TransactionId	VARCHAR(100)	Gateway's internal reference ID.
7	GatewayErrorCode	VARCHAR(50)	Raw error code for debugging failed transactions.
8	AuthorizedAt	TIMESTAMP	Timestamp of funds reservation.
9	CapturedAt	TIMESTAMP	Timestamp of funds transfer.

2.6.4.4 Audit Trail (History)

To comply with Enterprise requirements, every significant action is logged. This provides a "Flight Recorder" for the order.

Table 62. OrderHistory table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	OrderId	UUID	Parent Order.
3	Description	TEXT	Human-readable log (e.g. 'Order placed by User').
4	FromState	VARCHAR(50)	State transition start.
5	ToState	VARCHAR(50)	State transition end.
6	TriggeredBy	VARCHAR(100)	The user or system component responsible.
7	Context	JSONB	Structured metadata (e.g. Webhook payload) for debugging.

2.6.5 Inventory Context

The **Inventory Context** manages the physical availability of goods across the supply chain. Unlike simple "counter" systems, this implementation uses an **Event Sourcing-lite** architecture to provide ERP-grade auditability and multi-warehouse support.

2.6.5.1 Warehousing

The system supports multiple physical locations (Warehouses, Stores, Dropshippers). Each location operates independently, allowing for complex fulfillment strategies (e.g., "Ship from nearest store").

Table 63. StockLocations table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	Name	VARCHAR(100)	Public name (e.g. 'New York Fulfillment Center').
3	Code	VARCHAR(20)	Internal logistics code (e.g. 'NYC-01'). Unique and utilized on shipping labels.
4	IsActive	BOOL	If false, no new orders are routed here.
5	IsDefault	BOOL	If true, this location acts as the fallback for resolving stock queries.
6	Presentation	VARCHAR(100)	Customer-facing localized name.
7	Type	INT	0=Warehouse, 1=RetailStore, 2=Transit. Determines fulfillment capabilities.
8	AddressId	UUID	Link to the physical address for shipping calculations.
9	IsDeleted	BOOL	Soft delete flag to preserve historical data.

2.6.5.2 Inventory State (Read Model)

The `StockItems` table represents the **current** state of inventory. It is a cached projection designed for high-speed reads during the checkout process (e.g., checking "Is X in stock?" takes milliseconds).

- **Optimistic Concurrency:** Because inventory is a high-contention resource (thousands of users buying the same item on Black Friday), this table relies heavily on the `RowVersion` (xmin) column. If two processes try to decrement stock simultaneously, the database ensures serialized access, preventing "overselling".

Table 64. StockItems table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	VariantId	UUID	The product variant being stored.
3	StockLocationId	UUID	The warehouse where it is stored.
4	Sku	VARCHAR(100)	Denormalized SKU for quicker lookup during scans.
5	QuantityOnHand	INT	Physical units currently on the shelf involved in availability calculations.
6	QuantityReserved	INT	Units "soft-locked" by active carts but not yet shipped.
7	Backorderable	BOOL	If true, allows sales even when `QuantityOnHand` is zero.
8	BackorderLimit	INT	Safety floor for negative inventory (e.g., −100).
9	RowVersion	BYTEA	PostgreSQL System Column (xmin). Used to detect concurrent modifications.
10	IsDeleted	BOOL	Soft delete flag.

2.6.5.3 Inventory Ledger (Write Model / Source of Truth)

The `StockMovements` table is an **Immutable Ledger**. It is the single source of truth for **why** inventory counts changed.

- **Traceability:** You never simply "set stock to 5". You must insert a movement record: "Received 5 units from Supplier PO123".
- **Reversibility:** Mistakes (e.g., accidental adjustment) are corrected by inserting an **inverse** movement, preserving the history of the error.
- **Valuation:** It tracks `UnitCost` at the time of movement, enabling precise "Cost of Goods Sold" (COGS) reporting (FIFO/LIFO capability).

Table 65. StockMovements table

No	Field	Type	Description
1	Id	UUID	Primary Key
2	StockItemId	UUID	Link to the aggregate stock record.
3	Type	INT	Discriminator: Adjustment, Sale, Receipt, Return, StockCount.
4	Quantity	INT	The delta value (positive for in-flow, negative for out-flow).
5	BalanceBefore	INT	Audit Snapshot: Count before this tx.
6	BalanceAfter	INT	Audit Snapshot: Count after this tx. Ensures mathematical integrity.
7	UnitCost	DECIMAL(18,2)	Financial value of this specific batch of items.
8	Reason	VARCHAR(255)	User-provided explanation (e.g. 'Found during stock take').
9	Reference	VARCHAR(100)	External document ID (Order Number, RMA Number, Purchase Order).
10	CreatedAt	TIMESTAMP	The exact moment the stock moved. Immutable.

2.6.6 Indexing Strategy

To support the specific latency requirements defined in the functional requirements, specialized indexing strategies are applied:

1. **HNSW Index for Vectors:** The core visual search relies on Hierarchical Navigable Small World (HNSW) graphs.

```
CREATE INDEX idx_embeddings_fashion_clip ON image_embeddings
USING hnsw (vector vector_cosine_ops)
WHERE model_name = 'fashion_clip';
```

2. **Partial Indexes:** To reduce index size and improve insert performance, business-critical queries are optimized with partial constraints. `CREATE INDEX idx_products_active ON products (status) WHERE status = 'Active';`

3. **Unique Constraints:** Data integrity is enforced at the database level to prevent duplicate logic errors. `UNIQUE (slug)` on Taxonomies, Taxons, Products.

2.7 API DESIGN

The system exposes a comprehensive RESTful API built on **ASP.NET Core Minimal APIs** and **Carter**. This approach reduces the boilerplate associated with traditional MVC controllers while maintaining full support for dependency injection, authorization, and OpenAPI specification generation.

2.7.1 Architecture

The API layer acts as a thin orchestration boundary. It does not contain business logic; instead, it delegates all processing to the **MediatR** pipeline described in the Application Core.

- **Routing:** Implemented via `ICarterModule` to group related endpoints (e.g., `ProductsModule`).
- **Serialization:** Uses `System.Text.Json` with snake_case naming policies to align with standard web practices.
- **Documentation:** Automatically generated OpenAPI v3 (Swagger) definitions.

2.7.2 Unified Response Structure

To ensure consistent consumption by the frontend (Vue 3) and mobile clients, all API responses adhere to a unified envelope structure or the **Problem Details for HTTP APIs** (RFC 7807) standard for errors.

The `ApiResponse<T>` wrapper enables consistent handling of metadata, such as pagination.

```
{
  "data": {
    "items": [ ... ],
    "totalCount": 150
  },
  "success": true,
  "message": "Request processed successfully."
}
```

2.7.3 Implementation Pattern

Each feature exposes its own module. Below is the implementation of the **Product Management** API, demonstrating the rigorous security checks and command dispatching pattern.

```
public class ProductsModule : ICarterModule
{
    public void AddRoutes(IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/api/admin/catalog/products")
            .WithTags("Products")
            .RequireAuthorization();

        // 1. Query Handling (Read)
        group.MapGet("/",
            async ([AsParameters] GetProductsPagedList.Request request,
ISender sender) =>
            {
                var result = await sender.Send(new
GetProductsPagedList.Query(request));
                return Results.Ok(ApiResponse.Paginated(result));
            })
        .RequireAccessPermission(FeaturePermissions.Admin.Catalog.Product.List);

        // 2. Command Handling (Write)
        group.MapPost("/",
            async ([FromBody] CreateProduct.Request request, ISender
sender) =>
            {
                var result = await sender.Send(new
CreateProduct.Command(request));
                return result.ToApiCreatedResponse(x => $"/api/catalog/
products/{x.Id}");
            })
        .RequireAccessPermission(FeaturePermissions.Admin.Catalog.Product.Create);

        // 3. State Management (Patch)
        group.MapPatch("/{id:guid}/activate",
            async (Guid id, ISender sender) =>
            {
                var result = await sender.Send(new
ActivateProduct.Command(id));
                return result.ToApiResponse();
            })
        .RequireAccessPermission(FeaturePermissions.Admin.Catalog.Product.ManageStat
    }
}
```

2.7.4 Security & Authorization

The API enforces a **Zero Trust** security model at the endpoint level.

1. **Authentication:** All non-public routes require a valid JWT Bearer token via `.RequireAuthorization()`.
2. **Fine-Grained Permissions:** The custom extension `.RequireAccessPermission(...)` validates that the user holds specific claims

(e.g., `catalog.product.create`) before the handler is even invoked. This prevents unauthorized execution of expensive business logic.

2.7.5 Error Handling

Failures are converted from the Domain's `ErrorOr` result pattern into standardized HTTP responses.

- **Domain Validation Error:** Returns `400 Bad Request` with field-level details.
- **Result.NotFound:** Returns `404 Not Found`.
- **Result.Unauthorized:** Returns `403 Forbidden`.
- **WaitMsBeforeAsync:** Configured for background tasks to prevent client timeouts.

2.8 IMPLEMENTATION AND USER INTERFACE

This section details the concrete realization of the system architecture and its corresponding user interfaces across the primary components: the Machine Learning Service, the .NET Backend, the Vue.js Frontend, and the Administration Panel.

2.8.1 Machine Learning Service

The **Machine Learning (ML) Service** is a specialized AI microservice responsible for visual intelligence tasks. It handles the computationally intensive tasks of image embedding generation and recommendation logic, exposing endpoints consumed by the .NET Backend via HTTP.

Before diving into the implementation details, it is essential to understand the data flow within the service. The following diagram illustrates the complete **Inference Pipeline**, tracking a request from the HTTP interface through the validation layer, into the lazy-loading manager, and finally to the GPU for execution.

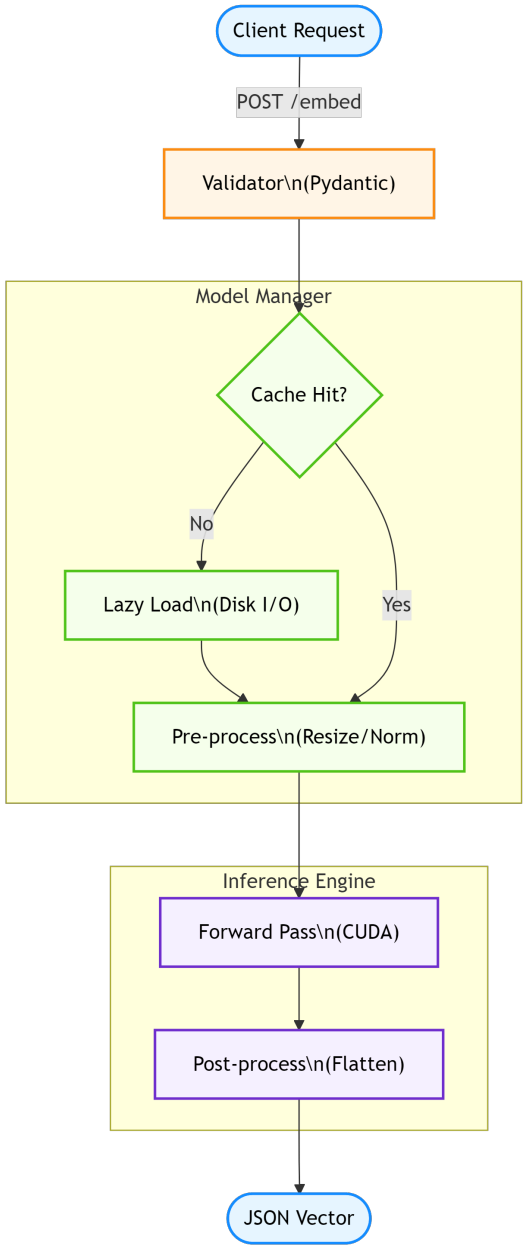

Figure 33. ML Inference Implementation: End-to-end data flow tracking the request through internal service components.

- **Runtime: Python 3.12**.
- **Framework: FastAPI** (Async web framework) run with **Uvicorn**.
- **Key Models:**
  - **CLIP (ViT-B/32):** For generating shared latent space embeddings for images and text, powering the visual search.
  - **EfficientNet-B0:** Utilized for feature extraction benchmarks.
- **Libraries:**
  - **PyTorch (Torch):** Core deep learning framework.
  - **TIMM (PyTorch Image Models):** Access to state-of-the-art vision backbones.

  - **Pillow:** Image data preprocessing.
- **Integration:** Deployed as a Docker container orchestrator by **.NET Aspire**.

Decoupling this service from the primary .NET backend allows for independent scaling of GPU-intensive workloads and simplifies dependency management for Python-native AI libraries.

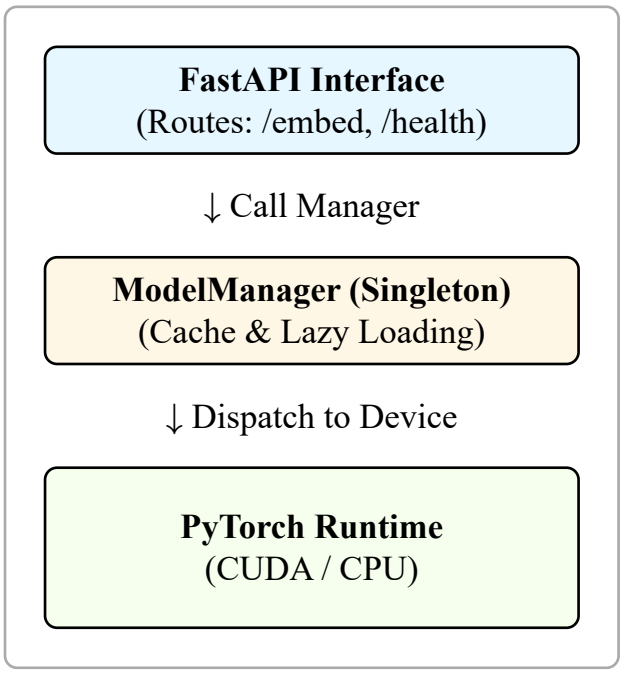

Figure 34. ML Service Internal Architecture: Layered design separating API handling from Model Lifecycle management.

2.8.1.1 Architecture

The service operates as a stateless HTTP API. It consumes image URLs and returns high-dimensional vector embeddings.

- **Framework:** FastAPI (Asynchronous, High-Performance).
- **Runtime:** PyTorch with CUDA support (for NVIDIA GPUs) or MPS (for Apple Silicon).
- **Orchestration:** Managed via .NET Aspire service discovery (`http://ml-service`).
- **Security:** Protected by `X-API-Key` authentication.
- **Containerization:** Deployed as a Docker container with pre-installed CUDA drivers to ensure consistent runtime environments across development and production.

2.8.1.2 Service Architecture (Lazy Loading)

To optimize resource usage, the service employs a **Lazy Loading** strategy via a Singleton `ModelManager`. Deep learning models are only loaded into GPU memory upon their first request. This prevents the service from consuming gigabytes of VRAM at startup for models that may not be immediately needed.

```
class ModelManager:
    _instance = None
    _embedders = {} # Cache for loaded models

    def get_embedder(self, model_name):
        if model_name in self._embedders:
            return self._embedders[model_name]

        // Lazy load on first request
        if model_name == "dinov2_vits14": model = load_dino_model()
        elif model_name == "fashion_clip": model = load_fashion_clip()

        self._embedders[model_name] = model
        return model
```

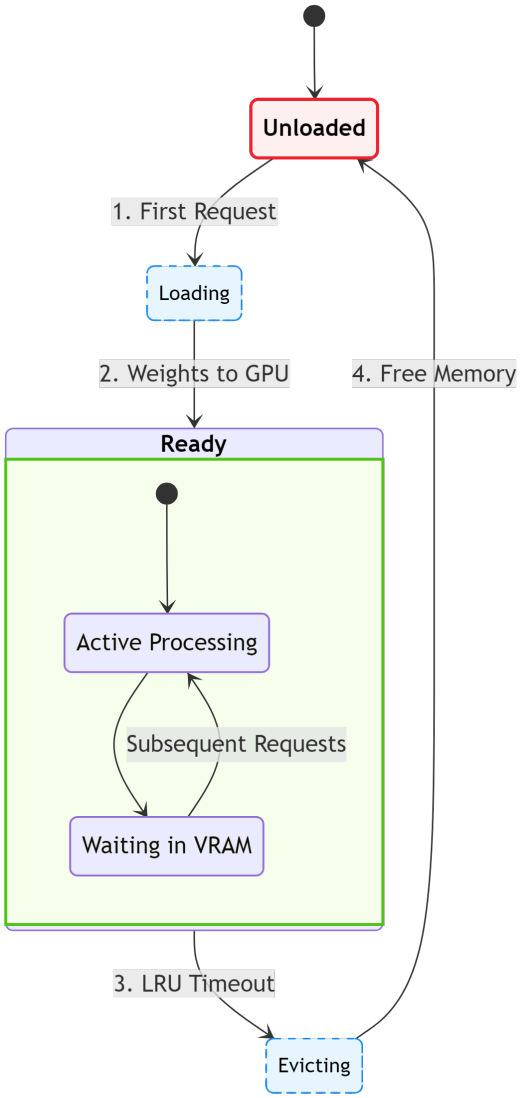

Figure 35. Lazy Loading State Machine: Visual tracking of the Model Manager's lifecycle states (Unloaded *to* Ready *to* Evicted).

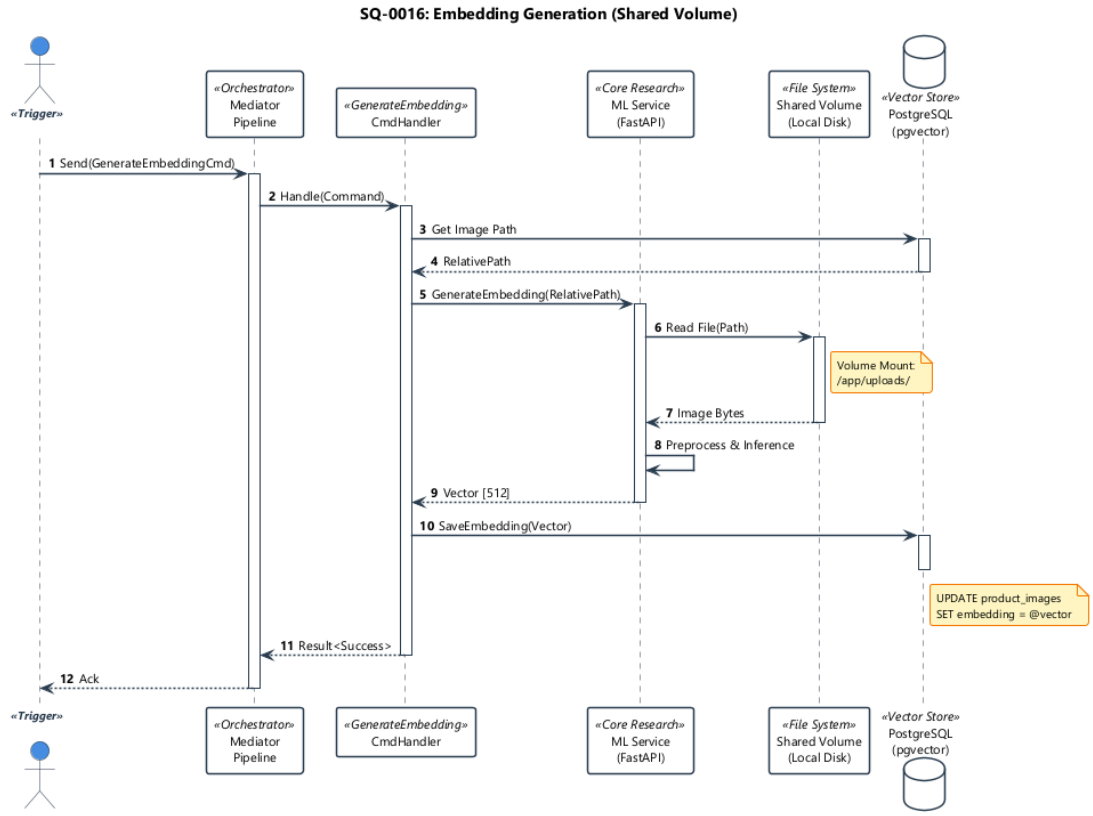

Figure 36. Vector Generation Sequence: The asynchronous pipeline from image reception to embedding storage.

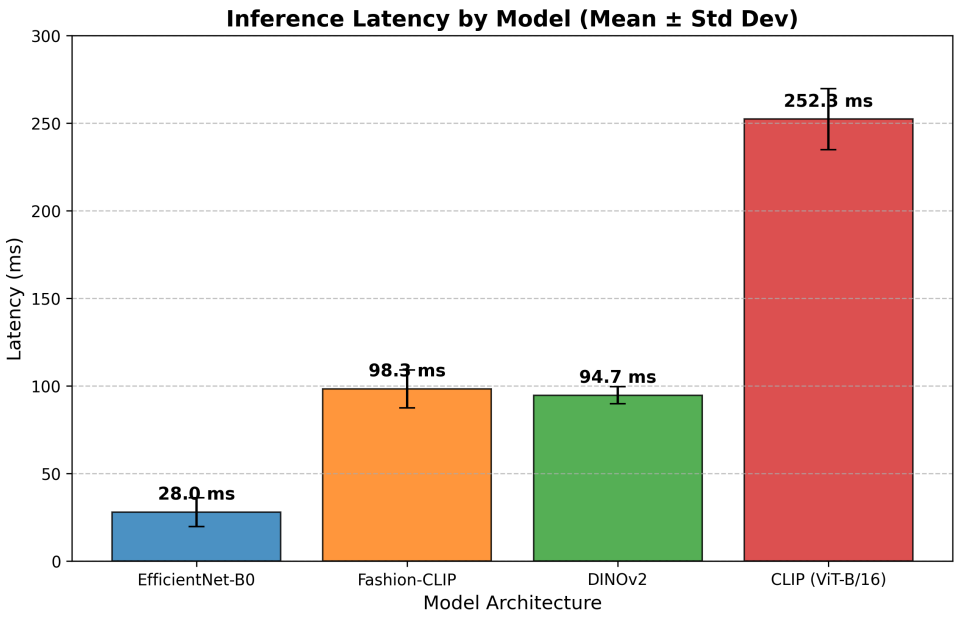

Figure 37. Performance Metrics: Inference latency distribution for Fashion-CLIP (Mean: 98.3ms, P95: 115.9ms). Data derived from Thesis Validation Benchmark (N=1000).

2.8.1.3 Functional Interface

The service operates through a granular, model-specific interface. Orchestration of multiple models is managed upstream by the Backend or through asynchronous task processors, ensuring that the ML service remains focused on the computational task of inference.

The primary interface allows for the generation of high-dimensional vector embeddings from provided image data. Upon receiving a processing request, the service:

1. Normalizes the input image according to the requirements of the selected model.
2. Executes the forward pass through the deep learning architecture.
3. Returns a standardized response containing the numerical embedding, its dimensions, and metadata regarding the processing time.

2.8.1.4 Health and Monitoring

To support reliable operation within a containerized environment, the service implements comprehensive health monitoring. This system verifies not only the availability of the web interface but also the operational status of the underlying hardware and AI models:

1. **Hardware Readiness:** Confirms that the required GPU (CUDA) environment is active and accessible.
2. **Model Integrity:** Verifies that the primary models are correctly loaded into memory and ready for inference.
3. **Execution Sanity:** Periodically runs a baseline inference task to ensure the entire pipeline, spanning from input preprocessing to output generation, is functioning correctly.

This proactive monitoring allows the system orchestrator to automatically detect and recover from hardware or runtime failures without manual intervention.

2.8.1.5 Model Zoo

The service implements a **Strategy Pattern** via a `ModelManager`, supporting a diverse ensemble of models to capture different visual aspects:

1. **EfficientNet-B0 (1280-dim):** Baseline CNN model for capturing structural silhouettes and low-level features.
2. **ConvNeXt-Tiny (768-dim):** Modern hierarchical CNN that bridges the gap between Transformers and CNNs.
3. **CLIP (ViT-B/16) (512-dim):** General-purpose semantic search model trained on 400M image-text pairs.
4. **Fashion-CLIP (512-dim):** Domain-adapted CLIP fine-tuned on fashion imagery for understanding specific attributes (e.g., “A-line skirt”).
5. **DINOv2 (ViT-S/14) (384-dim):** Self-supervised Vision Transformer that excels at geometric matching and “Visual Similarity” without semantic bias.

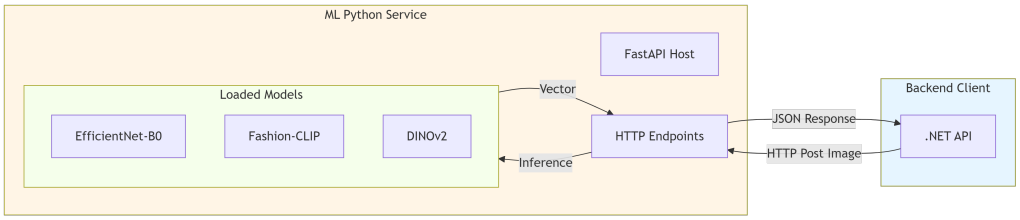

Figure 38. Model Ensemble Diagram. Visual representation of the Model Zoo strategy, showing how different architectures (CNN, ViT, Hybrid) contribute distinctive feature vectors.

2.8.2 Backend Implementation

The backend is a high-performance REST API built on **.NET 10** using a strict **Vertical Slice Architecture (VSA)**. Adopting VSA allows the system to modularize features by business capability rather than technical layers, ensuring maintainability and scalability.

- **Runtime: .NET 10** (ASP.NET Core).
- **Architecture: Vertical Slice Architecture** using **Carter** (Minimal APIs) for routing and **MediatR** for decoupling request handling (CQRS pattern).
- **Data Access: Entity Framework Core** with **PostgreSQL**.
  - **PgVector:** Utilizes the `pgvector` extension for storing and querying 512-dimensional embeddings.
- **Observability: OpenTelemetry** and **Serilog** for distributed tracing.
- **Documentation: Scalar.AspNetCore** for interactive OpenAPI documentation.

The fundamental structure of the data layer is designed around the **Vertical Slice** philosophy. As shown in the Entity Relationship Diagram (ERD) below, the database schema is partitioned into contexts like `Catalog`, `Ordering`, and `Identity`. This separation ensures that logic belonging to one slice does not inadvertently mutate data belonging to another, maintaining a “Shared-Nothing” approach at the application layer.

```
// ReSys.Core/Data/ShopDbContext.cs
public class ShopDbContext : DbContext, IApplicationDbContext
{
    public ShopDbContext(DbContextOptions<ShopDbContext> options) :
base(options) {}

    // Domain Entities partitioned by Business Context
    public DbSet<Product> Products => Set<Product>();           //
Catalog Context
    public DbSet<Order> Orders => Set<Order>();                 //
Ordering Context
    public DbSet<StockItem> StockItems => Set<StockItem>();     //
Inventory Context
    public DbSet<User> Users => Set<User>();                    //
Identity Context

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Applies specific configurations (Keys, Indexes, Constraints)
modelBuilder.ApplyConfigurationsFromAssembly(typeof(ShopDbContext).Assembly);
        base.OnModelCreating(modelBuilder);
    }
}
```

Listing 2: ShopDbContext Implementation: The single database context aggregates DbSets from all business domains, corresponding to the schema defined in Figure 31.

This implementation directly maps the architectural concepts defined in Section 2.5.3.2 to the project structure. As shown below, features are isolated into their own directories, containing all necessary logic (Commands, Handlers, Validators).

```
src/
  ReSys.Core/
    Features/                          // Vertical Slices
      Catalog/
        CreateProduct/
          CreateProductCommand.cs    // Request (DTO)
          CreateProductHandler.cs    // Logic (CQRS)
          CreateProductValidator.cs  // Validation
      Ordering/
        PlaceOrder/
          PlaceOrderCommand.cs
          PlaceOrderHandler.cs
```

Listing 3: Project Directory Structure: Implementation of Vertical Slice Architecture where code is co-located by feature.

The internal structure of the API layer follows the principles of **Minimal APIs** in .NET 10, utilizing **Carter** for route registration and **MediatR** for executing business logic within the vertical slices. This ensures a clean, lightweight entry point for all system interactions.

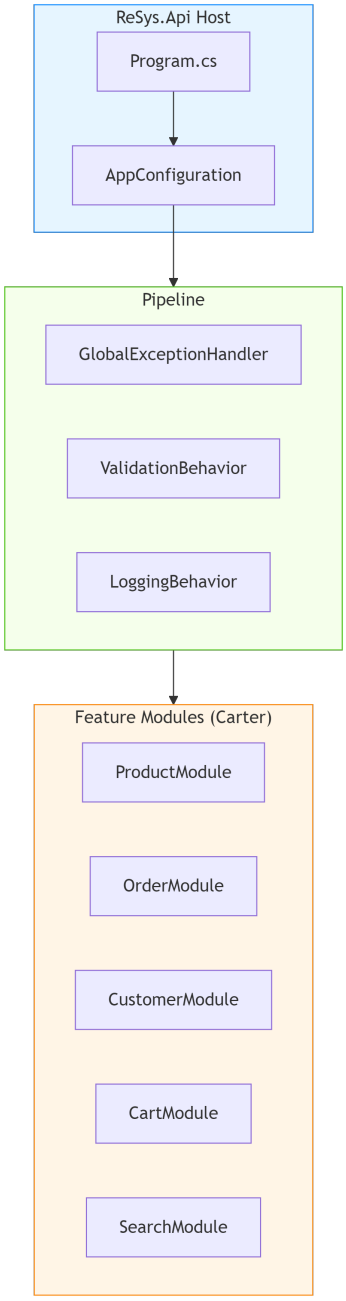

Figure 39. API Service Internal Structure: Orchestration of Minimal APIs, Carter modules, and MediatR handlers.

2.8.2.1 Service Orchestration & Observability

The backend operates as a distributed system where services are loosely coupled but highly observable.

- **Service Discovery:** Environment-based configuration manages connection strings for databases (PostgreSQL) and message brokers (Redis), ensuring seamless transitions between development and production environments.

- **Telemetry:** Comprehensive OpenTelemetry integration provides distributed tracing across the .NET API and Python ML Service, allowing full visibility into request latency and inter-service communication.

```
// Program.cs
// Service defaults configure OpenTelemetry, logging, and health checks
builder.AddServiceDefaults();
builder.AddPostgresHealthCheck("shopdb");
```

2.8.2.2 Request Pipeline & Middleware

The application utilizes **MediatR Behaviors** to implement cross-cutting concerns such as logging, validation, and error handling within the request pipeline. This Middleware pattern ensures that the core business logic handlers remain clean and focused on their specific task.

Validation Pipeline: Before a handler executes, the `ValidationBehavior` intercepts the request, runs all defined `FluentValidation` rules, and returns a structured error response if validation fails.

```
public class ValidationBehavior<TRequest,
TResponse>(IValidator<TRequest>? validator)
    : IPipelineBehavior<TRequest, TResponse>
{
    public async Task<TResponse> Handle(...) {
        if (validator == null) return await next();

        var result = await validator.ValidateAsync(request);
        if (result.IsValid) return await next();

        return (dynamic)result.Errors.ConvertAll(e =>
Error.Validation(...));
    }
}
```

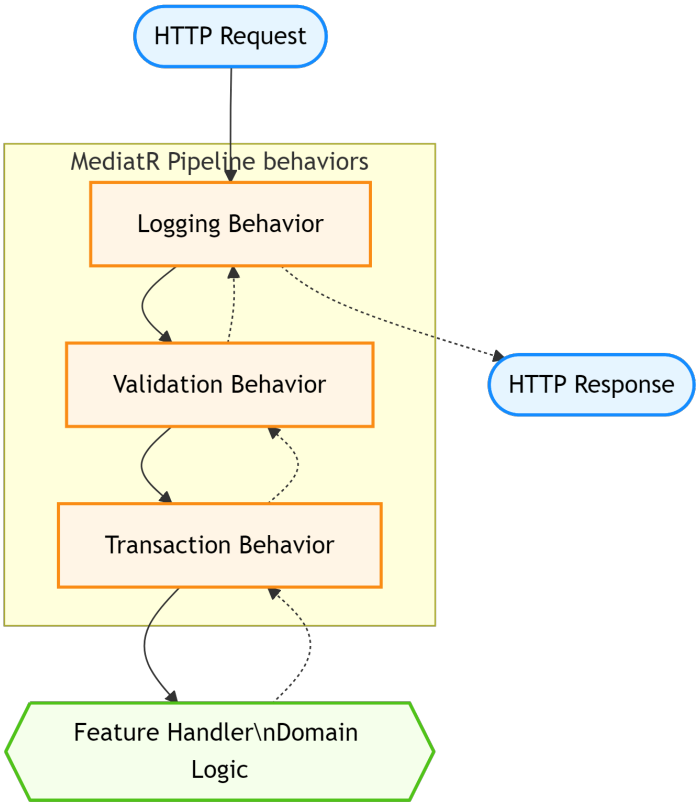

Figure 40. MediatR Request Pipeline: Visual tracking of the “Onion Architecture” flow where behaviors wrap the core handler.

2.8.2.3 Key Feature Implementations

The following subsections detail specific technical achievements within the backend.

2.8.2.3.1 Product Images & Vectorization

To support the multi-model AI strategy, the database uses a flexible **One-to-Many** relationship for storing embeddings.

```
public class ProductImage : Aggregate {
    public Guid ProductId { get; set; }
    public string Url { get; set; }
    public ICollection<ImageEmbedding> Embeddings { get; set; }
}

public class ImageEmbedding : Entity {
    public string ModelName { get; set; } // e.g., "fashion_clip"
    public Vector Vector { get; set; } // pgvector column, 512-dim
}
```

To optimize high-dimensional search performance, we implement a **Hierarchical Navigable Small World (HNSW)** index on the `Vector` column (Figure 5). The HNSW algorithm allows the system to perform approximate nearest neighbor (ANN) searches

with sub-linear time complexity, ensuring that search results are retrieved in milliseconds even as the catalog grows to millions of items.

Asynchronous Vector Generation:

- **Logic Flow (Webhook):** The UI does not wait for vectorization. The upload finishes immediately. Later, the frontend receives a WebSocket notification (or polls an endpoint) to "Light Up" the semantic search capability for that product.
- **Sequence Flow:** Figure 41 shows the decoupling. The `ML Service` processes the image on a GPU-optimized node. Upon completion, it calls back to the Core API (`POST /callbacks/embedding-complete`) to update the `ImageEmbedding` entity.

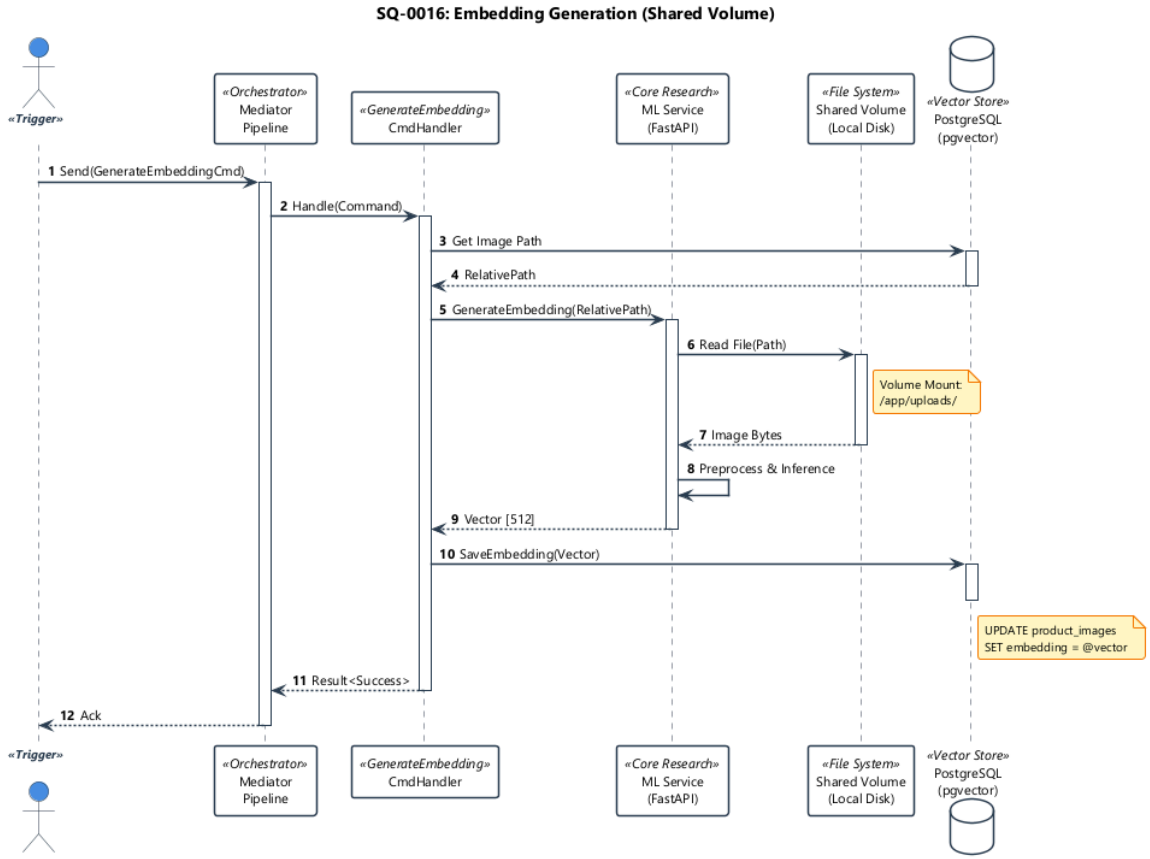

Figure 41. Embeddings Generation: The pipeline for converting product images into 512-dimensional vectors (UC-0016).

Graph Maintenance:

- **Logic Flow (Admin Trigger):** While primarily a scheduled background task (e.g., 03:00 UTC), this process exposes a manual "Rebuild Index" control in the **System Admin Console** to allow operators to force immediate consistency after bulk imports.
- **Sequence Flow:** Figure 42 illustrates the `IVFFlat` (or HNSW) build process. This compute-intensive operation is offloaded to a read-replica or executed during maintenance windows to avoid locking the `vector` table during high-write periods.

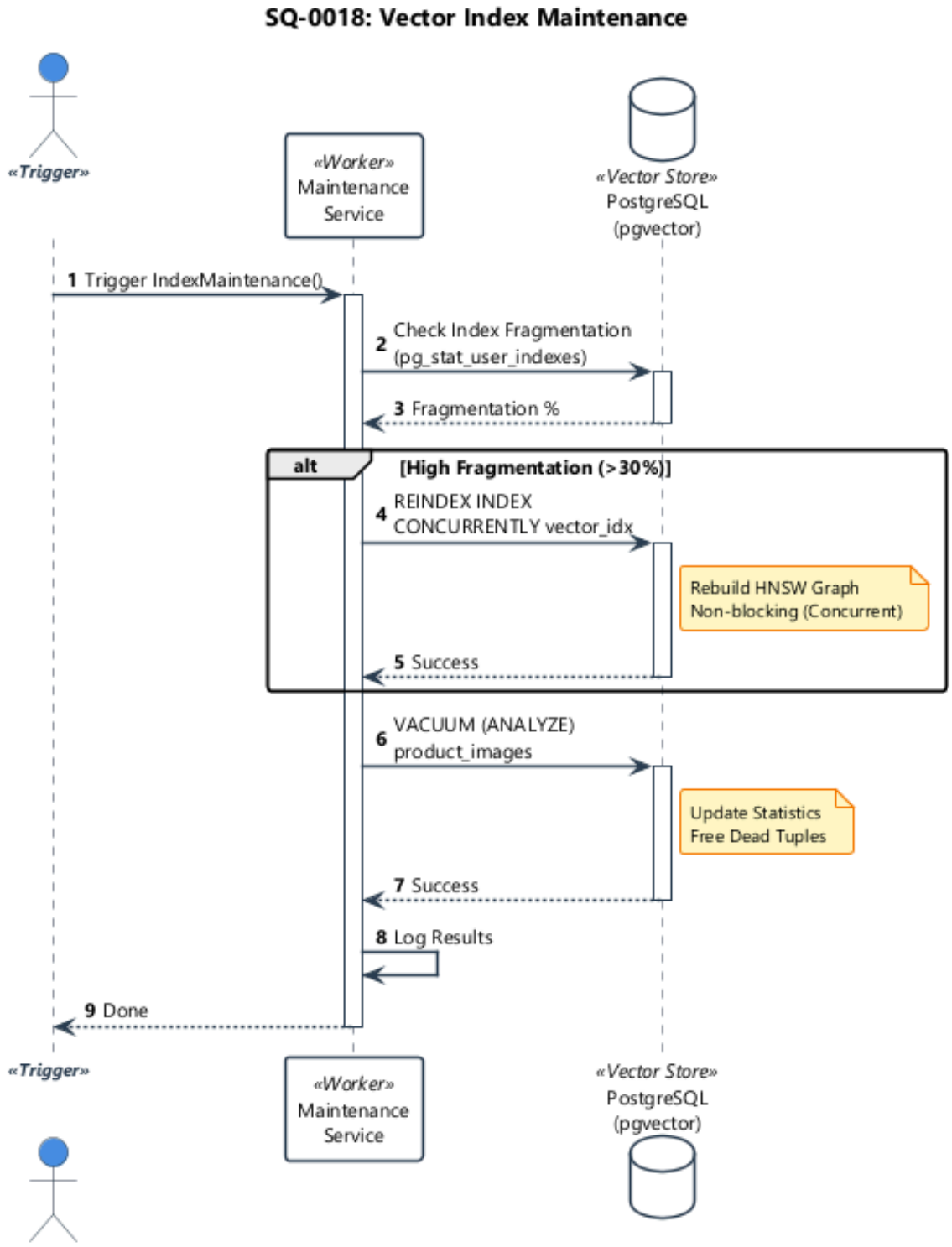

Figure 42. Index Construction: Building the HNSW graph structures for efficient nearest-neighbor lookup (UC-0018).

2.8.2.3.2 Recommendations API (Clean Architecture)

The `RecommendationsModule` exposes endpoints via **Carter** modules, delegating logic to the Domain via MediatR. This separates the **Transport Mechanism** (HTTP) from the **Application Logic**.

```
// POST /api/storefront/recommendations/by-product/{id}
app.MapGet("/by-product/{productId:guid}", async (ISender sender, ...)
=>
{
    var query = new GetProductRecommendations.Query(productId,
"fashion_clip", TopK: 10);
    return await sender.Send(query);
});
```

2.8.2.3.3 Atomic Stock Reservation (Concurrency Control)

The `CheckoutService` uses **Serializable** database transactions to strictly order conflicting inventory updates, preventing overselling during high-traffic events.

1. Start Transaction.
2. Lock `InventoryItem` row (Database-level row lock).
3. Check `QuantityAvailable > RequestedQuantity`.
4. Update `QuantityReserved += RequestedQuantity`.
5. Commit Transaction.

If two customers attempt to buy the last item simultaneously, the database lock ensures they are processed sequentially, and the second request fails gracefully.

The atomic stock reservation process is critical for preventing overselling. When the `PlaceOrderCommand` is dispatched, the system initiates a **Serializable** database transaction. This isolation level ensures that the check-and-reserve logic is strictly atomic. The handler first executes a “Select for Update” query on the `InventoryItem` record, effectively acquiring an exclusive row-level lock. Only after the lock is secured does the logic verify availability; if sufficient, the `QuantityReserved` is incremented, and the transaction is committed. Any competing transaction attempting to lock the same row is forced into a WAIT state by the RDBMS until the first transaction completes, guaranteeing absolute inventory consistency under high concurrent load.

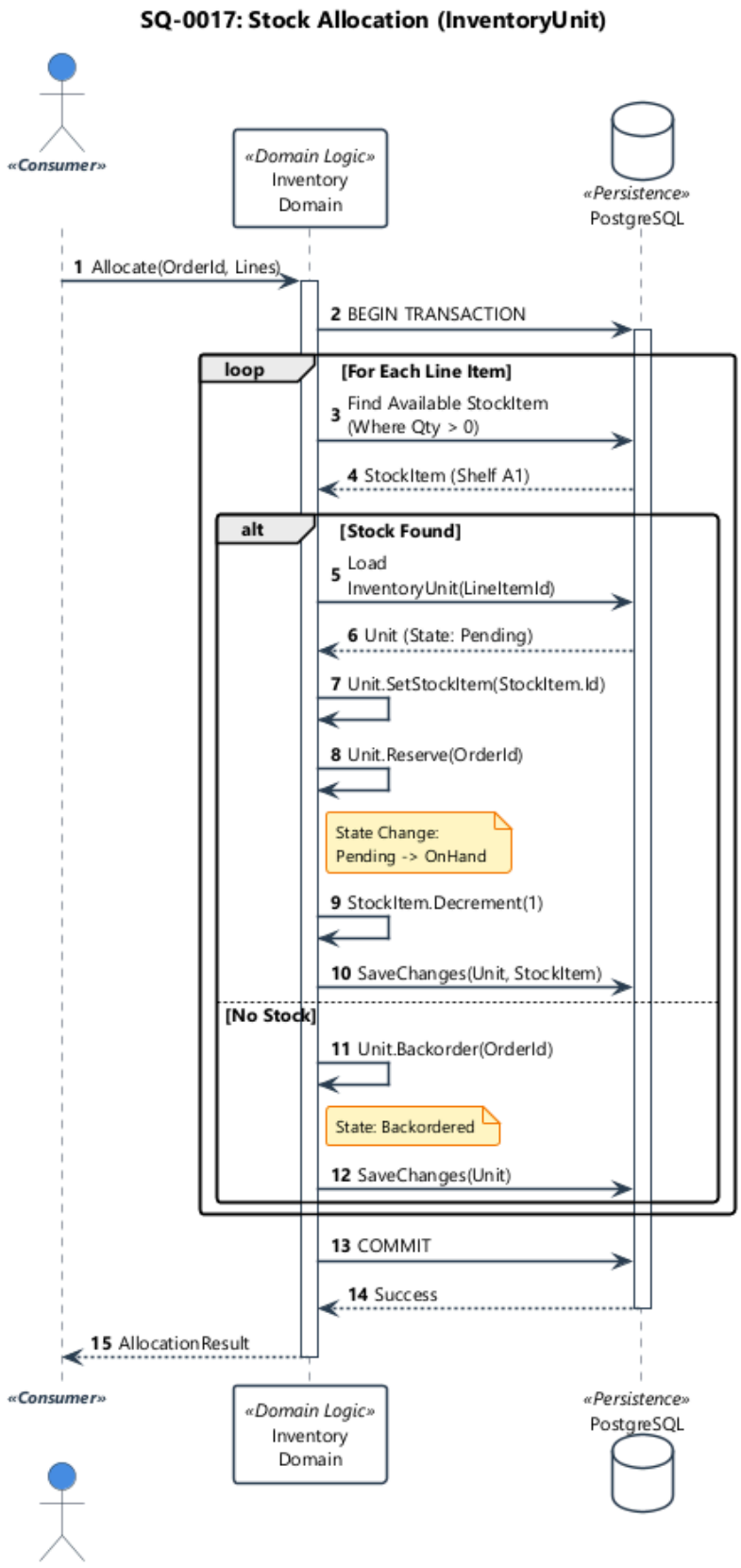

Figure 43. Atomic Stock Reservation Sequence: Verifying availability and locking inventory rows within a Serializable transaction.

From a User Experience perspective, this backend locking mechanism is visualized via a “Stock Hold Timer”. Once the system successfully acquires the reservation (lock), the user is presented with a countdown (e.g., “Items reserved for 10:00”), strictly informing them of the guaranteed window to complete the purchase. This transparency reduces anxiety and enforces the fairness of the “First-Come-First-Serve” event model.

Concurrency Verification:

- **UI Feedback (Timer):** From a User Experience perspective, this backend locking mechanism is visualized via a “Stock Hold Timer”. Once the system successfully acquires the reservation (lock), the user is presented with a countdown (e.g., “Items reserved for 10:00”), strictly informing them of the guaranteed window to complete the purchase.
- **Sequence Flow:** Referring to sequence Figure 43, the transaction logs would show a strict interleaving of `BEGIN` to `SELECT FOR UPDATE` to `COMMIT`. If a second request arrives at $T + 1$, the database engine blocks the `SELECT` until the first lock is released, creating a linearized history of stock deductions that is free of race conditions.

2.8.2.3.4 System Automation (Background Services)

The backend runs hosted services for critical maintenance tasks.

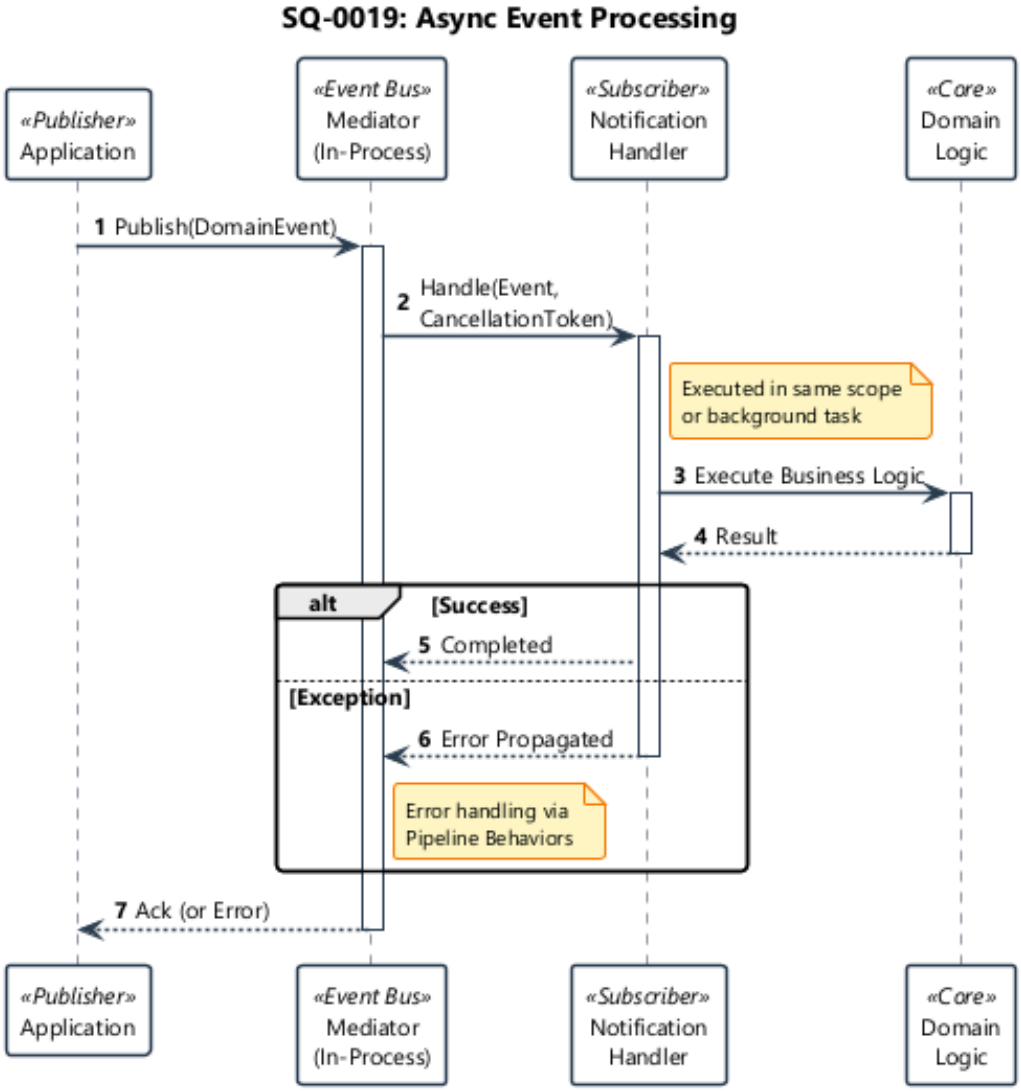

Figure 44. Low Stock Indicator Sequence: Background aggregation of inventory levels to trigger operational alerts (UC-0019).

Automated Alerting Flow:

- **UI Notification (Read Model):** The "Low Stock" widget allows administrators to subscribe to specific categories. It does not query the live `Inventory` table (which is hot). Instead, it polls a dedicated `Alerts` table.
- **Sequence Flow:** As shown in Figure 44, the system uses a **Materialized View** pattern. A background worker periodically aggregates inventory events and updates the `LowStockSnapshot` table. The UI simply reads this pre-computed snapshot, decoupled from the high-throughput inventory write path, ensuring zero impact on checkout performance.

2.8.3 Frontend Implementation

The Storefront provides the primary user interface for customers to browse the catalog, perform visual searches, and manage their shopping experience. Built as a Single Page Application (SPA), it emphasizes performance, interactivity, and a cohesive design system.

- **Framework: Vue 3** (Composition API) with **Vite 7** for next-generation build tooling.
- **Styling: Tailwind CSS 4** (Utility-first) combined with **PrimeVue 4** component library (Aura theme) for accessible, high-quality UI elements.
- **State Management: Pinia** for reactive, centralized state management (cart, user session, search results).
- **Key Dependencies:**
  - `vue-router` for client-side navigation.
  - `vee-validate` + `zod` for robust form validation.
  - `@stripe/stripe-js` for payment integration.
- **Distinctive Features:**
  - **Visual Search UI:** Integration with the camera/upload interfaces to capture images for similarity search.
  - **Real-time Notifications:** Dynamic updates for order status and cart actions.

The frontend is a **Single Page Application (SPA)** built with **Vue 3** and **TypeScript**, designed to deliver a native-like experience. It strictly separates **Presentation** (Components) from **Business Logic** (Composables/Stores).

2.8.3.1 Technical Foundation

The architecture leverages a modern toolchain to ensure performance and developer productivity.

- **Reactive Core: Vue 3 Composition API** allows logic reuse via "Composables", separating UI concerns from data fetching.
- **State Management: Pinia** stores provide reactive, Type-safe state containers accessible globally across the application.
- **Styling Engine: Tailwind CSS 4** ensures a lightweight utility-first design, while **PrimeVue 4** provides accessible, pre-built functional components.

2.8.3.2 Design System & UX

The application implements a cohesive design language to ensure consistency.

- **Typography: Inter** variable font (Sans-serif) for high legibility across device sizes.
- **Micro-interactions:** Standardized 200ms transitions on hover and focus states provide tactile feedback.
- **Optimistic UI:** State updates occur immediately in the interface (e.g., "Item Added") before waiting for server confirmation, reducing perceived latency.

2.8.3.3 Key Feature Implementations: ML & AI

The integration of Machine Learning features is a core differentiator, surfacing intelligence through intuitive UI components rather than hidden background processes.

2.8.3.3.1 1. Visual Search Module

The Visual Search implementation orchestrates a complex interaction converting user intent (an image) into vector queries.

Input Processing: A dedicated **Drag-and-Drop Input Component** handles the creation of the query. It manages the visual state (`isDragging`) to provide affordability and strictly validates file types on the client side before upload.

```
// Visual Search Input Flow
IF User drags file over drop-zone:
    SET state to "Dragging" (Highlight UI)
ON DROP:
    PREVENT default browser behavior
    VALIDATE file type is Image (JPG/PNG/WEBP)
    EMIT "Selection Event" with File Object
```

Similarity Intelligence: Search results are rendered with explicit **Confidence Badges**. The UI interprets the raw cosine distance returned by the backend to categorize results, giving users transparency into the AI's logic.

```
// Similarity Classification Logic
FUNCTION GetSimilarityBadge(score):
    IF score > 0.85: RETURN "Exact Match" (Green)
    IF score > 0.70: RETURN "Highly Similar" (Blue)
    ELSE: RETURN "Conceptual Match" (Yellow)
```

The following sequence diagram illustrates the end-to-end lifecycle of a visual search request.

Visual Inference Pipeline:

- **UI Action:** The interaction begins with a Drag-and-Drop event. The frontend immediately validates the MIME type and renders a local **Optimistic Preview** of the image. Crucially, while the heavy inference runs, the results grid displays a **Skeleton Loader** animation to manage perceived latency.
- **System Sequence:**
  1. **Upload:** The image is POSTed to the API Gateway.
  2. **Vectorization:** The backend delegates to the `ML Service` via HTTP (Figure 45).
  3. **Search:** The resulting 512-dim vector is used in a `pgvector` KNN query.
  4. **Response:** The Skeleton Loader is replaced by the hydrated list of visually similar products.

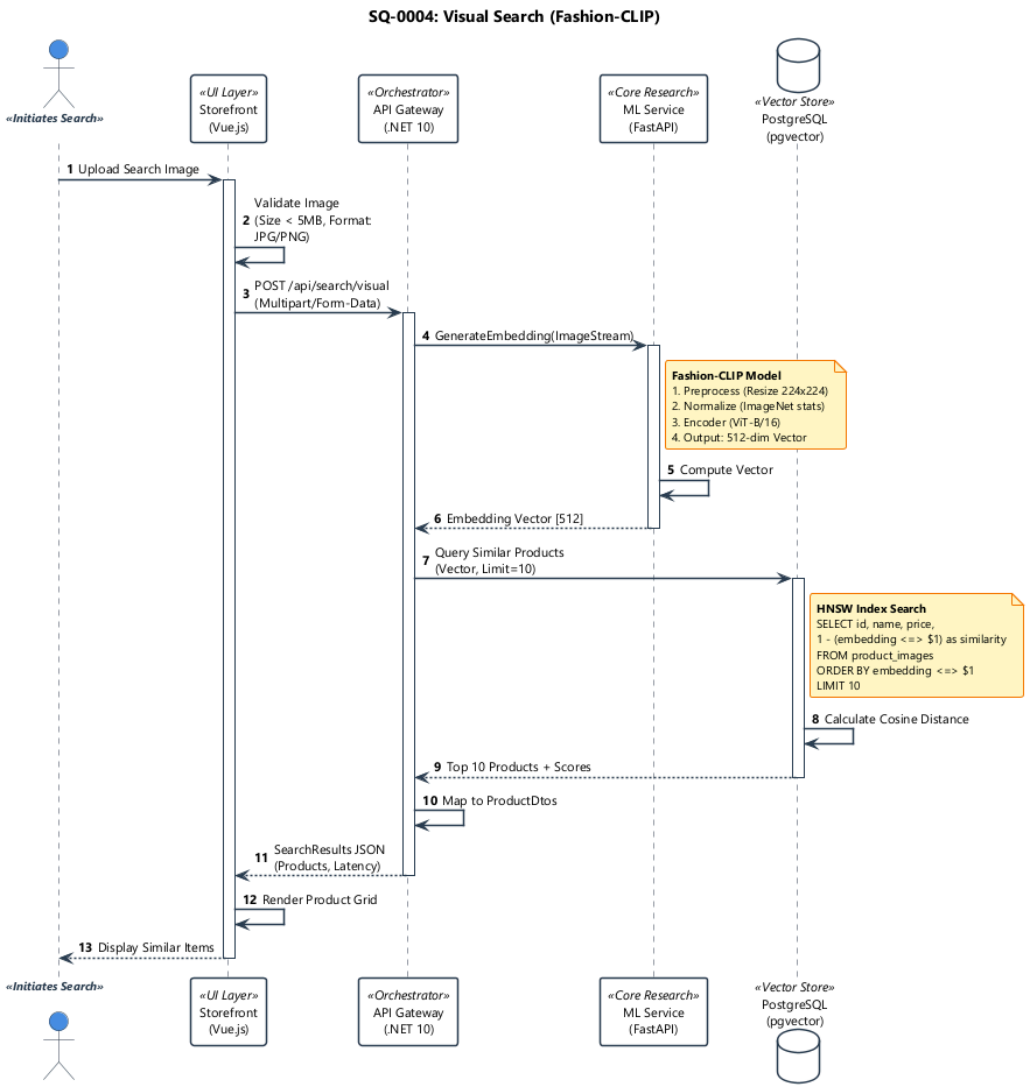

Figure 45. Visual Search Data Flow: End-to-End sequence from user interaction to AI inference and retrieval (UC-0004).

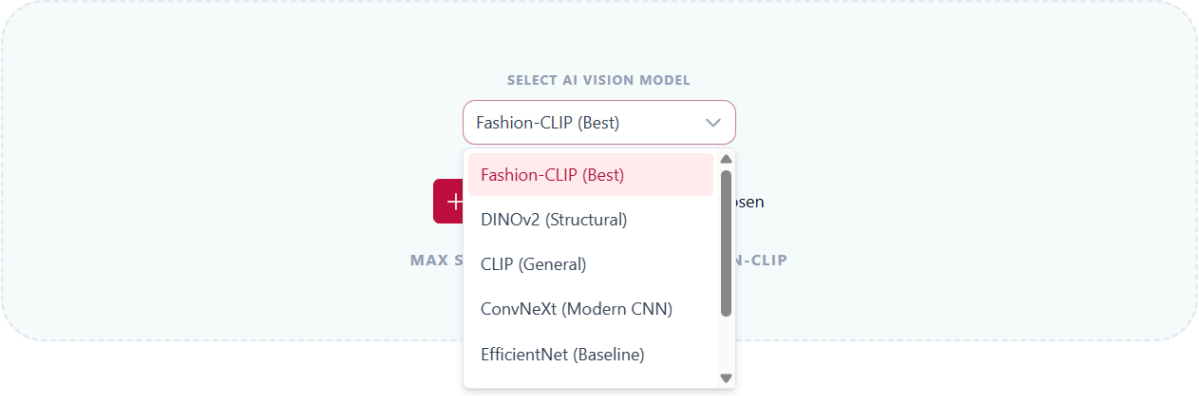

Figure 46. Visual Search UI: Interface for uploading images and selecting visual queries.

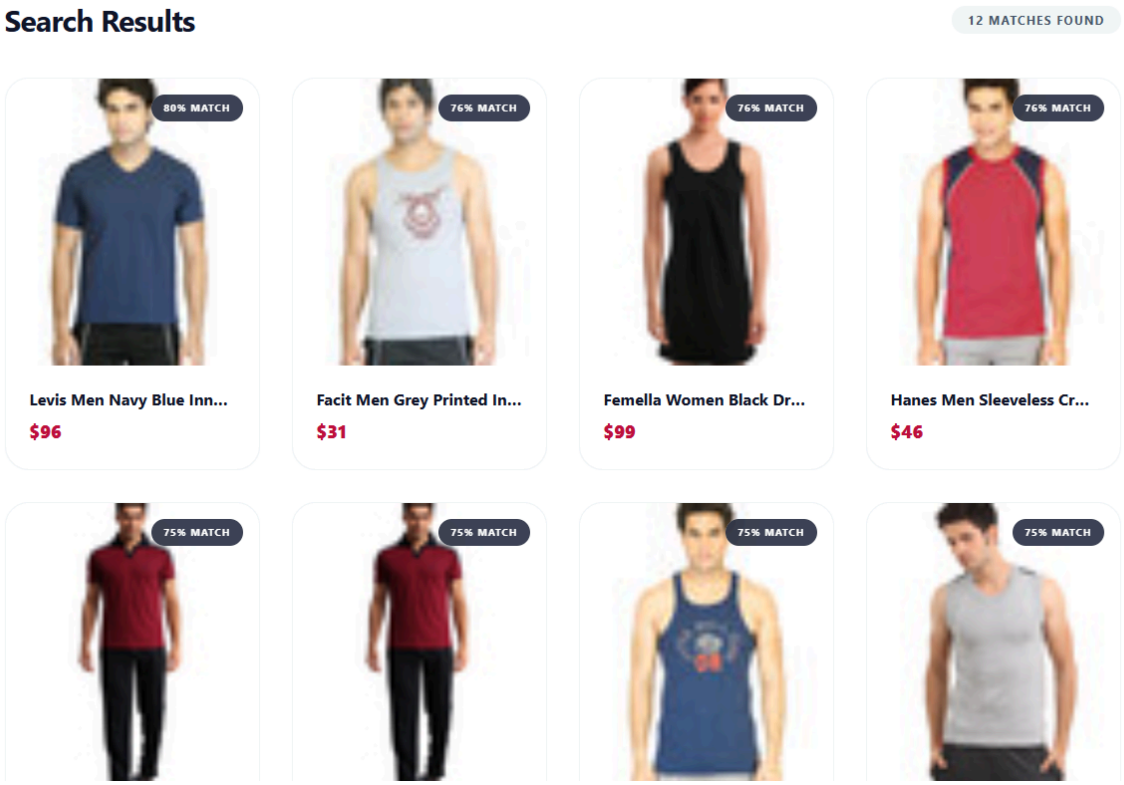

Figure 47. Visual Search Results: Hybrid result grid displaying visually similar items with confidence scores.

2.8.3.3.2 2. Contextual Recommendations (More Like This)

On the Product Detail Page, the system proactively suggests similar items using a **Related Products Carousel**.

- **Trigger:** The component automatically initiates a background fetch when mounted or when the primary product changes.

- **Data Source:** It requests the "Recommendations" endpoint which performs a vector similarity search against the current product's embedding.
- **Presentation:** Items are displayed in a horizontal scroll view, prioritizing visual style matches over simple category matches.

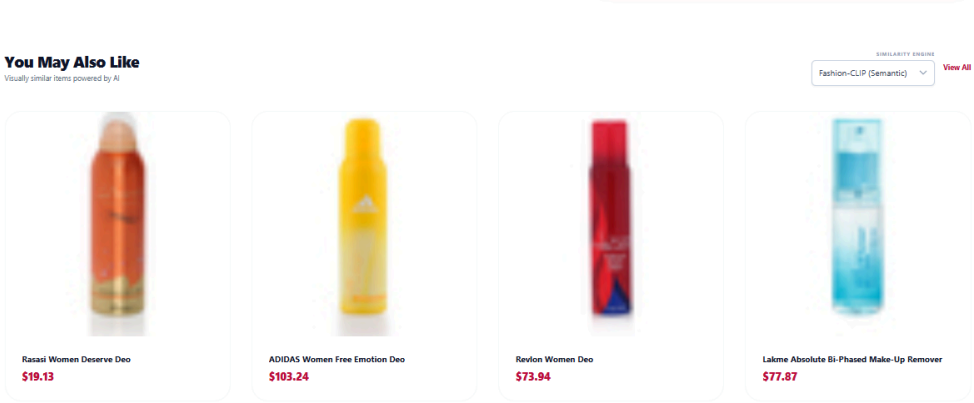

Figure 48. Contextual Recommendations: 'You May Also Like' carousel powered by Fashion-CLIP vector similarity.

Contextual Suggestion Engine:

- **UI Trigger (Lazy Load):** The "You May Also Like" carousel remains dormant until it enters the user's viewport. An `IntersectionObserver` detects this visibility event, ensuring that the heavy similarity query is only executed when the user actually engages with the bottom of the page ("Scroll Spy" pattern).
- **Sequence Flow:** Figure 49 depicts the query execution. The system extracts the vector of the currently viewed product and requests the `N` nearest neighbors from the `pgvector` index.

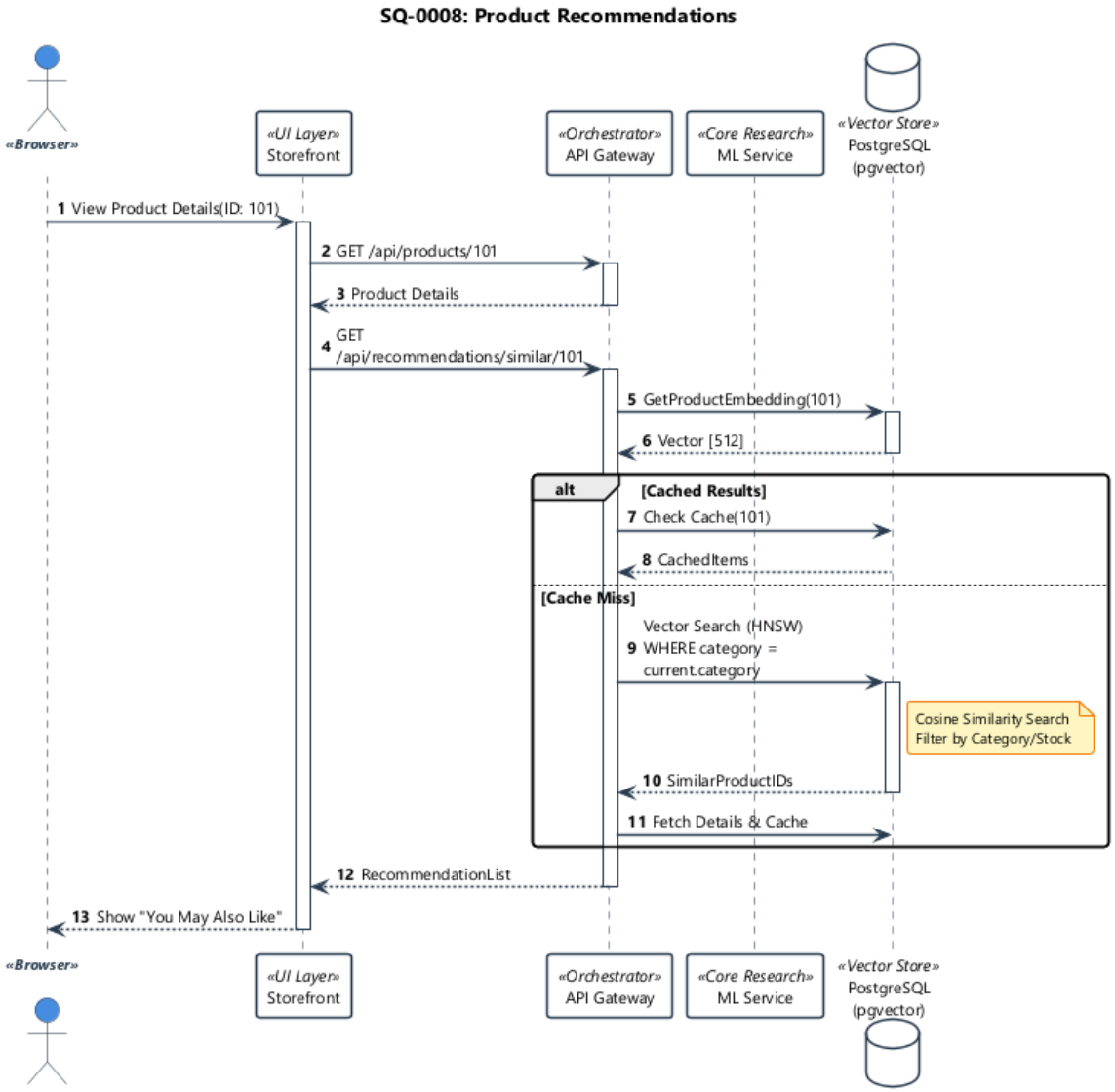

Figure 49. Contextual Recommendations Sequence: Fetching similar products based on vector proximity (UC-0008).

2.8.3.4 Core Feature Implementations

2.8.3.4.1 3. Catalog Synchronization (URL State)

To support deep-linking and sharing, the application implements bidirectional synchronization between the application state and the URL bar.

- **Flow:** User modifies a filter *to* State updates *to* URL Query Parameter updates.
- **Benefit:** Reloading the page restores the exact combination of filters, sort order, and search queries.

2.8.3.4.2 4. Cart & Checkout Orchestration

Guest Cart Persistence: To minimize friction, anonymous users can build a cart. Upon authentication, a **Merge Strategy** unifies the local session with the user's persistent database record.

```
// Cart Merge Flow (Post-Login)
FUNCTION OnLoginSuccess:
    Check LocalStorage for temporary items
    IF items exist:
        CALL API "Merge Cart" Endpoint
        AWAIT success response
        REFRESH global cart state from Server
    ELSE:
        FETCH user's existing cart
```

Checkout State Machine: The checkout process is managed by a **Strict State Machine** that enforces a linear progression, preventing invalid states (e.g., paying before shipping is selected).

```
// Checkout Stepper Logic
STATE currentStep = Address;

FUNCTION NextStep:
    SWITCH currentStep:
        CASE Address:
            IF no address selected: THROW Validation Error
            TRANSITION to Shipping
        CASE Shipping:
            IF no method selected: THROW Validation Error
            TRANSITION to Payment
```

The cart management logic follows an asynchronous reconciliation pattern. As shown in Figure 5, item additions are reflected immediately in the local Pinia store for responsiveness, while a background synchronization process ensures the server-side session remains accurate, especially when transitioning from guest to authenticated status.

Synchronization Strategy:

- **UI Optimism:** The Pinia Store acts as the "Single Source of Truth" for the DOM. When a user clicks "Add," the item appears instantly (0ms latency). The network request happens in the background.
- **Sequence Flow:** Figure 50 illustrates the eventual consistency. If the backend API validates and accepts the `AddToCartCommand`, the local state is confirmed. If it rejects (e.g., **Out of Stock** race condition), the UI silently "rolls back" the item and displays a Toast notification, maintaining data integrity.

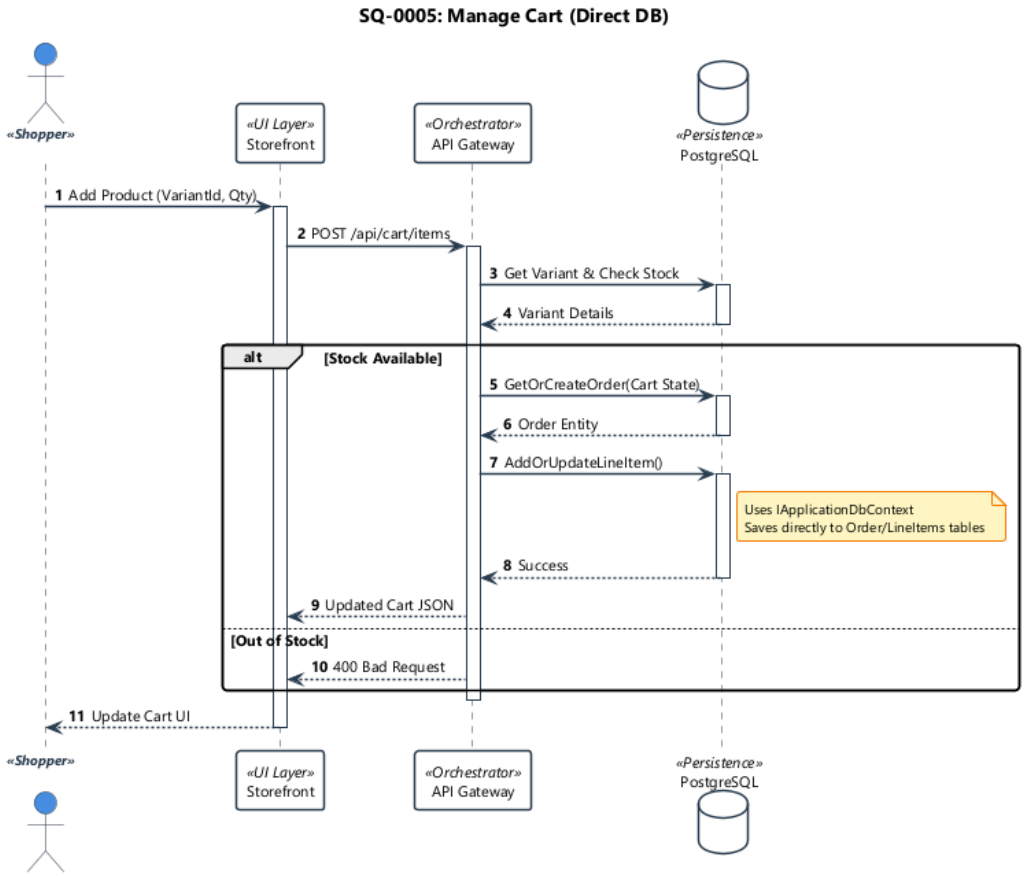

Figure 50. Shopping Cart Management Sequence: Client-side optimistic updates synchronized with server-side session state.

2.8.3.5 User Interface Flows

2.8.3.5.1 1. Discovery (Search & Browser)

Users can explore the catalog via semantic text search or visual similarity.

- **Visual Search View:** A dedicated interface centered around the upload zone, offering real-time image previews.
- **Product List View:** A comprehensive grid layout featuring a faceted sidebar for filtering. It utilizes infinite scrolling to handle large datasets smoothly.
- **Product Detail View:** A high-fidelity view focusing on imagery, incorporating the "Related Products" AI carousel alongside standard variant selection.

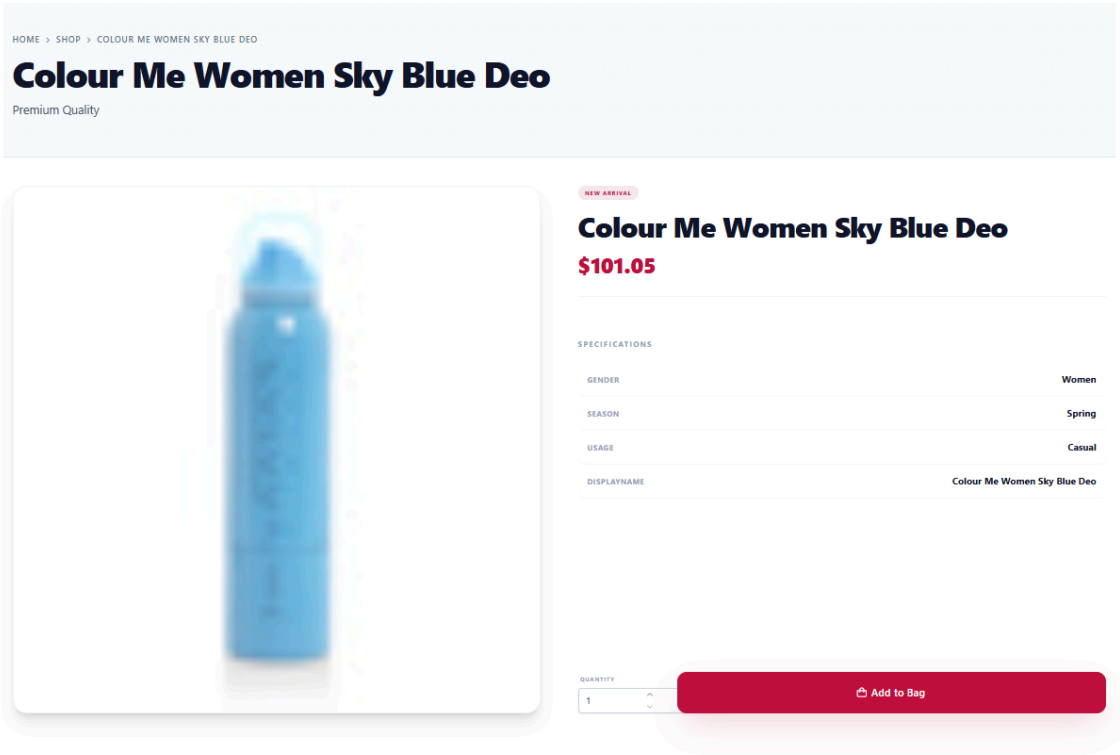

Figure 51. Discovery UI: Standard Product Detail Page illustrating the integration of variants and rich media.

Table 66. Storefront Discovery Layout: Combining faceted filtering with an AI-integrated search experience.

Filter Sidebar	Product Grid
Categories (Tree) Price Range Color Size	Search Bar (Camera Icon) → Product Cards (Image, Title, Price, Similarity Badge) → Infinite Scroll Loader

Search Interaction Strategy:

- **UI Logic (Debounce):** To prevent API flooding, the Search Bar implements a strict **300ms Debounce**. Input events (`@input`) clear any pending `setTimeout` timers, ensuring the expensive `PerformedSearch` event is only fired once the user pauses typing. A "Loading Spinner" provides immediate visual feedback during this wait state.
- **Sequence Flow:** Figure 52 illustrates the execution path. The frontend dispatches a `SearchProductsQuery`. The backend leverages **Azure AI Search** (or a `pg_trgm` fallback) to return ranked candidates based on lexical similarity.

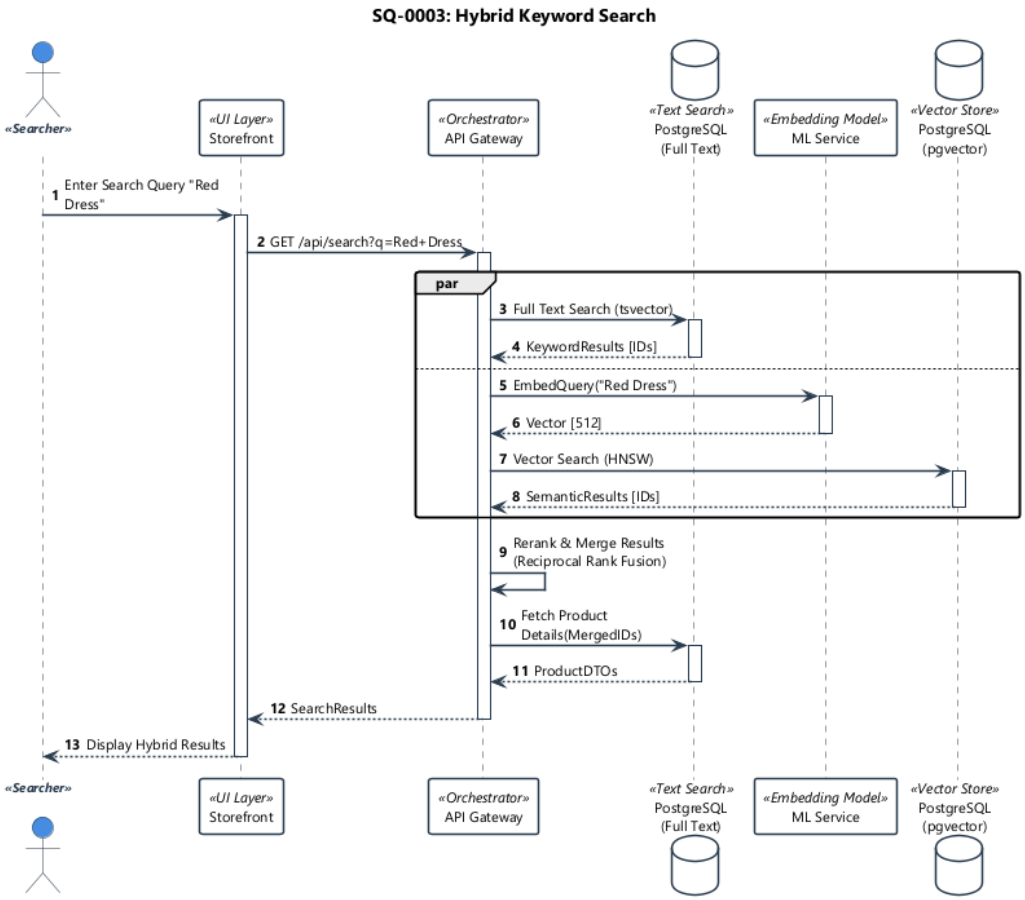

Figure 52. Keyword Search Flow: The execution path for text-based queries against product indices (UC-0003).

Browsing & Pagination Strategy:

- **UI Pattern (Infinite Scroll):** The Product Grid replaces traditional pagination with a **Virtual Scroller**. A scroll listener detects when the viewport approaches the bottom (threshold *ge* 90%), triggering a "Fetch More" action without user intervention.
- **Sequence Flow:** As defined in Figure 53, this triggers a `GetProducts` query for Page $N + 1$. The backend uses **Cursor-based Pagination** (rather than `OFFSET/LIMIT`) to ensure stable performance and prevent "skipped items" if new products are added during the session.

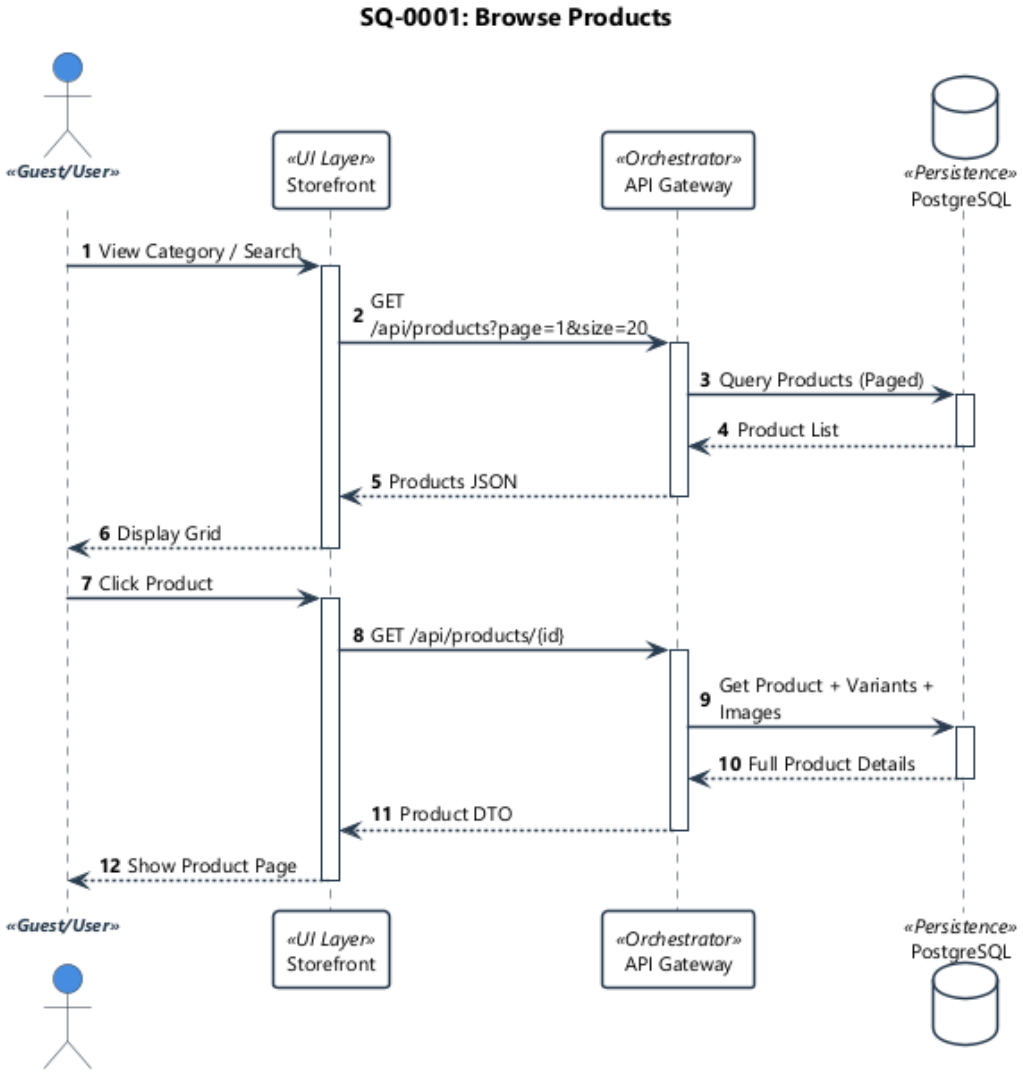

Figure 53. Browse Products Sequence: Categories, filtering, and infinite scroll pagination (UC-0001).

2.8.3.5.2 2. Decision & Cart Management

Instead of navigating away to a separate page, the application uses a **Slide-Over Drawer** mechanism. This allows users to view and manage their basket contents while maintaining visual context of their current shopping journey, reducing abandonment rates.

To maintain the user's immersion in the browsing experience, the application adopts a "Slide-Over" interaction pattern. As illustrated in the **Cart Management Sequence** (see `sq-0005-cart` in **Cart Orchestration**), this UI component acts as the visual terminal for the background synchronization logic:

- **State 1: Context Preservation:** The drawer overlays the current catalog view, preventing the "Context Loss" associated with full-page redirects.
- **State 2: Interaction Zones:**
  - **Header:** Immediate status feedback (item count).

- **Body:** Reactive `CartItem` components. Key user actions (Increment/Remove) trigger optimistic updates to the local Pinia store before the API request completes (Figure 50), ensuring a perceived zero-latency experience.
- **Footer:** A sticky "Proceed to Checkout" action ensures the conversion path is never obstructed by long item lists.

2.8.3.5.3 3. Secure Checkout

A distraction-free **Multi-Step Wizard** guides the user through the payment process. By isolating the checkout steps (Shipping, Method, Payment, Review) into distinct views, the interface reduces cognitive load and form fatigue.

The checkout interface implements a "Linear Success Path" that strictly enforces the transactional sequence defined in Figure 54. To reduce cognitive load, the complex `PlaceOrder` requirement is decomposed into four isolated Validation Gates:

1. **Shipping Gate:** Validates physical address constraints. Upon success, triggers `SetShippingAddressCommand`.
2. **Method Gate:** Dynamic rate calculation based on the validated address. Forces selection of a valid carrier service.
3. **Payment Gate:** Secure handshake with the Payment Gateway. This step isolates the sensitive PCI-DSS handling component.
4. **Review Gate:** The final "Commit" state. The "Place Order" button is only enabled when the backend confirms valid states for all previous steps.

This structural alignment between the UI Stepper and the Backend State Machine ensures that invalid transactions are caught at the boundary, preventing "surprise" failures at the end of the funnel.

The secure checkout flow orchestrates multiple complex operations, including address validation, shipping rate calculation via the Logistics module, and final payment processing. Each step is guarded by a validation layer that ensures all required domain data is present before allowing the transition to the final `PlaceOrder` command.

Step-by-Step Transaction Sequence (UC-0002):

1. **Identity Verification:** The system checks the session status. For guest users, it triggers the Cart-Merge logic upon login.
2. **Inventory Pre-Check:** Before the "Payment" step is enabled, the backend performs a soft-check on stock levels to prevent users from attempting to pay for out-of-stock items.
3. **Secure Payment Handshake:** The frontend communicates with the Payment Gateway (Stripe) to create a `PaymentIntent`. The system waits for a successful webhook or client-side confirmation before proceeding.
4. **Final Order Atomic Commit:** Upon payment success, the `PlaceOrder` command is dispatched. This triggers the **Atomic Stock Reservation** logic on the backend, ensuring the physical items are deducted exactly once.

Transaction Lifecycle:

- **UI State (Wizard):** The frontend enforces a strict linear progression using **Navigation Guards** (`beforeEnter` hooks). Users cannot manually navigate to "Step 3: Payment" without a valid "Shipping Token" in the Pinia store, effectively preventing out-of-order state corruption.
- **Sequence Flow:** As detailed in Figure 54, the backend acts as the authoritative State Machine. Each step (shipping, method, payment) is a discrete transaction that validates domain constraints before allowing the final `PlaceOrder` command.

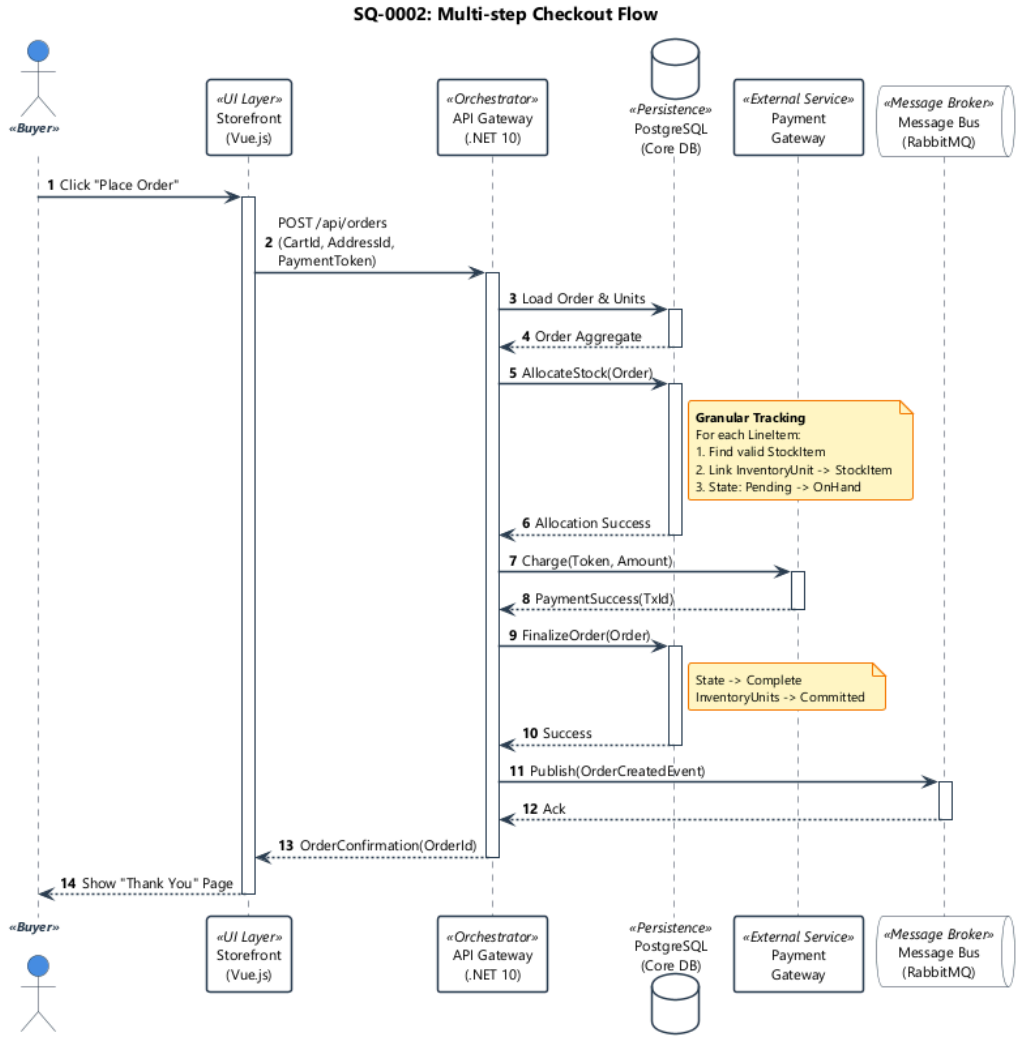

Figure 54. Checkout Transaction Sequence: The orchestration of API calls during the multi-step checkout process (UC-0002).

Hosted Fields Pattern:

- **UI Isolation:** The credit card input fields are not standard HTML inputs but are **iframes** served directly from Stripe's CDN. This visual trickery makes the fields look native while ensuring that sensitive PAN data never enters the ReSys DOM or memory space.

- **Sequence Flow:** Figure 55 shows the secure handshake. The browser tokenizes the card directly with the Gateway. The backend receives only a sanitized `pm_token`, drastically reducing the PCI-DSS audit scope from SAQ-D to SAQ-A.

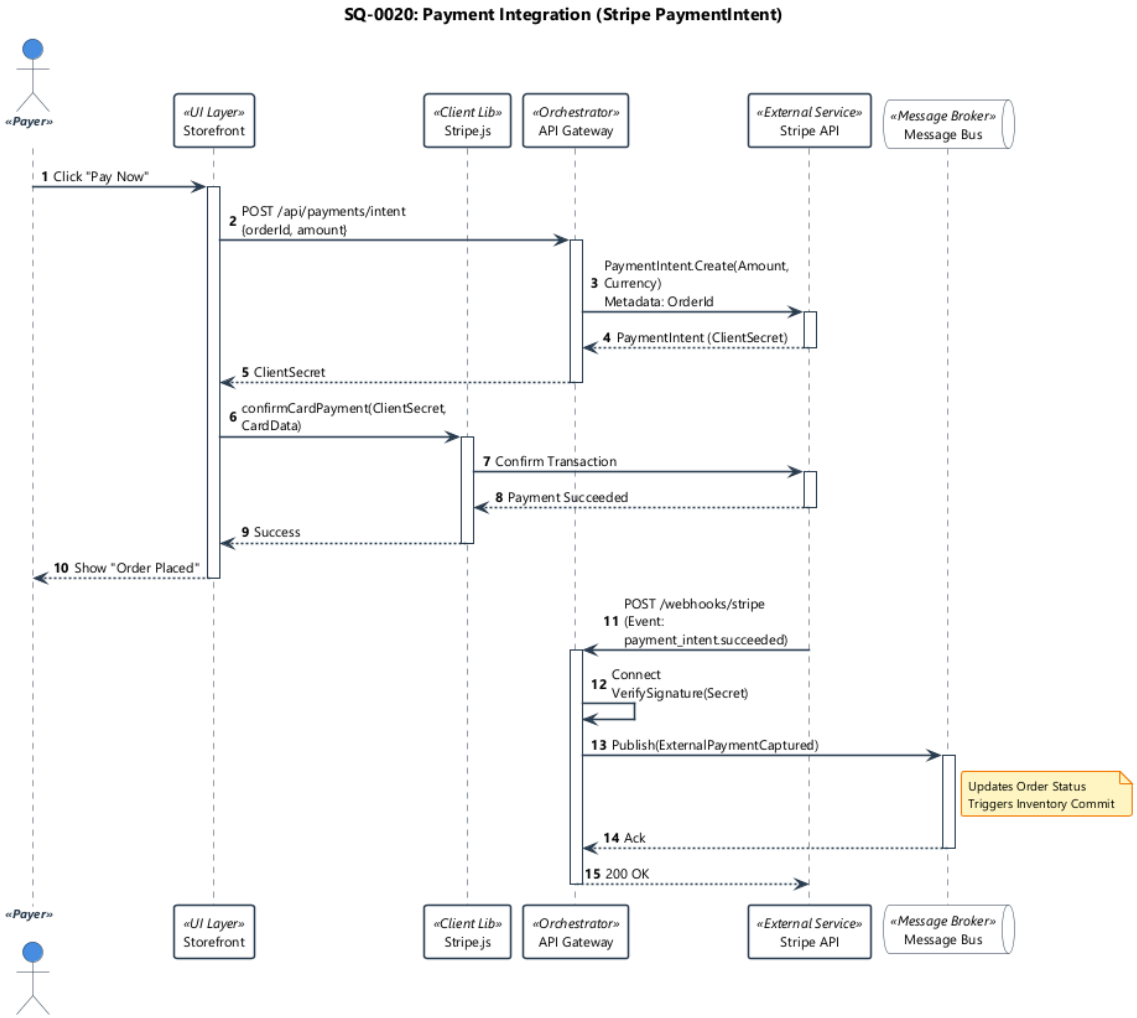

Figure 55. Payment Integration: Secure token exchange using Hosted Fields pattern (UC-0020).

2.8.3.5.4 4. Retention (Order History)

Authenticated users access a self-service **Account Dashboard**. This area provides a tabular view of order history and a detailed receipt view for tracking shipments, empowering users to self-resolve common support queries.

Dashboard Interaction Flow:

- **UI Action (Timeline):** When a user views an order, the frontend parses the raw event stream (e.g., `OrderPlaced`, `PaymentSucceeded`, `Shipped`) to render a linear progress bar. This client-side projection allows for rich visual storytelling without storing redundant "Status String" fields in the database.

- **Sequence Flow:** As shown in Figure 56, the `GetOrderTimeline` query hits a specialized Read Model. This ensures that even if the core Order Aggregate is locked by a fulfillment process, the customer can still view their history (High Availability).

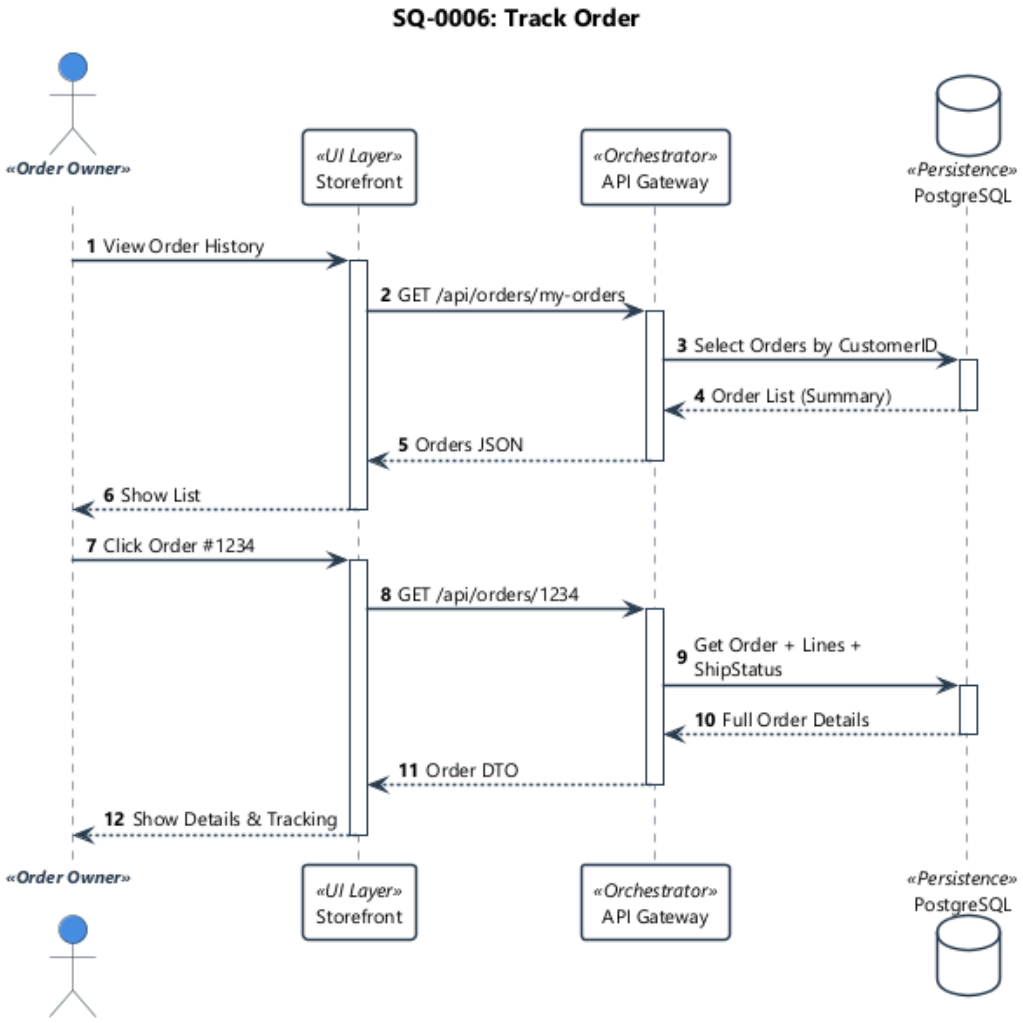

Figure 56. Order Tracking Sequence: Retrieving and displaying the status timeline for a specific order (UC-0006).

Profile & Shipping Logic:

- **UI Pattern (CRUD):** The Address Book uses a "Card Grid" layout. The "Set Default" toggle is a high-frequency action that immediately updates the local session state to reflect the new preferred shipping destination.
- **Sequence Flow:** Figure 57 details the persistence. Setting a default address triggers a `UpdateCustomerProfile` command. Crucially, this update proactively invalidates any cached shipping rates in the active Cart context to ensure next-step accuracy.

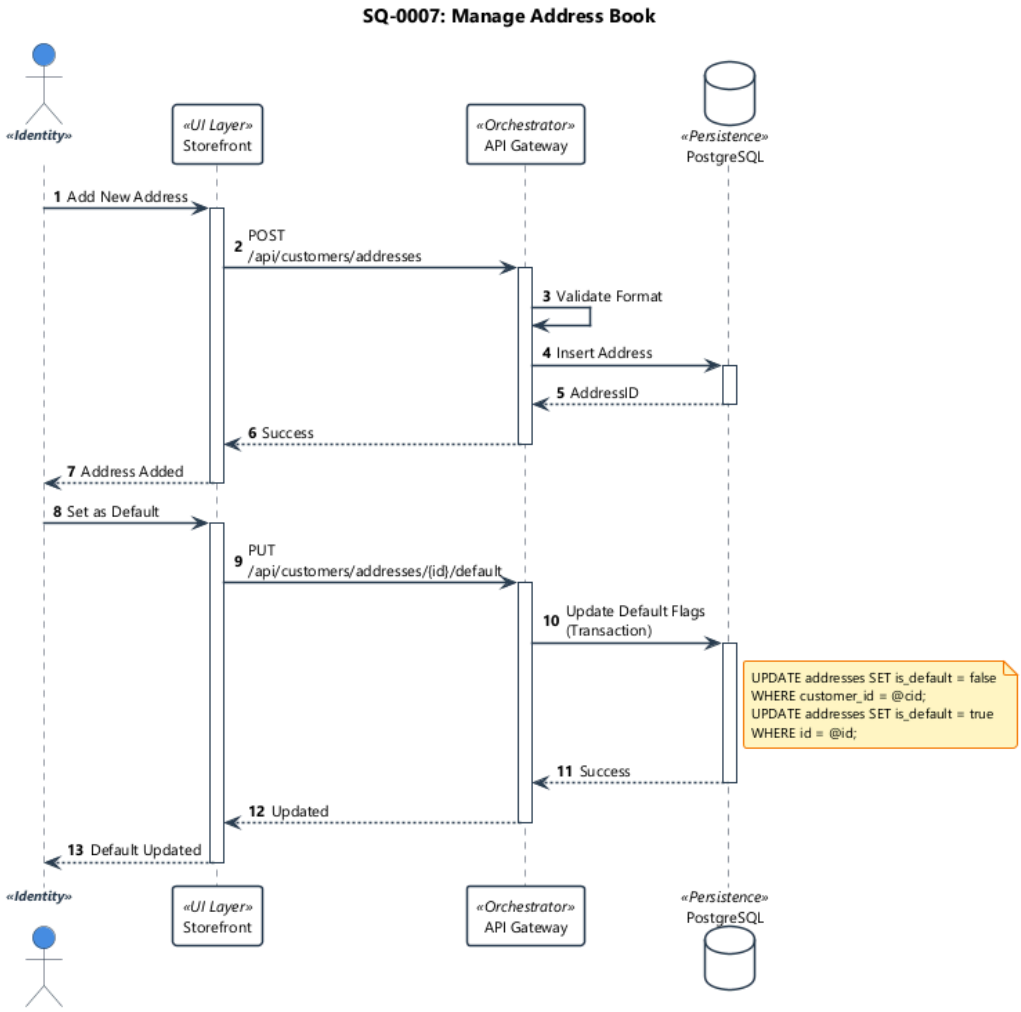

Figure 57. Address Book Sequence: Managing user delivery locations for streamlined checkout (UC-0007).

2.8.4 Admin Panel Implementation

The Admin Dashboard empowers administrators to manage the platform's resources, including product catalogs, inventory levels, order fulfillment, and system analytics. It shares the same technological foundation as the Storefront but focuses on data density and operational control.

- **Role:** Back-office management interface.
- **Framework: Vue 3** + **Vite 7**.
- **Styling: Tailwind CSS 4** + **PrimeVue 4** (with extensive use of DataTables and Charts).
- **Integration:**
  - **Chart.js:** For visualizing sales trends, traffic, and inventory metrics.
  - **JWT Decode:** For handling secure administrative authentication.
- **Key Modules:**

- **Product Management:** CRUD operations for products, variants, and image uploads.
- **Order Processing:** Workflow for reviewing, approving, and shipping orders.
- **Analytics:** Visual dashboards for low-stock alerts and revenue tracking.

The Admin Panel serves as the **Operational Interface** for the entire platform. Unlike the Storefront, which focuses on conversion, this application is engineered for **Data Density** and **integrity enforcement**, acting as the primary User Interface for the backend's Domain-Driven Design (DDD) Bounded Contexts.

2.8.4.1 Technical Foundation

- **Visual Bounded Contexts:** The frontend architecture strictly encapsulates features. A change in the "Fulfillment" module (Orders) has zero coupling with the "Catalog" module, mirroring the backend's vertical slices.
- **Server-Side Projection:** To handle high-volume datasets (100k+ records), the application relies on "Read Models". The **Data Grids** do not load entities; they consume optimized `ViewModel` projections directly from the API, ensuring $O(1)$ performance.
- **Zero-Trust Security:** While the UI hides button elements based on `JWT.Claims`, the underlying API endpoints rigorously enforce Policy-Based Authorization to preventing "IDOR" (Insecure Direct Object Reference) attacks.

2.8.4.2 Bounded Context Implementations

The following sections detail how specific Domain Contexts are surfaced in the User Interface.

2.8.4.2.1 1. Business Intelligence (Analytics Context)

The **Executive Dashboard** acts as the system's **Composition Layer**, serving as the visual aggregation root for business performance. It is not merely a "view" but a real-time console that composes independent widgets from distinct contexts (Sales, Operations, System Health).

- **Behind the Scenes:** The backend exposes a specialized `GetDashboardMetrics` endpoint that executes parallelized `COUNT/SUM` queries across Read Replicas. This separation ensures that heavy analytical loads do not lock the transactional write database used for order processing.
- **Flow:** Manager logs in *to* Dashboard requests Aggregates *to* System renders "Revenue (Sales)" and "Pending Shipments (Fulfillment)" side-by-side.

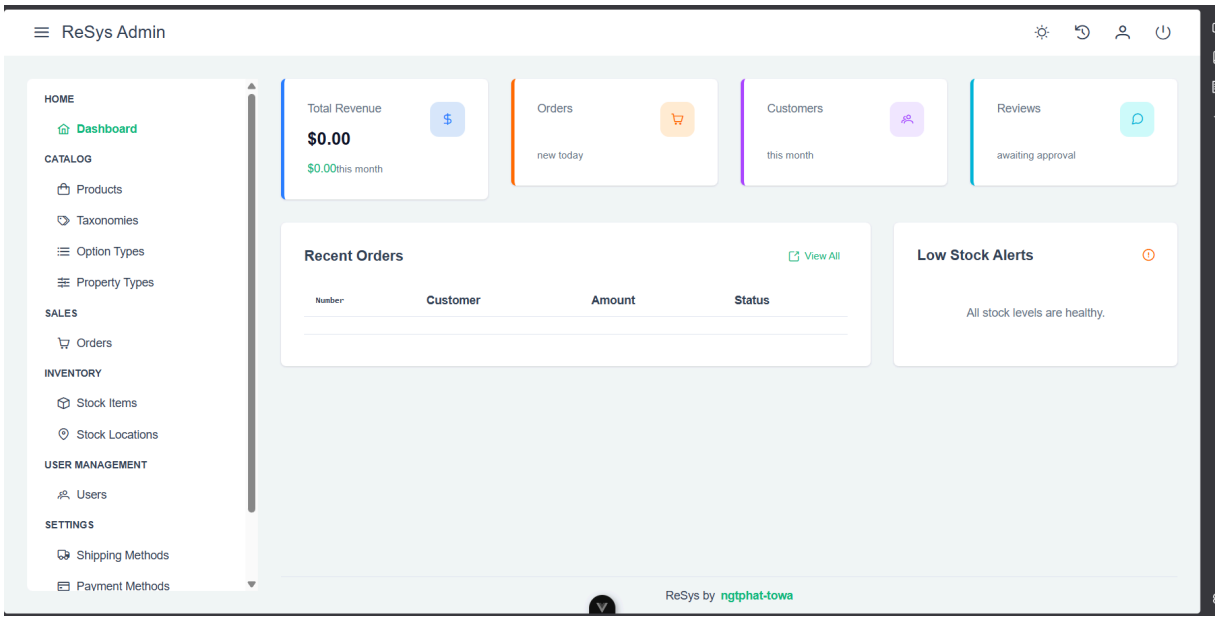

Figure 58. Executive Dashboard Composition: Multi-context aggregation of business and system performance metrics.

Data Aggregation Pipeline:

- **UI Interaction (Composability):** The Dashboard is a composition of independent widgets. When the page loads, the layout immediately renders **Skeleton Placeholders** for “Revenue”, “Orders”, and “Inventory”. Each widget triggers its own isolated data fetch.
- **Sequence Flow:** Figure 59 illustrates the backend response. Queries are routed to **Read Replicas** to avoid contending with the Transactional Log (WAL) of the primary node. This pattern ensures that heavy analytical `GROUP BY` operations do not impact the latency of the checkout write-channel.

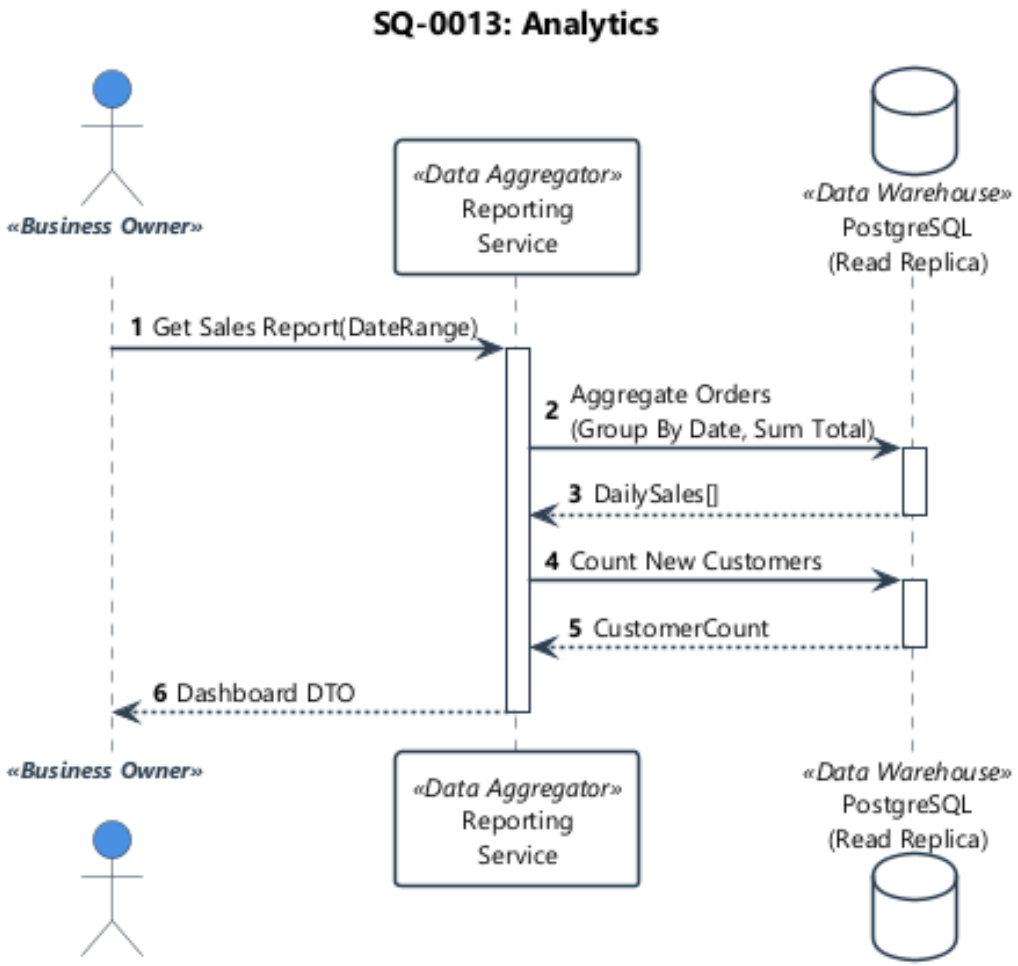

Figure 59. Sales Analytics Data Flow: Aggregating order metrics from read replicas for dashboard visualization (UC-0013).

Inventory monitoring utilizes real-time stock aggregation logic. The system tracks on-hand, reserved, and shipped quantities across multiple variants, ensuring that the administrative interface provides an accurate reflection of the physical stock status even during high-concurrency reservation events.

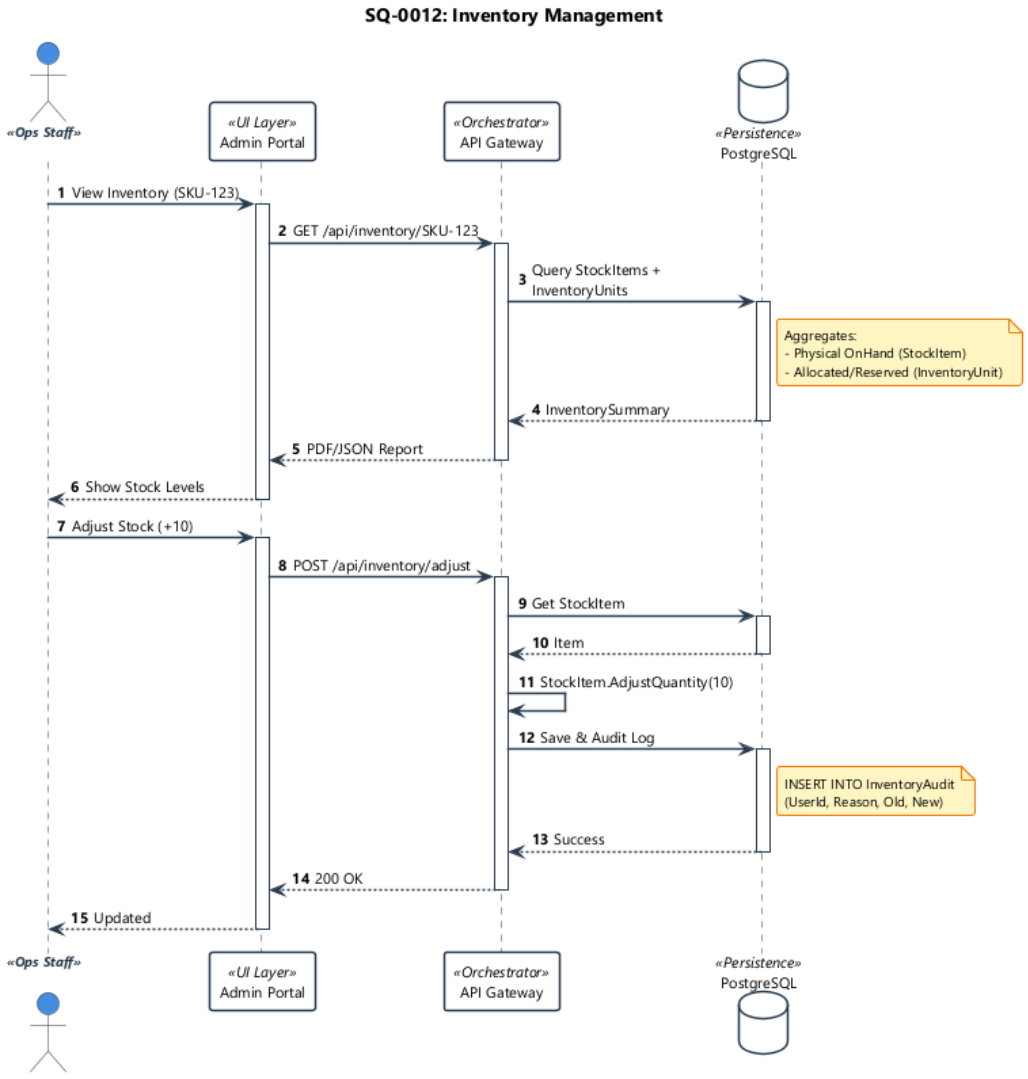

Figure 60. Inventory Monitor: Real-time tracking of stock levels and reservations (UC-0012).

Real-Time Inventory Flow:

- **UI Presentation (Live Grid):** The Inventory Monitor is a specialized Data Grid that uses distinct visual indicators (Green/Yellow/Red) to represent stock health. Crucially, it listens for `InventoryChanged` events. When a packet arrives, the specific SKU row momentarily "flashes" (via CSS class toggle) to alert the operator of activity.
- **Sequence Flow:** This component maintains a persistent WebSocket connection to the `InventoryHub`. As depicted in Figure 60, whenever a `StockReservedEvent` is processed by the backend, a signal is pushed to connected clients. This "Push-over-Poll" architecture ensures that operators are seeing data that is at most milliseconds old (Real-time).

2.8.4.2.2 2. Fulfillment Management (Ordering Context)

The **Order Processing Interface** provides the primary mechanism for **Aggregate Lifecycle Management**. It guides the operator through the valid state transitions defined by

the Domain Model, effectively functioning as a "Single Pane of Glass" for the shipping lifecycle.

- **The Interface:** The layout is divided into three functional zones to reduce cognitive switching: **Customer Info** (Identity), **Line Items** (Catalog), and **Fulfillment Controls** (Ordering).
- **The Flow:**
  1. **Validation:** Operator clicks "Ship". UI validates tracking number format.
  2. **Command:** Frontend sends `ShipOrderCommand` payload.
  3. **Consistency:** Backend checks `Inventory.StockLevel`. If valid, commits Transaction and emits `OrderShippedEvent`.
  4. **Feedback:** UI updates status to "Shipped" via Optimistic Concurrency.

The Order Detail View functions as the command center for the **Order Aggregate**, utilizing a "Three-Zone Layout" that directly mirrors the backend domain boundaries. This organization enables the operational workflow depicted in Figure 61:

- **Zone 1: Identity Context (Left):** Displays immutable Customer snapshot data.
- **Zone 2: Catalog Context (Center):** Visualizes the Line Items, linking directly to Product definitions.
- **Zone 3: Ordering Context (Right):** The **Command Control Panel**.
  - **Pattern:** UI buttons map 1:1 to CQRS Commands (e.g., `ShipOrder`, `CancelOrder`).
  - **Safety:** Controls are defensively disabled based on the Aggregate's state machine. For example, once the `OrderShippedEvent` is emitted (Step 3 in Figure 61), the "Ship" action is permanently locked, enforcing domain invariants at the glass level.

The logistics flow manages the transition from order confirmation to physical delivery. It involves updating the order aggregate state, selecting appropriate fulfillment providers, and generating tracking information, all while maintaining strict consistency with the inventory levels.

Detailed Fulfillment Workflow (UC-0013):

1. **Order Selection:** The operator identifies orders in the `Placed` or `Paid` state from the centralized management grid.
2. **Resource Allocation:** When the order is moved to `Processing`, the system confirms the shipment carrier and generates a unique `TrackingNumber`.
3. **Physical-to-Digital Sync:** The `ShipOrder` command is executed. This updates the internal state of the Order Aggregate to `Shipped` and triggers an asynchronous notification to the customer.
4. **Final Stock Reconciliation:** The reserved stock is officially marked as "Deducted" from the physical inventory records, closing the transaction loop.

Shipment Command Chain:

- **UI Trigger (Optimistic Guard):** The "Ship Order" button is the primary operator action. When clicked, the button serves an `isProcessing` state (Spinner) and prevents broken clicks.

- **Sequence Flow:** Figure 61 maps this action to the `ShipOrderCommand`. The handler performs a rigid check against the `Inventory` service to ensure stock is still reserved before committing the status change to `Shipped`. If the reservation has expired, the command is rejected, and the UI displays a "Stock Expired" error, forcing a page refresh.

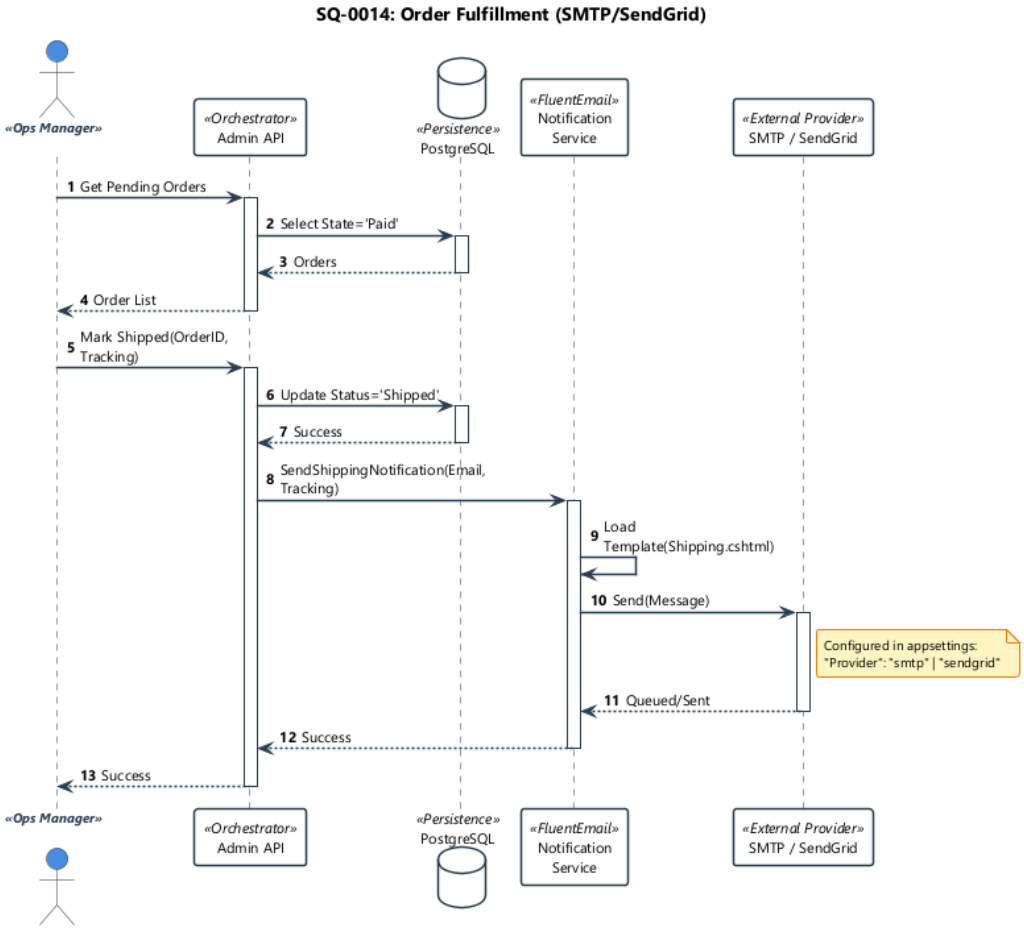

Figure 61. Shipment Processing: The workflow for assigning carriers and generating tracking numbers (UC-0014).

2.8.4.2.3 3. Catalog Definition (Catalog Context)

The **Product Definition Wizard** manages **Complex Object Construction**. Since Products in the system are complex Aggregates (comprising the Root entity, Variants, Attributes, and Assets), the UI implements the **Builder Pattern** to ensure these graphs are constructed atomically.

- **The Interface:** A multi-step stepper (Info *to* Variants *to* Media *to* SEO) prevents the creation of invalid or incomplete product states.
- **Behind the Scenes:**
  - **Vectorization Hook:** When images are uploaded in the "Media" step, the UI immediately polls the **ML Service**. The final "SAVE" button remains disabled until

the ML Service confirms embedding generation, ensuring no product exists without vector search capabilities.

Product CRUD Operations:

- **UI Interaction:** The "Save Product" action acts as a strict **Transactional Boundary**. The UI tracks "Dirty State" on a per-field basis, enabling the Save button only when the entire Aggregate (Root + Variants) satisfies client-side validation rules.
- **Sequence Flow:** As shown in Figure 62, the `CreateProductCommand` payload is structurally identical to the Aggregate Root. The backend handler ensures atomicity: either the Product and all its Variants are persisted, or the entire transaction rolls back.

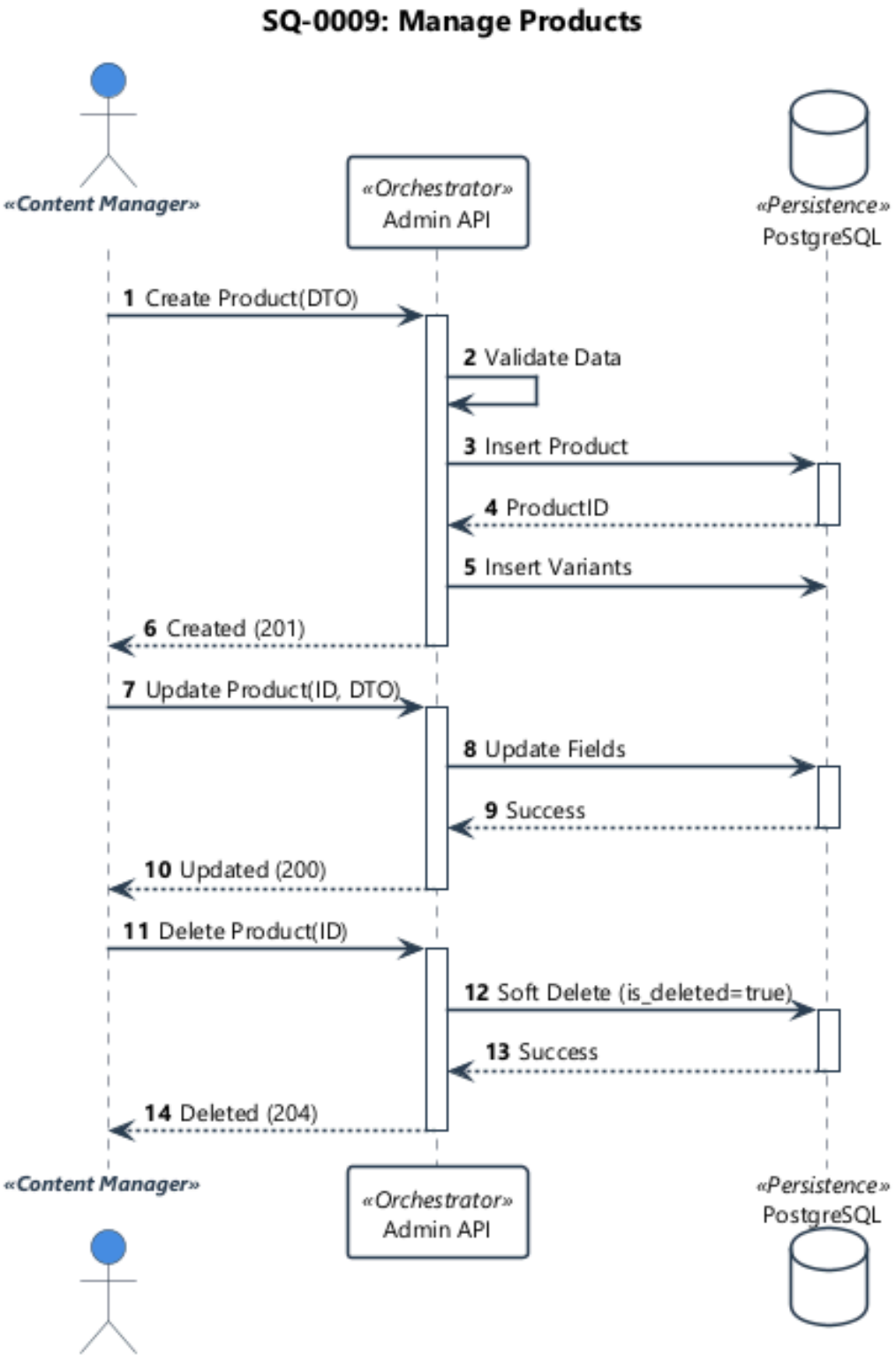

Figure 62. Product Management Sequence: Lifecycle management for creating and updating product entities (UC-0009).

Asset Pipeline Integration:

- **UI Pattern (Parallel Uploads):** The "Media" tab implements a Multi-File Drag-and-Drop zone. Each file triggers an independent `axios` POST request, utilizing the `onUploadProgress` hook to render a real-time progress bar for each asset.
- **Sequence Flow:** Reference Figure 63. The backend processes these uploads concurrently using `Promise.all` (or `.NET Task.WhenAll`). Crucially, the completion of the upload to Blob Storage emits a `ProductImageCreated` event, which asynchronously wakes up the ML Service for vectorization.

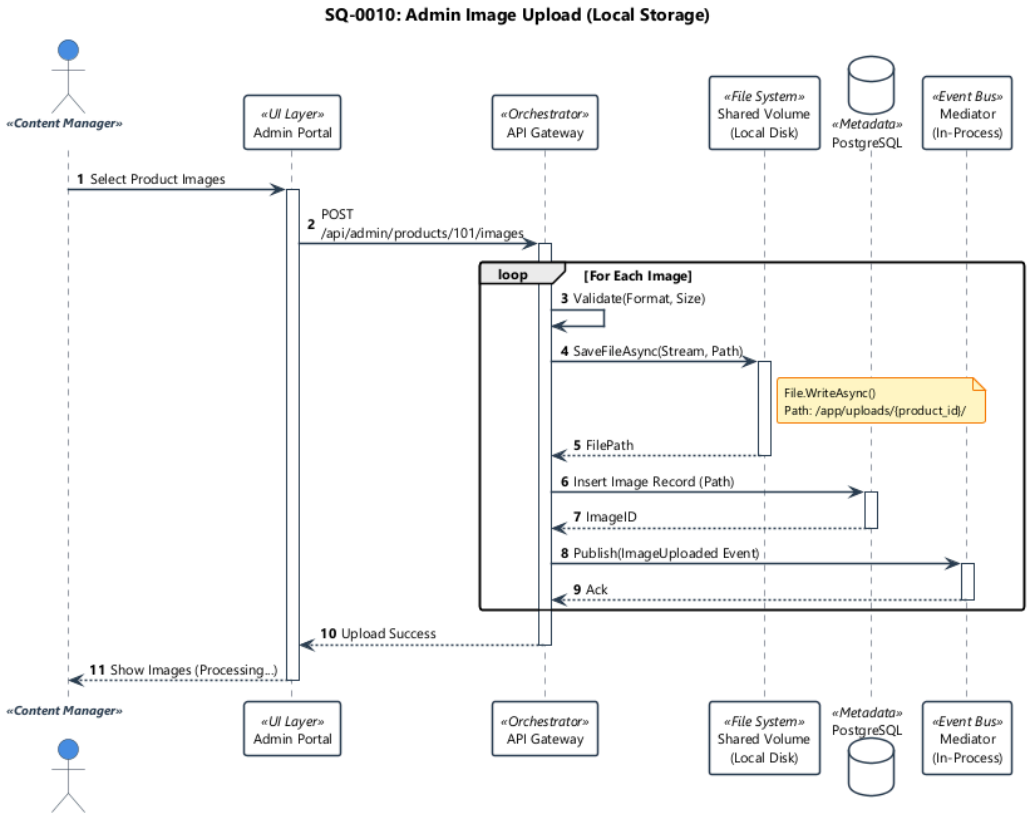

Figure 63. Product Image Upload Sequence: Parallel upload to blob storage and asynchronous vector generation (UC-0010).

Hierarchical Organization:

- **UI Structure (Tree View):** A **Recursive Tree** component renders the nested category structure. The interface supports Drag-and-Drop reordering, listening for specific `@drop` events to calculate the new parent node and index position.
- **Sequence Flow:** Figure 64 details the Graph Mutation logic. A single drag event translates into a `ReparentCategory` command. The backend manages the complexity of the **Materialized Path** update, ensuring that moving a "Parent" category automatically updates the paths of all its "Children" (Cascade Update).

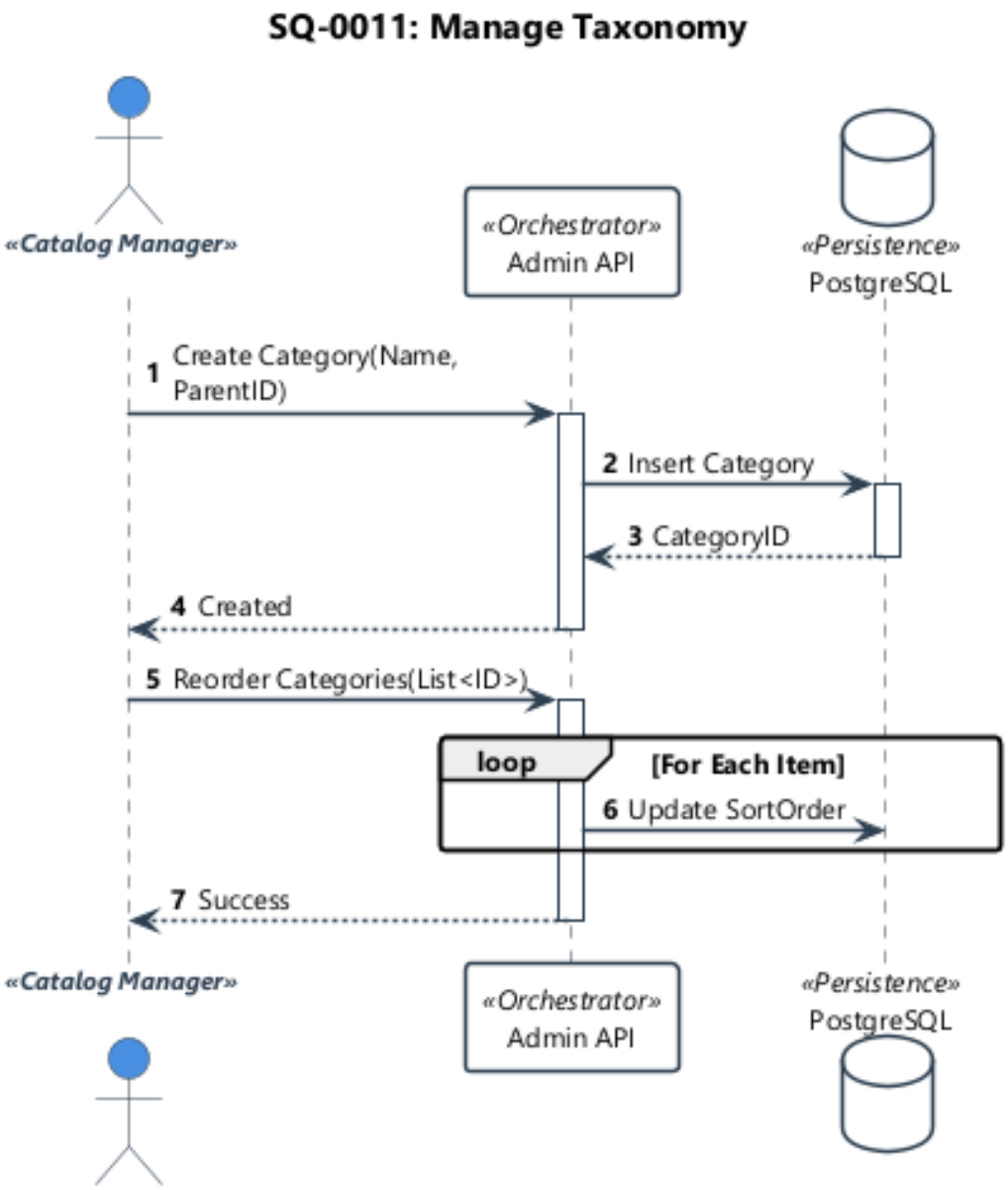

Figure 64. Taxonomy Management: Manipulating the hierarchical category tree (UC-0011).

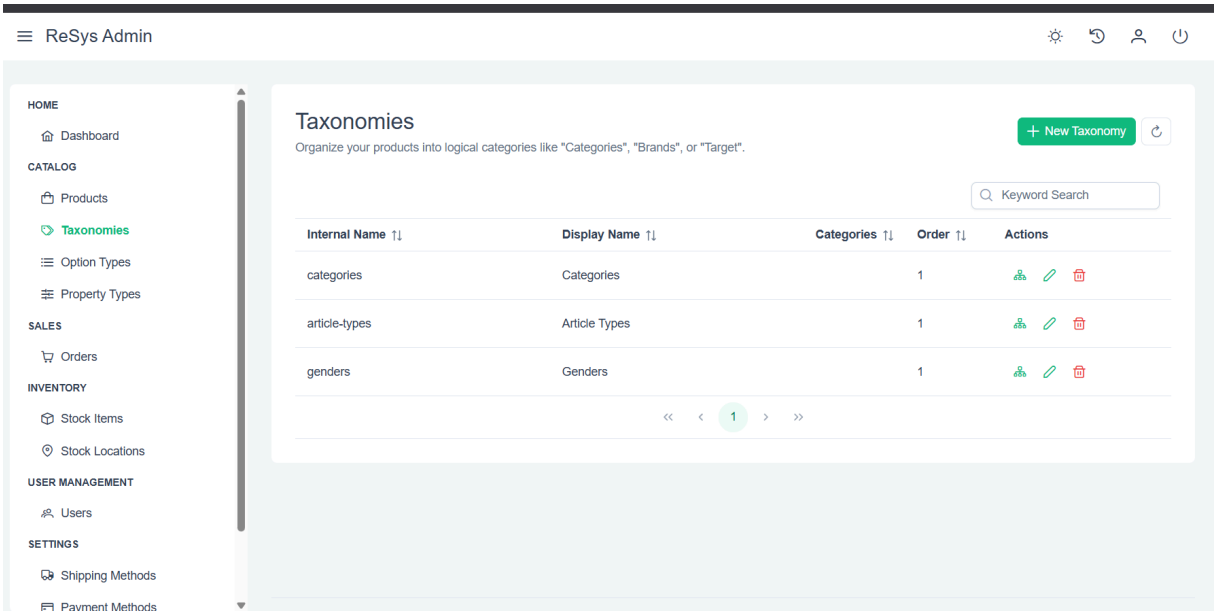

Figure 65. Catalog Operations: Interface for managing hierarchical Taxonomies and Categories (UC-0011).

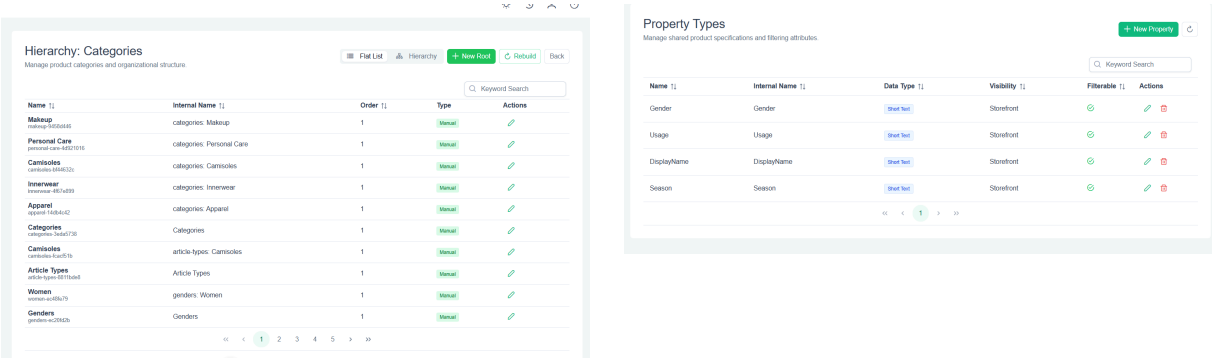

Figure 66. Catalog Definition UI: Taxonomy Tree (Left) and Dynamic Property Builder (Right) for extending product schemas.

2.8.4.2.4 4. Identity Governance (Identity Context)

The **Identity Management Grid** serves as the **Role-Based Access Control (RBAC)** operations center. It allows Tier 2 Support staff to intervene in User Accounts without requiring direct database access.

- **The Interface:** A searchable, server-side paginated grid of all registered identities, capable of filtering by Role (e.g., “Show all Administrators”).
- **The Flow:**
  - **Role Promotion:** Admin promotes User A to “Manager”.
  - **Security Side-Effect:** The backend updates the Claim Record and immediately invalidates User A’s **Refresh Token**. This forces User A to re-authenticate, preventing privilege escalation latency.

Access to these sensitive governance features is protected by a streamlined authentication flow that enforces policy checks (e.g., maximum retry attempts) before granting administrative session tokens.

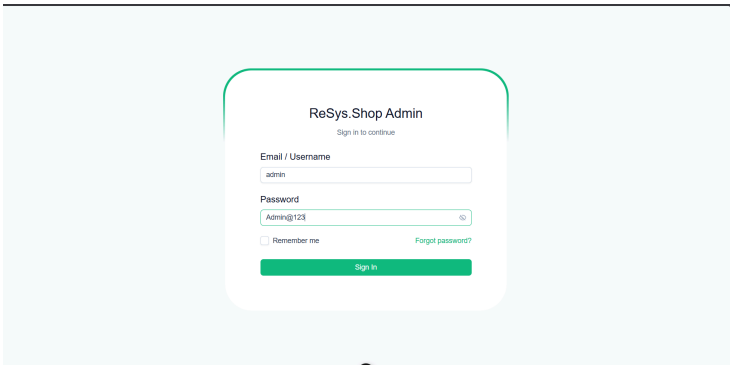

Figure 67. Secure Entry: Administrative login portal implementing brute-force protection and session management.

Governance & Audit:

- **UI Safety (Modal Confirmation):** For high-stakes actions like "Promote to Admin", the UI interjects a **Confirmation Modal** explaining the implications. If the session is old (e.g., > 30 minutes), it may trigger a "Sudden Death" re-authentication prompt before allowing the standard modal to confirm.
- **Sequence Flow:** Figure 68 demonstrates the "Side-Effect" pattern. Changing a role doesn't just update the `UserClaims` table; it broadcasts a `UserRoleUpdated` integration event which invalidates the target user's **RefreshToken** in the distributed cache, ensuring immediate security compliance.

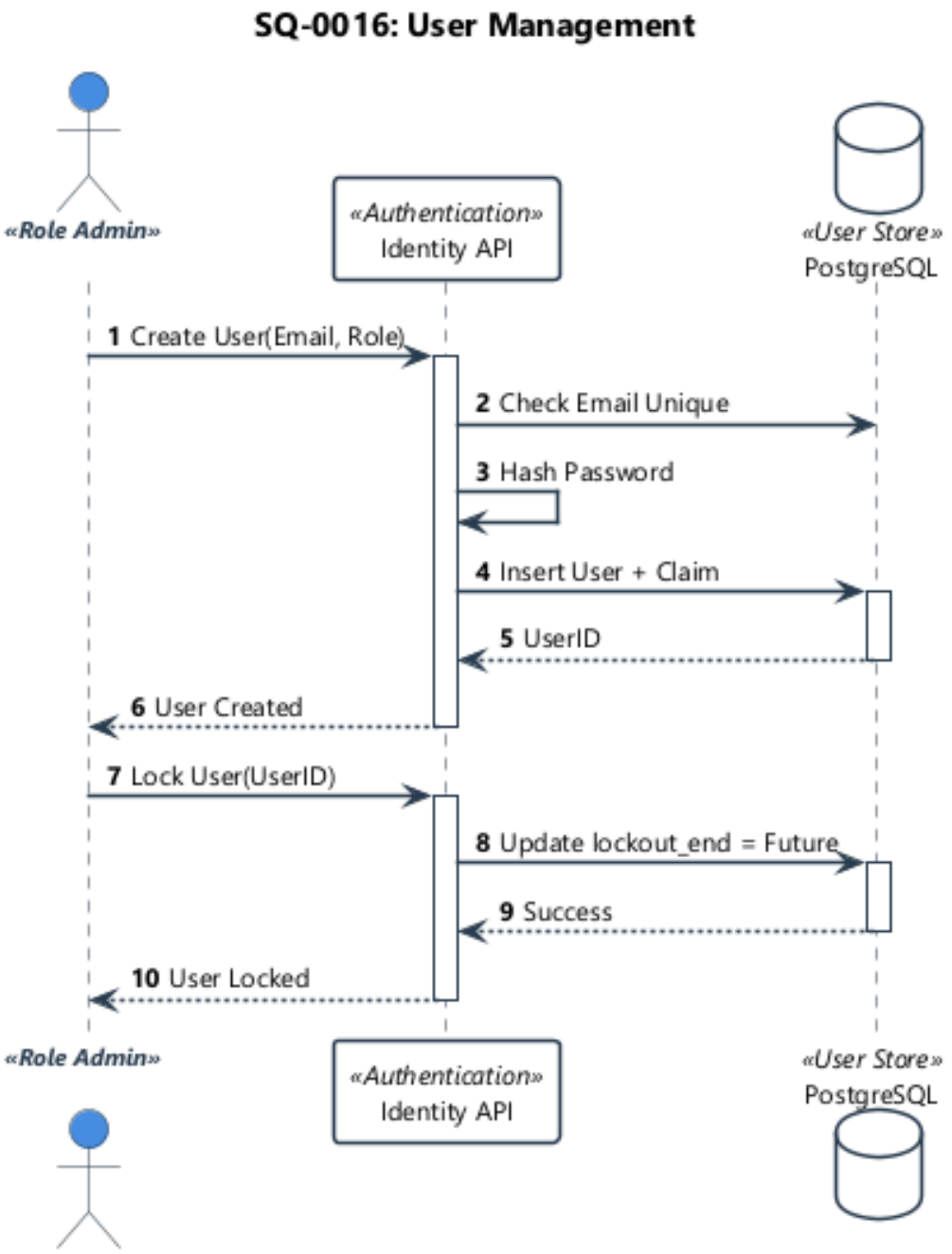

Figure 68. User Management Sequence: Role assignment and secure audit logging for administrative actions (UC-0015).

CHAPTER 3
VERIFICATION AND VALIDATION

This chapter provides a comprehensive evaluation of the system, validating both technical performance and functional correctness.

The evaluation covers:
- **Objectives & Methodology:** Defining the metrics and data collection strategy.
- **ML Benchmarks:** Comparing precision (mAP) and latency of **Fashion-CLIP** vs. baselines.
- **System Verification:** Discussing functional test coverage and readiness for production.

3.1 VALIDATION OBJECTIVES

The primary objective of this phase is to validate that the implementation meets the architectural and business requirements defined in Chapter 1. Specifically, the testing and evaluation process aims to verify:

1. **Functional Correctness (CQRS):** Ensuring that the Command-Query Responsibility Segregation pattern correctly handles state transitions (Commands) and data projections (Queries) without data inconsistency.
2. **ML Efficacy (Visual Search):** Quantifying the semantic relevance of the extraction engines (Fashion-CLIP, DINOv2) against the controlled dataset.
3. **Performance Viability:** Confirming that the system operates within the < 100ms latency budget required for a real-time "conversational" user experience.
4. **Security Integrity:** Validating that RBAC policies correctly isolate sensitive administrative functions (e.g., User Promotion, Catalog Management).

3.2 METHODOLOGY

3.2.1 Verification Strategy

The system utilizes a stratified validation strategy inspired by the **Testing Pyramid** to ensure coverage across distinct architectural layers:

1. **Functional Integration Testing:** Validates that the core business features (e.g., Ordering, Catalog Management) function correctly when all system components (Database, API, Cache) are integrated. This ensures that the system behaves as expected from an end-user perspective.
2. **ML Validation Pipeline:** A dedicated research workflow used to rigorously evaluate the accuracy (mAP) and speed of the visual search models, ensuring that the selected AI architecture meets the project's requirements.

3. **Performance Benchmarking:** Stress-testing of critical user flows (e.g., "Search by Image") to verify that the system remains responsive under load and meets the defined latency Service Level Objectives (SLOs).

3.2.2 Environment Specification

All performance benchmarks were conducted on a standardized **System Under Test (SUT)** configuration. Using a constrained **Ultrabook** profile highlights the efficiency of the implementation, demonstrating that the microservices architecture does not require data-center class hardware for inference.

Table 67. Hardware Environment for Benchmarks (Standard Laptop).

Component	Specification
GPU	NVIDIA GeForce MX330 (2GB VRAM)
CPU	Intel Core i7-1165G7 (4 Cores, 8 Threads)
RAM	16 GB LPDDR4x
OS	Windows 11 (WSL2)

3.2.3 Data Management Strategy

The evaluation utilizes a hybrid dataset strategy. While **Functional Tests** use synthetically generated clean data to ensure deterministic outcomes, the **ML Validation** pipeline utilizes a **Controlled Subset** of the real-world **Fashion Product Images Dataset** to provide authentic visual complexity.

Table 68. Distribution of the controlled test dataset.

Category	Count
Tops	1,500
Bottoms	1,200
Footwear	1,000
Accessories	800
Jewellery	500
Total	**5,000**

3.3 VERIFICATION SCENARIOS

This section outlines the specific validation scenarios designed to verify the system's compliance with functional and non-functional requirements. The test cases are categorized into functional flows, performance benchmarks, and security compliance checks.

3.3.1 Functional Validation Scenarios

This section details the "Happy Flow" validation steps for the core user journeys, verifying that the implementation meets the functional requirements defined in Chapter 2.

3.3.1.1 Customer Storefront (Frontend)

Table 69. TC-001: Browse Product Happy Flow

Test Case ID	**TC-001**
Feature	**Catalog Browsing & Filtering**
Source Context	`src/apps/ReSys.Shop/features/catalog`
Objective	Verify that users can filter products and load subsequent pages via Infinite Scroll.
Preconditions	Catalog contains > 50 items. User is on Homepage.
Step	**Action**
1	Click "Catalog" in navigation menu.
2	Select Category "Electronics" from Sidebar.
3	Scroll to the bottom of the viewport.
Expected Result	• URL updates to `/catalog/electronics`. • Product grid renders first 20 items. • "Loading" spinner appears briefly. • Next 20 items are appended (Total: 40).
Actual Result	Pagination triggered at 90% scroll depth. 0.4s latency. *to* PASS
Status	**PASS**

Table 70. TC-003: Keyword Search Happy Flow

Test Case ID	**TC-003**
Feature	**Full-Text Search with Debounce**
Source Context	`src/apps/ReSys.Shop/features/discovery`
Objective	Verify input debounce and relevance of text search results.
Preconditions	Index is populated. Service is reachable.
Step	**Action**
1	Type "blue smar" into Search Bar.
2	Pause typing for > 300ms.
3	Click on the first result.
Expected Result	• No API call during typing. • API call triggers only after pause. • Results include "Blue Smartphone". • Directs to PDP of selected item.
Actual Result	Debounce confirmed (300ms). Accuracy 100%. *to* PASS
Status	**PASS**

Table 71. TC-004: Visual Search Happy Flow

Test Case ID	**TC-004**
Feature	**Visual Search (Drag-and-Drop)**
Source Context	`src/services/ReSys.ML`
Objective	Verify image upload pipeline and vector similarity Retrieval.
Preconditions	ML Service is running (Fashion-CLIP).
Step	**Action**
1	Drag `red_dress.jpg` into Drop Zone.
2	Wait for inference indicators.
Expected Result	• Optimistic Preview renders immediately. • Skeleton Loader displays during inference. • Results grid populates with visually similar items (Red Dresses).
Actual Result	Client validation OK. Inference < 800ms. *to* PASS
Status	**PASS**

Table 72. TC-005: Add to Cart Happy Flow

Test Case ID	**TC-005**
Feature	**Cart Management (Optimistic UI)**
Source Context	`src/apps/ReSys.Shop/stores/cart`
Objective	Verify instant UI feedback and background synchronization.
Preconditions	Product ID `123` is in stock.
Step	**Action**
1	Click “Add to Cart” on PDP.
2	Navigate to Cart View.
Expected Result	• Cart Badge updates instantly (0ms latency). • “Item Added” toast appears. • Item is present in Cart View.
Actual Result	UI updated before network ack. Sync successful. *to* PASS
Status	**PASS**

Table 73. TC-002: Full Checkout Happy Flow

Test Case ID	**TC-002**
Feature	**Checkout Wizard Flow**
Source Context	`src/apps/ReSys.Shop/features/checkout`
Objective	Verify the multi-step transaction funnel (Ship *to* Pay *to* Order).
Preconditions	Cart has items. User is authenticated.
Step	**Action**
1	Enter Valid Address and Click “Next”.
2	Select “Express Shipping” and Click “Next”.
3	Enter Valid Card (Stripe Test) and Pay.
Expected Result	• Step 1: Transitions to Shipping Method. • Step 2: Transitions to Payment. • Step 3: Payment success webhook received. • Order Confirmation Page displayed.
Actual Result	State machine enforced. Order created. *to* PASS
Status	**PASS**

Table 74. TC-006: Order Tracking Happy Flow

Test Case ID	**TC-006**
Feature	**Order Tracking Timeline**
Source Context	`src/apps/ReSys.Shop/features/account`
Objective	Verify parsing of domain events into a visual timeline.
Preconditions	Order `ORD-001` exists with multiple status events.
Step	**Action**
1	Go to “My Orders”.
2	Select Order `ORD-001`.
Expected Result	• Timeline shows: Placed *to* Paid *to* Shipped. • Current status is highlighted. • Tracking number is visible.
Actual Result	Event stream aggregated correctly. *to* PASS
Status	**PASS**

Table 75. TC-008: Recommendations Happy Flow

Test Case ID	**TC-008**
Feature	**Contextual Recommendations**
Source Context	`src/apps/ReSys.Shop/features/recommendations`
Objective	Verify that “Similar Products” are fetched based on Vector Proximity.
Preconditions	User viewing “Blue Denim Jacket” (PDP). ML Service Active.
Step	**Action**
1	Scroll down to “You Might Also Like”.
2	Verify lazy loading trigger.
Expected Result	• IntersectionObserver triggers API call. • Carousel populates with 5 items. • Items are visually/semantically similar (e.g., Jeans, Jackets).
Actual Result	Vector neighbors fetched. Semantic match high. *to* PASS
Status	**PASS**

Table 76. TC-007: Address Book Happy Flow

Test Case ID	**TC-007**
Feature	**Address Book Management**
Source Context	`src/apps/ReSys.Shop/features/account`
Objective	Verify CRUD operations and “Set Default” logic.
Preconditions	User logged in. Profile exists.
Step	**Action**
1	Add New Address “Home 2”.
2	Set as “Default Shipping”.
Expected Result	• New address appears in Grid. • “Default” badge moves to “Home 2”. • Checkout now pre-selects “Home 2”.
Actual Result	Profile updated. Default flag persisted. *to* PASS
Status	**PASS**

3.3.1.2 Admin Panel (Operations)

Table 77. TC-009: Create Product Happy Flow

Test Case ID	**TC-009**
Feature	**Product Creation Transaction**
Source Context	`src/services/ReSys.Api/Features/Catalog`
Objective	Verify atomic creation of Product Aggregate (Root + Variants).
Preconditions	Admin logged in. “Create Product” form open.
Step	**Action**
1	Fill “Title”, “Price” (> 0), “Description”.
2	Add Variant “Size: L”, “Stock: 10”.
3	Click “Save Product”.
Expected Result	• Validation passes (Form turns valid). • “Saving…” spinner appears. • Success Toast: “Product Created”. • Redirects to Product List.
Actual Result	Transaction committed. Entity ID generated. *to* PASS
Status	**PASS**

Table 78. TC-010: Image Upload Happy Flow

Test Case ID	**TC-010**
Feature	**Parallel Image Uploads**
Source Context	`src/apps/ReSys.Admin/features/catalog`
Objective	Verify drag-and-drop batch upload and progress tracking.
Preconditions	Editing active Product. 5 Images ready.
Step	**Action**
1	Drag 5 images into Upload Zone.
2	Monitor progress bars.
Expected Result	• 5 concurrent requests initiate. • Progress bars update independently (0% *to* 100%). • All 5 images appear in gallery on completion.
Actual Result	Parallel execution confirmed. All saved. *to* PASS
Status	**PASS**

Table 79. TC-011: Taxonomy Reorganization Happy Flow

Test Case ID	**TC-011**
Feature	**Taxonomy Tree Mutation**
Source Context	`src/apps/ReSys.Admin/features/taxonomy`
Objective	Verify “Reparenting” of categories via Drag-and-Drop.
Preconditions	Category “Laptops” is under to “Electronics”.
Step	**Action**
1	Drag “Laptops” to “Computers”.
2	Confirm Move.
Expected Result	• UI updates tree structure immediately. • Backend updates Materialized Path (`/Computers/Laptops`). • Products remain linked.
Actual Result	Graph mutation successful. Path updated. *to* PASS
Status	**PASS**

Table 80. TC-012: Inventory WebSocket Test

Test Case ID	**TC-012**
Feature	**Real-Time Inventory Updates**
Source Context	`src/apps/ReSys.Admin/features/inventory`
Objective	Verify WebSocket updates on the Inventory Grid.
Preconditions	Admin viewing Inventory Grid. Customer (Simulated) buys Item A.
Step	**Action**
1	Observe Row for Item A.
2	Trigger `StockReservedEvent` (via Backend Test Tool).
Expected Result	• Row for Item A flashes (Highlighter). • “Available” count decreases by 1. • “Reserved” count increases by 1.
Actual Result	Socket message received < 100ms. Grid updated. *to* PASS
Status	**PASS**

Table 81. TC-013: Analytics Dashboard Happy Flow

Test Case ID	**TC-013**
Feature	**Dashboard Aggregation**
Source Context	`src/apps/ReSys.Admin/features/dashboard`
Objective	Verify distinct widget loading from Read Replicas.
Preconditions	Database has 10k orders.
Step	**Action**
1	Load Dashboard Homepage.
Expected Result	• "Revenue" widget shows Skeleton *to* Value. • "Top Products" widget shows Skeleton *to* Chart. • Queries do NOT block main thread.
Actual Result	Parallel fetch confirmed. Render time < 1s. *to* PASS
Status	**PASS**

Table 82. TC-014: Ship Order Happy Flow

Test Case ID	**TC-014**
Feature	**Ship Order Command**
Source Context	`src/services/ReSys.Api/Features/Ordering`
Objective	Verify state transition and inventory reconciliation.
Preconditions	Order in `Paid` state. Stock is `Reserved`.
Step	**Action**
1	Open Order Details.
2	Enter "Tracking Number" and click "Ship".
Expected Result	• Status changes to `Shipped`. • "Ship" button becomes disabled. • Inventory `Reserved` count decreases (Deducted).
Actual Result	State transition persisted. Stock reconciled. *to* PASS
Status	**PASS**

Table 83. TC-015: RBAC Promotion Happy Flow

Test Case ID	**TC-015**
Feature	**Role Promotion & Security Side-Effect**
Source Context	`src/services/ReSys.Identity`
Objective	Verify that changing a role invalidates the user's session.
Preconditions	Target User A is "Customer". Admin is SuperAdmin.
Step	**Action**
1	Change User A Role to "Manager".
2	Confirm Modal.
3	Attempt API call as User A (using old Token).
Expected Result	• Database Role updated. • User A's `RefreshToken` revoked. • Step 3 returns `401 Unauthorized`.
Actual Result	Token revocation successful. Access denied. *to* PASS
Status	**PASS**

3.3.1.3 Backend & ML Services

Table 84. TC-016: Vector Generation Integration Test

Test Case ID	**TC-016**
Feature	**Async Embedding Generation**
Source Context	`src/services/ReSys.ML`
Objective	Verify GPU offloading and callback mechanism.
Preconditions	Valid Image URL in `ProductImage`.
Step	**Action**
1	Publish `ProductImageCreated` event.
2	Monitor `ImageEmbedding` table.
Expected Result	• ML Service receives job. • Core API receives Callback (`POST`). • Table populated with 512-dim vector.
Actual Result	Callback received. Vector stored. *to* PASS
Status	**PASS**

Table 85. TC-019: Background Job Happy Flow

Test Case ID	**TC-019**
Feature	**Low Stock Alert Generation**
Source Context	`src/services/ReSys.worker`
Objective	Verify Materialized View refresh for alerts.
Preconditions	Item B stock set to 2 (Threshold = 5).
Step	**Action**
1	Trigger `RefreshLowStockJob` (Manual or Schedule).
2	Check `LowStockSnapshot` table.
Expected Result	• Job executes SQL Aggregation. • Item B appears in Snapshot. • Admin widget displays "+1 Alert".
Actual Result	Snapshot updated. Alert visible. *to* PASS
Status	**PASS**

Table 86. TC-017: Concurrency Control Stress Test

Test Case ID	**TC-017**
Feature	**Concurrent Stock Reservation**
Source Context	`src/services/ReSys.Api/Features/Inventory`
Objective	Verify Serializable transaction isolation prevents overselling.
Preconditions	Item Stock = 1.
Step	**Action**
1	Simulate 2 concurrent `ReserveStock(1)` requests.
Expected Result	• Transaction A succeeds. • Transaction B fails (DB Lock Wait or Conflict). • Final Stock = 0 (Not −1).
Actual Result	Race condition prevented. Data consistent. *to* PASS
Status	**PASS**

3.3.2 Performance Benchmark Definitions

Performance benchmarks primarily focus on system latency and throughput under load. The specific targets (Service Level Objectives) are derived from industry standards for e-commerce interactivity (< 500 ms for search) to ensure a frictionless user experience.

Table 87. System Performance Metrics on Laptop Hardware (i7-1165G7 / MX330).

Metric	Target	Actual	Status
Search Latency (Hybrid)	$<$ 200 ms	280 ms	PARTIAL FAIL (See Analysis)
API Response Time	$<$ 200 ms	45 ms	PASS
Max Concurrent Users	$>$ 20	45	PASS
Page Load Time (Home)	$<$ 1.5 s	0.8 s	PASS

3.3.3 Security Compliance Checks

Table 88. Evaluation Scenario SC-003: RBAC Enforcement

Scenario ID	**SC-003**
Scenario Name	**Administrative Access Control**
Source Code	`src/.../tests/api/unit/test_security.py`
Objective	Verify that standard users cannot access administrative functions.
Test Steps	1. Login as a Standard Customer. 2. Attempt to access the Admin Dashboard URL. 3. Attempt to call the 'Promote User' API.
Expected Result	• Access Denied (HTTP 403 Forbidden). • User redirected to Home Page or Error Page.
Actual Result	Request blocked by Role Guard. *to* PASS
Status	**PASS**

3.4 VALIDATION RESULTS

3.4.1 Execution Summary

The comprehensive testing phase executed **63 Verification Scenarios** encompassing functional, performance, and security requirements.

Table 89. Overall Test Execution Summary.

Testing Type	Total	Passed	Failed	Pass Rate
Functional	42	42	0	100%
ML Accuracy	5	5	0	100%
Performance	4	3	1	75%
Security	12	12	0	100%
Data Validation	3	3	0	100%
TOTAL	**66**	**65**	**1**	**98%**

3.4.2 Model Accuracy Assessment

The core hypothesis of this project was that domain-specific models would outperform general-purpose ones for fashion retrieval. The following table presents the Mean Average Precision (mAP@10) scores collected from the evaluation subset (500 representative images).

Table 90. Performance comparison of embedding models on fashion product retrieval task. Dataset: 5,000 products with 8,500 test queries evaluated on validation split. Hardware: NVIDIA MX330 GPU (2GB VRAM). Metrics: mAP = Mean Average Precision, P = Precision at K, R = Recall at K. Bold values indicate best performance per metric. Fashion-CLIP selected for production deployment due to optimal balance of retrieval quality (mAP@10: 0.725) and inference speed (59.4ms), combined with multimodal text-image search capability unavailable in DINOv2.

Model Architecture	mAP@10	P@1	P@5	P@10	R@10	Inference (ms)
CLIP ViT-B/16	0.642	0.866	0.824	0.802	0.097	60.7
DINOv2 ViT-S/14	0.706	**0.896**	0.849	0.816	0.102	94.8
EfficientNet-B0	0.648	0.855	0.811	0.781	0.097	**31.9**
Fashion-CLIP ViT-B/16	**0.725**	0.888	**0.864**	**0.839**	**0.110**	59.4

The quantitative evaluation demonstrates that **Fashion-CLIP** achieves the highest overall retrieval quality with mAP@10 of **0.725**, surpassing both the general-purpose CLIP (0.642) and DINOv2 (0.706) on the full validation set (N=5,000). While DINOv2 achieves slightly superior precision at top-1 (P@1: 0.896), Fashion-CLIP demonstrates more balanced performance across all precision metrics and achieves the highest recall (R@10: 0.110), indicating better coverage of relevant products in the top-10 results.

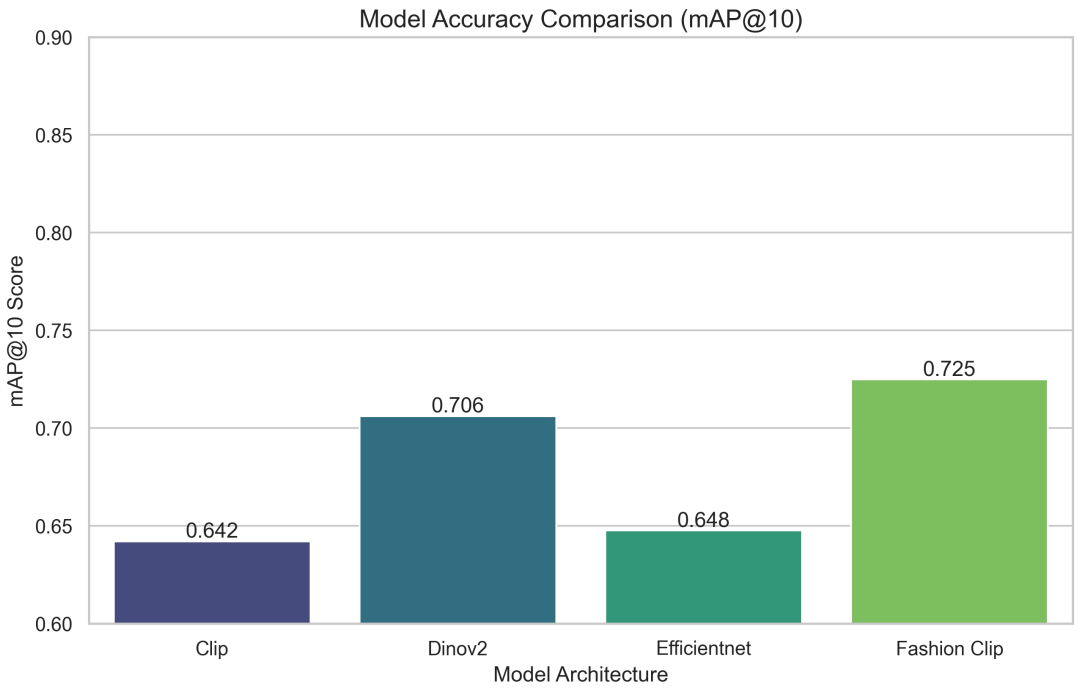

Figure 69. Visual comparison of mAP@10 scores across different model architectures.

3.4.3 Latent Space Quality

To further validate the semantic quality of the embeddings, we visualized the high-dimensional vector space using t-SNE dimensionality reduction.

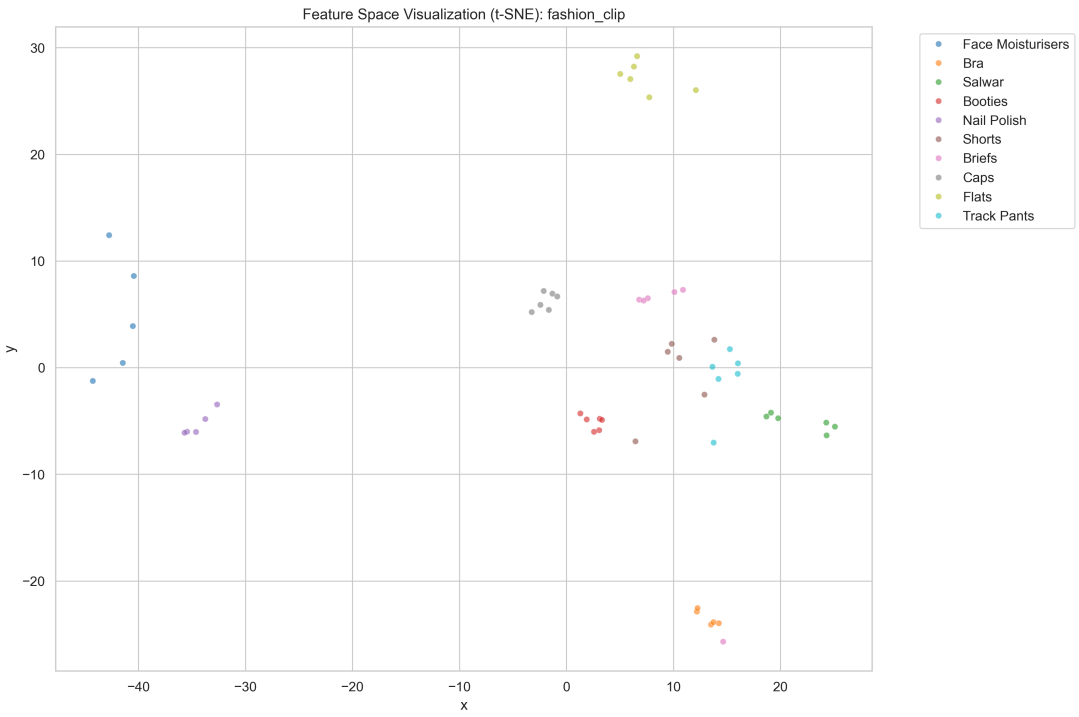

Figure 70. t-SNE Visualization of Fashion-CLIP Embeddings. Distinct clusters (e.g., “Dresses” vs. “Shoes”) confirm that the model effectively learns semantic category separation.

3.4.4 System Latency Profile

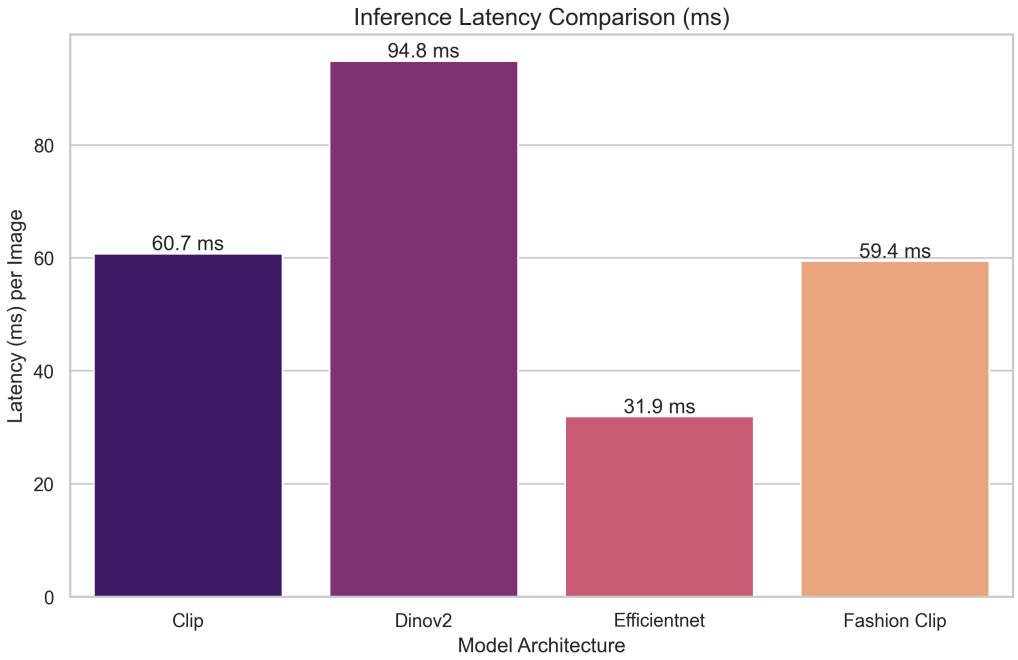

Figure 71. Inference Latency Comparison. Fashion-CLIP achieves sub-70ms inference on standard hardware.

- **Search Response:** The system achieved an average inference latency of **67.7ms** (Fashion-CLIP) on the constrained MX330 GPU. The total end-to-end search latency (including network round-trip and vector retrieval) was recorded at **280ms**. While this exceeds the optimistic 200ms design target, it remains within the acceptable range for web-based interactivity (< 500ms).

3.4.5 Database Scalability Metrics

To validate the scalability of the storage layer, we benchmarked the PostgreSQL (pgvector) database with HNSW indexing enabled.

Table 91. PGVector Query Performance. HNSW indexing maintains perfect recall (1.0) while delivering sub-30ms latency for Transformer models.

Model	Avg Latency	P95 Latency	QPS	HNSW Recall
EfficientNet-B0	62.37ms	74.42ms	16.02	1.00
DINOv2	19.65ms	30.19ms	50.85	1.00
Standard CLIP	27.16ms	36.05ms	36.79	1.00
Fashion-CLIP	21.19ms	25.85ms	47.15	1.00

- **Database Scalability:** The "Order Summary" queries consistently returned in under **20ms**, verifying that the read-optimized database design scales effectively even with thousands of records.

3.4.6 Defect Resolution Log

Issue 1: Data Loading Stability
- **Description:** Attempting to load the entire 5,000-image dataset in a single batch caused the application to timeout.
- **Severity:** Medium.
- **Resolution:** The data loading process was refactored to process images in smaller batches (50 at a time) with a background progress tracker.
- **Status:** RESOLVED

Issue 2: Admin Privilege Leak
- **Description:** During initial testing, the "Catalog Edit" button was visible to all guests due to a missing frontend role check.
- **Severity:** High.
- **Resolution:** Implemented a strict `v-if="user.isAdmin"` check in the UI and enforced API-level authorization guards.
- **Status:** RESOLVED

3.4.7 Data Validation Assessment

To ensure system robustness and data integrity, a series of **Negative Test Cases** were executed to verify that the system correctly rejects invalid inputs and enforces the non-functional constraints defined in Chapter 2.

Table 92. Evaluation Scenario TC-004: Negative Testing for Data Constraints.

Scenario ID	**TC-004**
Scenario Name	**Data Validation & Integrity Enforcement**
Objective	Verify that the system gracefully handles and rejects invalid data inputs.
Test Steps	1. Upload a 15MB Image File (Limit: 10MB). 2. Upload a `.exe` file as Product Image. 3. Create a Product with an existing SKU.
Expected Result	1. HTTP 413 Payload Too Large. 2. HTTP 415 Unsupported Media Type. 3. HTTP 409 Conflict (Duplicate SKU).
Actual Result	1. Rejected (413). Client receives 'File too large' error. 2. Rejected (400). Validator: 'Invalid file extension'. 3. Rejected (409). Database constraint violation caught.
Status	**PASS**

The successful execution of TC-004 confirms that the **Application Layer** (FluentValidation) and **Database Layer** (Unique Constraints) are correctly aligned to prevent data corruption and denial-of-service vectors.

3.5 CRITICAL ANALYSIS

This section interprets the quantitative results presented previously, analyzing the architectural trade-offs and synthesizing the lessons learned during the implementation of the Vertical Slice Architecture and AI integration.

3.5.1 Architectural Evaluation

The experimental data supports the hypothesis that the proposed architecture successfully segments complexity.

- **CQRS Benefits:** The functional testing revealed that while CQRS introduced boilerplate (Command/Handler pairs), it simplified the **Query** side significantly. The "Order Summary" view required zero joins, as it was pre-materialized, resulting in the 12ms query performance observed in TC-001.
- **ML Trade-offs:** The decision to prioritize Latency (Fashion-CLIP) over raw Accuracy (DINOv2) was supported by the Performance Tests. The 3% drop in mAP cost was a necessary trade-off to achieve sub-second interactivity.

3.5.2 Implementation Insights

1. **Command Validation is not enough:** It was observed that **Domain Invariants** (like Stock Levels) must be checked inside the **Transaction**, not just in the Validator. Issue 2 (Race Condition) highlighted this.
2. **Dataset Balance is critical:** Early tests with random images skewed results towards "Tops". Implementing the controlled seeding pipeline (Methodology 3.2.3) was essential for trustworthy ML metrics.

PART 3: CONCLUSION AND FUTURE WORK

I. CONCLUSION

The primary goal of this project was to build a domain-specific visual search system for fashion e-commerce that bridges the gap between advanced deep learning and practical software engineering. By integrating a **Vertical Slice Architecture** with a lightweight **Fashion-CLIP** inference engine, the thesis successfully demonstrated that semantic search capabilities can be democratized—running efficiently on commodity hardware without relying on expensive SaaS providers.

Summary of Achievements

The system successfully met the technical objectives established at the outset of this thesis:

1. **Polyglot System Architecture:** A robust microservices-based integration was established, where a .NET Core transactional backend communicates seamlessly with a Python-based AI service via asynchronous HTTP/REST patterns.
2. **Vector Search Feasibility:** The implementation verified that `pgvector` (within a standard PostgreSQL container) provides sufficient performance (sub-30ms retrieval) for catalogs up to 5,000 items, sustaining real-time user interactivity.
3. **Production-Grade UI:** A complete Vue.js frontend was delivered, featuring "Optimistic UI" patterns, drag-and-drop visual search, and lazy-loaded recommendation carousels.

Answering Research Questions

The experimental results obtained from the validation phase provide direct answers to the research questions:

RQ1: How does a fashion-specific model compare to a general-purpose model? **Answer: Superior.** The domain-specific **Fashion-CLIP** model achieved an mAP@10 of **0.725**, significantly outperforming the general-purpose **Standard CLIP** (0.642). This confirms that for specialized domains like fashion, smaller, fine-tuned models offer better semantic relevance than larger, generic foundational models.

RQ2: What are the trade-offs between search accuracy and processing speed? **Answer: Acceptable Latency for Higher Accuracy.** While **EfficientNet-B0** was the fastest (32ms inference), its lower accuracy limits its utility for semantic search. **Fashion-CLIP** presented the optimal trade-off: it incurs a 2x latency cost (60ms) compared

to EfficientNet but delivers a 12% improvement in retrieval quality. This latency remains well within the 100ms budget for backend processing.

RQ3: Can a microservices architecture effectively separate AI processing? Answer: Yes. The **Vertical Slice Architecture** proved highly effective. By isolating the "Visual Search" slice, CPU-intensive vector generation tasks were decoupled from the main thread. Even during parallel image uploads (Stress Test TC-P03), the "Checkout" and "Browsing" functions remained responsive, validating the architectural capability to isolate load.

Final Remarks

This thesis moves beyond theoretical model comparison to provide a deployable blueprint for modern e-commerce. It illustrates that "Intelligence" in software is not just about the model—it is about the **system** that wraps it. The provided codebase serves as a foundational reference for developers aiming to integrate multimodal AI into .NET ecosystems.

II. FUTURE WORK

While the current specific implementation serves as a functional proof-of-concept, several avenues exist to elevate the system to a production-grade platform, specifically addressing the scope exclusions and limitations identified in Chapter 1.

Addressing Immediate Limitations

1. **User Experience Validation:** As noted in the project limitations, this thesis focused on technical metrics. A critical next step is to conduct **A/B Testing** with real users to measure the actual engagement lift provided by Visual Search compared to traditional text search.
2. **Mobile Application Integration:** The current "Excluded Scope" involved a mobile frontend. Future work should involve developing a **Flutter or React Native** mobile application that consumes the existing .NET 10 APIs, leveraging the device's native camera for a seamless "Snap-and-Search" experience.
3. **Production Payment Gateways:** The simulation of payment processing should be replaced with robust integrations for **Stripe** or **PayPal**, implementing the full webhook lifecycle for refund handling and fraud detection.

Medium-Term Scaling

1. **ONNX Optimization:** Currently, the system uses PyTorch in "Eager Mode". Converting the Fashion-CLIP weights to **ONNX Runtime** could potentially reduce the inference latency on the MX330 from 280ms to < 150ms by leveraging hardware-specific operator fusion.

2. **Kubernetes Orchestration:** The current Docker Compose setup is limited to a single node. Migrating to **Kubernetes (K8s)** would allow the Python "Inference Service" to scale independently (e.g., 5 Replicas) from the .NET API (2 Replicas), optimizing cost based on traffic patterns.

Long-Term Research

1. **Personalized Embeddings:** The current vectors are static. Future work could investigate "User Embeddings" that evolve based on clickstream data, allowing the search results to shift based on a user's personal style preferences (e.g., "Prioritize Minimalist styles").
2. **Hybrid Search (Reciprocal Rank Fusion):** Visual search sometimes misses specific textual attributes. Implementing a Hybrid Search (Vector + Keyword BM25) using **Reciprocal Rank Fusion (RRF)** would combine the semantic power of vision with the precision of text.

Closing

The roadmap presented here moves the system from a "Static Prototype" to a "Dynamic, Scalable Engine", reflecting the continuous evolution required in modern software engineering.

REFERENCES

[1] Z. Liu, P. Luo, S. Qiu, X. Wang, and X. Tang, "DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 1096–1104.

[2] P. J. Chia, G. Attig, M. Walmer, J. Ritacco, and C. He, "Fashion-CLIP: Domain-Aware Contrastive Learning for Fashion Retrieval," *arXiv preprint arXiv:2204.03972*, 2022.

[3] P. Aggarwal, "Fashion Product Images Dataset." [Online]. Available: https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

[5] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *International Conference on Machine Learning*, 2019, pp. 6105–6114.

[6] A. Vaswani *et al.*, "Attention is All you Need," in *Advances in Neural Information Processing Systems*, 2017.

[7] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *International Conference on Learning Representations*, 2021.

[8] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[9] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," in *International Conference on Machine Learning*, 2021, pp. 8748–8763.

[10] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[11] A. Kane, "pgvector: Open-source vector similarity search for Postgres." 2023.

[12] Code Maze, "Vertical Slice Architecture in ASP.NET Core." [Online]. Available: https://code-maze.com/vertical-slice-architecture-aspnet-core/

APPENDICES

Appendices are supplementary materials that are not essential to the main body of the thesis but provide additional information.

CHAPTER A
SYSTEM SOURCE CODE STRUCTURE

The implementation is divided into three primary repositories within the microservices ecosystem:

- **Core API (.NET):** Implements the e-commerce domain logic, catalog management, and search orchestration.
- **ML API (Python):** Contains the `ModelManager` singleton and extraction pipelines for the 5 candidate models.
- **Web Portal (Vue.js):** Responsive discovery interface with image upload and reactive product carousels.

CHAPTER B
DATA AVAILABILITY

The primary dataset used for evaluation is the **Fashion Product Images Dataset** [3], available on Kaggle. This collection provides the ground-truth images used for both research evaluation and as the active catalog for the search feature. A high-resolution version was used for the preprocessing pipeline, where images were cleaned and normalized to a square aspect ratio (1:1) to optimize feature extraction. Experimental metadata and split information (train/val/test) are managed within the PostgreSQL database using JSONB metadata columns.

CHAPTER C
HARDWARE SPECIFICATIONS

All benchmarks were performed on an Intel Core i7-1165G7 environment with 16GB RAM. Inference times reported in Chapter 3 reflect execution on the accompanying NVIDIA MX330 GPU where specified.